On the Adaptive Security of Key-Unique Threshold Signatures

Michele Ciampi[1], Elizabeth Crites[2], Chelsea Komlo[3], and Mary Maller[4]

[1] University of Edinburgh
[2] Parity Technologies
[3] University of Waterloo & NEAR One
[4] Inversed Tech

Abstract. In this work, we investigate the security assumptions required to prove the adaptive security of threshold signatures. Adaptive security is a strong notion of security that allows an adversary to corrupt parties at any point during the execution of the protocol, and is of practical interest due to recent standardization efforts for threshold schemes. Towards this end, we give two different impossibility results.

We begin by formalizing the notion of a *key-unique* threshold signature scheme, where public keys have a unique correspondence to secret keys and there is an efficient algorithm for checking that public keys are well-formed. Key-uniqueness occurs in many threshold schemes that are compatible with standard, single-party signatures used in practice, such as BLS, ECDSA, and Schnorr signatures.

Our first impossibility result demonstrates that it is impossible to prove the adaptive security of *any* key-unique threshold signature scheme under *any* non-interactive computational or decisional assumption for a broad class of reductions, in the range $\lfloor t/\ell \rfloor < t_c \leq t$, where $t+1$ is the threshold out of n parties, t_c is the number of corrupted parties (polynomially related with the security parameter), and ℓ is a constant. Such assumptions include, but are not limited to, the discrete logarithm (DL), recently introduced circular DL (CDL), computational Diffie-Hellman (CDH), decisional DH (DDH), and q-Strong DH (q-SDH) assumptions.

Our second impossibility result applies specifically to key-unique threshold Schnorr signatures, currently an active area of research. We demonstrate that, even under the *interactive* computational assumptions one-more DL (OMDL), algebraic OMDL (AOMDL) and algebraic one-more CDH (AOMCDH), it is impossible to prove adaptive security for $\lfloor t/2 \rfloor < t_c \leq t$ in the ROM for a natural class of rewinding reductions. Taken together, our results underscore the difficulty of achieving adaptive security for key-unique threshold signatures, but at the same time, they open a new line of research, by indicating assumptions and properties to aim for when constructing adaptively secure threshold schemes, or informing new attacks.

michele.ciampi@ed.ac.uk
elizabeth@parity.io
ckomlo@uwaterloo.ca
mary@inversed.tech

Table of Contents

1 Introduction

A threshold signature scheme allows multiple parties to jointly sign a message, where the signature is valid if a sufficient number of signers participate. In a $(t+1, n)$-threshold signature scheme, the secret signing key is distributed among a set of n parties, where a threshold $t+1$ of them are required to successfully issue a signature. Threshold signature schemes are attractive for their failure resilience properties; they remain unforgeable even if some participants fall under adversarial control.

Threshold signatures have attracted significant interest in recent years [58, 32, 69, 17, 72, 80, 43, 38], driven by their use in securing digital wallets as well as an upcoming standardization effort by NIST [30]. A main criteria of interest in the NIST call for threshold schemes is adaptive security. Adaptive security is a stronger notion of security that allows an adversary to corrupt parties *at any point during the protocol*, based on its view of the protocol execution so far. This is in contrast to static security, which requires the adversary to declare at the beginning of the protocol which parties are corrupt. This model places an artificial restriction on the adversary's capabilities: in reality, malicious actors may observe a system before targeting specific parties. Static corruption is easier to capture in proofs of security, and indeed most threshold signature schemes in the literature are proven secure in this model [17, 72, 80, 38, 42, 37].

Adaptive Security with $\binom{n}{t_c}$ Tightness Loss. There are generic methods for transforming a statically secure scheme into an adaptively secure one [33]. Given an adversary that is allowed to corrupt up to t_c parties, the reduction could guess the set of t_c to-be-corrupted parties at the beginning of the experiment. If the adversary adaptively corrupts a party that is not in this set, then the reduction aborts. But since the reduction will guess correctly with probability $1/\binom{n}{t_c}$, such a reduction has $\binom{n}{t_c}$ tightness loss.

While this is acceptable for small numbers of parties, the NIST call for threshold schemes classifies schemes based on different numbers of parties and corruption profiles, with $n = 1024$ and large corruption tolerance t_c as the highest achievable categories. Thus, investigating under what conditions we can demonstrate proofs of adaptive security with less than $\binom{n}{t_c}$ tightness loss for threshold signatures is of both theoretical and practical value.

Challenges of Adaptive Security. The focus of this work is to investigate the assumptions required to prove adaptive security of threshold signatures, with improved tightness loss over the generic guessing technique [33]. A host of protocols have been designed recently with this goal. However, many have undesirable tradeoffs, such as incompatibility with widely used single-party signature schemes [71, 10, 35], limiting adversaries to controlling fewer than $t < n/2$ parties [8, 1], prohibitive performance overhead [64, 74, 1], stronger security assumptions [43, 41], or requiring secure erasure of secret state [33, 75][5]. However, there are recent exceptions, such as the BLS-compatible scheme in [46],

[5] Secure erasure, especially on demand, is difficult to guarantee in practice.

which is adaptively secure for up to t corruptions under the co-computational Diffie-Hellman assumption (co-CDH) and DDH in the ROM. Thus, in this work, we address the following natural question:

Under what conditions is it possible to prove adaptive security of threshold signatures, and what security assumptions are inherently required to achieve it?

Key-Unique Threshold Signatures. We initiate our study by introducing the notion of *key-unique* threshold signatures. As we will see, this classification is both natural and allows us to prove strong results on adaptive security.

Our classification pertains to how key generation is performed. In a threshold signature scheme, the key generation algorithm or protocol outputs a threshold public key PK representing all n potential signers. At a minimum, $t+1$ signers must collaborate in a signing protocol to produce a signature that verifies under PK. We define key generation to output public key shares in addition to PK and secret key shares: $(\mathsf{PK}, (\mathsf{PK}_1, \ldots, \mathsf{PK}_n), (\mathsf{sk}_1, \ldots, \mathsf{sk}_n)) \leftarrow \mathsf{KeyGen}(1^\kappa)$, for security parameter κ. We then define a *key-unique* threshold signature scheme to be one in which each public key has a unique correspondence to its respective secret key, and where an efficient verification algorithm $\mathsf{VerifyPKs}$ exists to check that public keys are well-formed. In this paper, we show that this property of key-uniqueness fundamentally changes the conditions under which threshold signature schemes can be proven adaptively secure. We formally define key-uniqueness in Definition 4 but capture the intuition in the following.

Definition. (Informal.) *A threshold signature scheme* $\mathsf{KeyUTS}_{\mathcal{SS},f,\mathsf{VerifyPKs}}$ *with respect to a secret-sharing scheme* $\mathcal{SS}$ *is* **key unique** *if for each share* $\mathsf{sk}_i \in \{\mathsf{sk}_j\}_{j=1}^n$ *of a secret key* sk*, the corresponding public key share* PK_i *is generated via* $f : (i, \mathsf{sk}_i) \mapsto \mathsf{PK}_i$*, where* $f(j, \cdot)$ *is an injective map for each index* $j \in \{0, \ldots, n\}$*, the threshold public key is generated via* $f : (0, \mathsf{sk}) \mapsto \mathsf{PK}$*, and an efficient algorithm* $\mathsf{VerifyPKs}(\mathsf{PK}, \{\mathsf{PK}_j\}_{j=1}^n) \to \{0, 1\}$ *exists for the scheme, which outputs* 1 *if an honest execution of* KeyGen *returns* $(\mathsf{PK}, \{\mathsf{PK}_j\}_{j=1}^n)$*, indicating that the public key shares are valid with respect to* PK*. Moreover, there exists an efficient algorithm* $\mathsf{Sign}(\mathsf{sk}, m) \to \sigma$ *that takes as input the secret key* sk *and a message* m *and returns a signature* σ*.*

For example, a scheme with Shamir secret-shared keys $\mathsf{PK} = g^{\mathsf{sk}}$ and $\mathsf{PK}_i = g^{\mathsf{sk}_i}$ for all $i \in \{1, \ldots, n\}$ is key unique. In contrast, a scheme where public key shares are Pedersen commitments to secret key shares, i.e., $\mathsf{PK}_i = g^{\mathsf{sk}_i} h^{\rho_i}$ for random ρ_i, is not key unique. Interestingly, the trivial threshold Schnorr signature scheme, which has perfectly binding public keys $\mathsf{PK}_i = g^{\mathsf{sk}_i}$ for all $i \in \{1, \ldots, n\}$ but no secret-sharing scheme, is not key unique. We discuss this point more in Section 1.1. More examples of key-unique and non-key-unique schemes are given in Section 3.1 and Appendix B.[6]

[6] [71, 75, 10, 46, 35, 6, 5, 60] use non-perfectly binding commitments; [66, 36, 4, 12] hide the public key shares; [1, 2, 86, 46] perform non-key-unique key resharing.

One might wonder if key-uniqueness is an artificial distinction. However, key-uniqueness is exhibited by an extensive range of concurrently-secure, state-of-the-art threshold signature schemes, as shown in Table 1.

Why key-uniqueness, practically? Key-uniqueness is often leveraged to efficiently detect misbehaving parties via simple partial signature verification. For example, FROST [69, 17] can identify a misbehaving party with public key share PK_i by checking if their signature share z_i is valid with respect to the single-party Schnorr verification algorithm,[7] namely $g^{z_i} = R_i \cdot \mathsf{PK}_i^{c\lambda_i}$ with Lagrange coefficient λ_i and overall nonce commitment R_i. If this check fails, the party can be removed and the protocol re-run with a new signing set [80]. In contrast, the threshold BLS scheme of [46] mentioned above has (non-key-unique) public key shares of the form $\mathsf{PK}_i = g^{s(i)}h^{r(i)}v^{u(i)}$, where s, r, u are polynomials of degree t, allowing equivocation of the secret keys similar to Pedersen commitments. In order to verify misbehaving parties, a non-interactive zero-knowledge (NIZK) proof of well-formedness must be appended to each party's signature share.

Impossibility Results for Adaptive Security of Key-Unique Threshold Signatures. In this work, we provide two different impossibility results regarding the adaptive security of key-unique threshold signature schemes. We employ the metareduction technique [27, 50, 52] (Section 2.3), constructing a "reduction against a reduction" to rule out different classes of reductions for key-unique schemes. We provide a technical overview of our techniques in Section 1.1 and capture how our results relate to a range of existing schemes in Table 1. We discuss why each scheme is key unique or not in Appendix B.

Impossibility Result 1. Our first result demonstrates that it is impossible to prove the full adaptive security (i.e., $t_c = t$, as in static security) of any key-unique threshold signature scheme under *any* non-interactive computational (search) or decisional assumption using a fully black-box reduction (Definition 2; Theorem 1). Our result captures both straight-line and rewinding reductions, as long as the reduction runs the adversary on the same public parameters and public keys in each instance. (We discuss why this constraint is reasonable in Section 1.1.) Moreover, our metareduction interacts with the reduction in a black-box manner (hence making no assumptions about the reduction's internal state), even allowing the reduction to freely program any random oracles for the adversary.

Generalizing Impossibility Result 1. We then prove a generalization of this result to any corruption threshold $t_c \leq t$ (Theorems 2 and 3). This result quantifies the extent to which our metareduction from Impossibility Result 1 (for the same class of reductions) succeeds for lower corruption profiles. In particular, our result implies that no key-unique scheme can be proven adaptive-secure certain corruption thresholds $\lfloor t/\ell \rfloor < t_c \leq t$ for any constant $\ell \in \mathbb{N}$ (and n and t polynomially related with the security parameter).

Non-interactive computational and decisional assumptions (Appendix C) include, but are not limited to, the discrete logarithm (DL), recently introduced

[7] Recall that a Schnorr signature (R, z) on message m verifies as $g^z = R \cdot \mathsf{PK}^c$, where $c = \mathsf{H}(R, \mathsf{PK}, m)$ (Definition 8).

Scheme	Static Security	Adaptive Security	Results Apply
Threshold BLS (I) [22]	GDH	✗	1
[8]	-	OMDL+AGM (full)	1
Threshold BLS (II,III) [46]	co-CDH	co-CDH+DDH (full)	✗*
Libert et al. [71]	-	SXDH† (full)	✗*
Threshold Waters [86]	-	CDH† (full)	✗*
Threshold RSA [2]	RSA†	RSA† (full)	✗*
HBTS-Mask [36]	-	DL (full)	✗*
Threshold ECDSA [58]	Various†	✗	1
Threshold ECDSA [32]	Various	✗	1
Threshold BBS+ [48]	Various†	✗	1
Twinkle_1 [10]	-	AOMCDH (full)	1
Twinkle_2 [10]	-	DDH (full)	✗*
Dazzle/-T [35]	-	DDH (full)	✗*
FROST/2/3	(A)OMDL	AOMDL+LDVR+AGM (full)	1,2
[44, 17, 80, 38, 41, 84]		AOMDL ($t/2$)	
FROST-Mask [36]	-	AOMDL (full)	✗*
ms-FROST [4]	-	AOMDL+AGM (full)	✗*
FaFROST [12]	-	AOMDL+AGM (full)	✗*
Sparkle+ [43]	DL	AOMDL ($t/2$)	1,2
[37]	CDL	✗	1,2
Lindell [72]	Schnorr, $\mathcal{F}_{\mathsf{zk}}$	✗	1,2
Classic S. [75]	DL	✗	1,2
Zero S. [75]	DL	DL (full, erasures)	✗*
Crackle & Snap [66]	-	DL (full)	✗*
Glacius [6]	-	DDH (full)	✗*
Gargos [5]	-	DDH (full)	✗*
GCRS [60]	-	DDH (full)	✗*
Abe-Fehr [1]	-	Schnorr+DDH (full)	✗*
Stinson-Strobl [83]	Schnorr	✗	1,2
ROAST [80]	OMDL	✗	1,2
SPRINT [20]	DL	✗	1,2
HARTS [9]	-	-	1,2
GKMN [57]	n-party Schnorr, $\mathcal{F}_{\mathsf{com-zk}}^{R_{\mathrm{DL}}}$	✗	1,2
Arctic [68]	DL	✗	1,2

Table 1: Threshold signature schemes. The bottom half are compatible with Schnorr signatures and assume the hardness of DL. Results Apply with a 1 means that the scheme is key unique and our first main result applies (Theorems 1 to 3). Results Apply with a 2 means that the scheme is key unique and our second main result applies (Theorems 4, 5 and 6). Full means full adaptive security (i.e., $t_c = t$), and $t/2$ means adaptive security for up to $t_c = t/2$ corruptions. ✗ denotes no adaptive security proof, or with $\binom{n}{t_c}$ loss. Erasures means that adaptive security relies on erasure of secret state. ✗* indicates the scheme is not key unique. All but those marked with † assume the ROM.

circular DL (CDL) [37], computational Diffie-Hellman (CDH), decisional DH and q-Strong DH (q-SDH) assumptions, which underpin many threshold signature schemes. The fact that we can rule out adaptive security under these assumptions is of particular interest given the motivation to prove the (static) security of threshold signatures with respect to these minimal assumptions, such as DL [43, 72, 75, 20, 68], in light of the NIST call. Additionally, the q-SDH assumption underpins the security of BBS signatures [3, 25, 85] and their thresholdization [48], which are also the subject of an RFC [73] and were recently proposed for use in the new digital identity wallet for European citizens [16]. The CDL assumption enables a tight security reduction for the threshold Schnorr scheme Sparkle+ [43] against static corruption [37]. The question of adaptive security, arguably the main novelty of Sparkle+, under CDL is left as an open problem, which we resolve in this work.

Impossibility Result 2. Our second impossibility result applies to key-unique threshold signatures that are compatible with standard, single-party Schnorr signature verification (including EdDSA). We demonstrate that even if we allow the *interactive* computational assumption one-more discrete logarithm (OMDL),[8] it is impossible to define a fully black-box adaptive reduction to OMDL, for $\lfloor t/2 \rfloor < t_c \leq t$ in the ROM, where the reduction rewinds the adversary and changes the coins or programs the random oracle on at least one point (Theorem 4).[9]

Similar to our first impossibility result, we require the reduction to run the adversary on consistent public parameters and public keys. Our second metareduction, like our first metareduction, interacts with the reduction in a black-box manner and applies even to reductions in the secure erasure model.

A reduction that programs the random oracle is natural strategy of many existing Schnorr-based schemes in the ROM. Indeed, the classic reduction for single-party Schnorr signatures rewinds under the DL assumption in the programmable ROM [78]. Threshold Schnorr signature schemes have been proven statically and adaptively secure using rewinding in the programmable ROM under the DL assumption [43, 20, 66], the DDH assumption [6, 5, 60], and the (algebraic) OMDL assumption [44, 17, 80, 38, 36, 41, 84].

Extending Impossibility Result 2. Finally, we extend our second impossibility result for key-unique threshold Schnorr signature schemes to *falsifiable* interactive assumptions. The *algebraic* OMDL (AOMDL) assumption[10] is of interest because it is a falsifiable variant of classic OMDL. We show that it is impossible to have a fully black-box adaptive reduction for $\lfloor t/2 \rfloor < t_c \leq t$ to AOMDL for the same class

[8] The OMDL assumption states: Given $q+1$ challenge group elements $(X_0, \ldots, X_q)$ (in addition to the generator g) and q-time access to a discrete logarithm solution oracle, compute x_i such that $X_i = g^{x_i}$ for all $i \in \{0, \ldots, q\}$. (See Appendix G.)

[9] Note that a reduction that does not fork is a straight-line reduction.

[10] The AOMDL assumption is the same as OMDL, except any discrete logarithm solution oracle query must include an algebraic representation in terms of the challenge group elements and generator g. This makes the oracle polynomial time and therefore AOMDL a falsifiable assumption. (See Appendix G.)

of rewinding reductions considered in our impossibility result for OMDL. However, our AOMDL metareduction itself is not fully black box, as we additionally require that the reduction output an algebraic representation of the threshold public key with respect to its AOMDL challenges. We expand on this in Appendix H, and show the same holds for the algebraic one-more CDH (AOMCDH) assumption (Appendix G) used to prove the security of Twinkle_1 [10] . Prior impossibility results [27, 77], including those for threshold BLS [8], have required a fully algebraic reduction, which must provide an algebraic representation of *every* group element that it outputs.

Taken together, our results underscore the limitations of achieving proofs of adaptive security for key-unique threshold signature schemes. On the positive side, we believe our results are also constructive, by indicating assumptions and properties to target for constructing adaptively secure threshold schemes in the future (explored further in Section 1.1 and Appendix B.2). We see that many recent, prominent papers on threshold signature constructions achieving adaptive security utilize combinations of techniques outlined in this paper [46, 10, 6, 66, 35, 5, 35, 60, 4, 12].

1.1 Technical Overview

In this work, we demonstrate impossibility results against a broad class of reductions using the metareduction technique [27, 50, 52]. A metareduction is similar to a security reduction, but eliminates the existence of a successful reduction instead of an adversary.

In more detail, a standard security reduction $\mathcal{B}$ uses an adversary $\mathcal{A}$, which is trying to break the security of a scheme (and does not need to be efficient), as a subroutine to try to break some problem Δ, and simulates the scheme to $\mathcal{A}$. If Δ is hard and $\mathcal{B}$ is efficient, then no efficient $\mathcal{A}$ can exist. Similarly, a metareduction $\mathcal{M}$ uses $\mathcal{B}$, which is trying to break Δ, as a subroutine to try to break some problem Δ' (possibly equal to Δ), and simulates $\mathcal{A}$ as well as the interaction with Δ to $\mathcal{B}$. If Δ' is hard and $\mathcal{M}$ is efficient, then no efficient $\mathcal{B}$ can exist. A more formal description of metareductions is given in Section 2.3.

Impossibility Result 1. We now give a high-level overview of how our first metareduction for full (i.e., $t_c = t$) adaptive security works. The generalization to lower corruption profiles $\lfloor t/\ell \rfloor < t_c \leq t$ is more involved due to combinatorial arguments about corruption sets.

Given a reduction $\mathcal{B}$ (as defined in the previous section, and formally in Theorem 1), we construct an efficient metareduction $\mathcal{M}$ against a non-interactive assumption NIA (defined formally in Appendix C) as follows.

First, we define one particular (inefficient) adversary $\mathcal{A}$ against the adaptive security of a key-unique threshold signature scheme, which $\mathcal{M}$ will efficiently simulate to $\mathcal{B}$. $\mathcal{A}$ is executed on coins, public parameters, and public keys given by $\mathcal{B}$. $\mathcal{A}$ then randomly selects a set of $t_c = t$ parties to corrupt, using a random function evaluated on the above inputs given by $\mathcal{B}$. $\mathcal{A}$ queries its corruption oracle $\mathcal{O}^{\mathsf{Corrupt}}$ in the adaptive unforgeability game (Fig. 2) to $\mathcal{B}$ to obtain the secret

keys of the parties in this set, and brute-forces the secret key of an arbitrary $t+1^{st}$ party. $\mathcal{A}$ then reconstructs the secret key sk and runs $\mathsf{Sign}(\mathsf{sk}, m^*)$ (defined above) on an arbitrary message m^* to produce a valid forgery.

We then show that $\mathcal{M}$ can simulate $\mathcal{A}$ efficiently. To simulate $\mathcal{A}$, $\mathcal{M}$ must produce a valid forgery at the end of its execution of $\mathcal{B}$ with the ability to corrupt only $t_c = t$ parties. To do so, $\mathcal{M}$ initializes a second execution of $\mathcal{B}$. Importantly, both executions, $\mathcal{B}_1(\mathsf{NIAChal}; \rho)$ and $\mathcal{B}_2(\mathsf{NIAChal}; \rho)$, are run on the same challenge $\mathsf{NIAChal}$ (exactly $\mathcal{M}$'s challenge $\mathsf{NIAChal}$) and the same random coins ρ. Thus, both instances of $\mathcal{B}$ will generate the same public parameters and public keys $(\mathsf{PK}, \{\mathsf{PK}_i\}_{i=1}^n)$ (and hence the same corresponding secret keys, by key-uniqueness), which are provided to the adversary. $\mathcal{M}$ then selects two corruption sets, cor_1 and cor_2, each of size t, using pseudorandom functions,[11] F_1 and F_2, evaluated on the coins, public parameters, and public keys given by $\mathcal{B}_1$ and $\mathcal{B}_2$, respectively. $\mathcal{M}$ queries $\mathcal{O}^{\mathsf{Corrupt}}$ to $\mathcal{B}_1$ for the parties in cor_1, and queries $\mathcal{O}^{\mathsf{Corrupt}}$ to $\mathcal{B}_2$ for the parties in cor_2. If $|\mathsf{cor}_1 \cup \mathsf{cor}_2| < t+1$, $\mathcal{M}$ aborts. This bad event occurs with probability $1/\binom{n}{t}$ (i.e., when $\mathsf{cor}_1 = \mathsf{cor}_2$), giving rise to the bound in Theorem 1. Else, $\mathcal{M}$ learns at least $t+1$ distinct secret key shares, and so can solve for sk and run $\mathsf{Sign}(\mathsf{sk}, m^*)$ to output a valid forgery. Since $\mathcal{B}_1$ and $\mathcal{B}_2$ output the same public keys and the scheme is key unique, cor_1 and cor_2 can indeed be combined as valid inputs to the secret-sharing recovery algorithm.

After obtaining sk, $\mathcal{M}$ terminates its execution with $\mathcal{B}_2$ and completes its execution with $\mathcal{B}_1$. If $\mathcal{B}_1$ fails and outputs $\perp$, then $\mathcal{M}$ similarly fails. However, when $\mathcal{B}_1$ outputs a solution NIASol for the challenge $\mathsf{NIAChal}$, $\mathcal{M}$ outputs it as its own solution for the challenge $\mathsf{NIAChal}$. The full proof is given in Section 4.

Technical Challenges and Insights. While the overall structure of our first metareduction may seem relatively simple, the correct formulation presented a number of surprising challenges and insights, as we now discuss.

The Requirement of Key-Uniqueness. Let us discuss how key-uniqueness plays an integral role in our results. First, observe that by the correctness of a threshold signature scheme defined with respect to a secret-sharing scheme $\mathcal{SS} = (\mathcal{SS}.\mathsf{Setup}, \mathcal{SS}.\mathsf{Share}, \mathcal{SS}.\mathsf{Recover})$ (Definition 1), when given $t+1$ secret key shares $\{\mathsf{sk}_i\}_{i\in\mathcal{S}}, |\mathcal{S}| = t+1$, associated with (valid) public key shares $\{\mathsf{PK}_i\}_{i\in\mathcal{S}}$, the key recovery algorithm $\mathcal{SS}.\mathsf{Recover}(t+1, \{(i, \mathsf{sk}_i)\}_{i\in\mathcal{S}})$ will return the secret-shared secret key sk. If the scheme is key unique, then by the injectivity of the map f (defined informally above and formally in Definition 4), one can recover the same secret key sk given the pre-images of $\{\mathsf{PK}_i\}_{i\in\mathcal{S}}$. Our metareductions leverage this property of key-uniqueness to guarantee that ℓ sets of corrupted parties' secret key shares, each of size $t_c > \lfloor t/\ell \rfloor$, reconstruct the true secret key sk. Further intuition is provided in Remark 2.

The reason for reconstructing sk at all is not obvious. The key insight is that the distribution of forgeries cannot depend on the signing set, which is guaranteed by secret key reconstruction. Indeed, consider the trivial threshold Schnorr signature scheme, where each party P_i generates a key-pair $(\mathsf{sk}_i, \mathsf{PK}_i = g^{\mathsf{sk}_i})$, the threshold public key is $\mathsf{PK} = \{\mathsf{PK}_1, \ldots, \mathsf{PK}_n\}$, and a threshold signature

[11] Since $\mathcal{M}$ must be efficient, it uses a PRF instead of a random function.

consists of individual signatures from at least $t+1$ parties.[12] Note that public keys are perfectly binding commitments to secret keys, but there is no secret reconstruction. Now, consider a reduction $\mathcal{B}$ that replaces the public key PK_{i^*} for some party P_{i^*} with a DL challenge and simulates signatures under PK_{i^*} for P_{i^*} in the usual way for Schnorr signatures by programming the random oracle. Only forgeries containing a signature from P_{i^*} allow $\mathcal{B}$ to break DL. We argue that $\mathcal{B}$ wins its DL game with probability at least $\epsilon(t+1-t_c)/n$,[13] where ϵ is $\mathcal{A}$'s probability of producing a successful forgery. In the worst case, $\mathcal{A}$'s forgery set includes all t_c corrupted parties. In this case, there are $t+1-t_c$ honest parties in the forgery set. So, the probability that P_{i^*} remains honest and is in this set is $(t+1-t_c)/n$. Thus, interestingly, the trivial scheme is adaptively secure under a non-interactive computational assumption.

This example seems to highlight that, in order for our impossibility results to hold, the threshold scheme needs to somehow hide the identities of the signers. It might therefore seem reasonable to think that our impossibility results apply to all key-unique schemes that achieve this. However, this is not the case. The reason is that we need to hide the identities of the signers *from the reduction*, which could potentially extract the identities by observing/programming the random oracle, or by rewinding the signers.

To elaborate more on this point, consider how a notion of security that hides the identities of the signers would normally be defined. We could define a security game with a distinguisher whose goal is, upon receiving the aggregated signature generated via the threshold scheme, to distinguish which parties participated in the generation of such a signature. We would also normally assume that such a distinguisher cannot rewind the signers or control the random oracle.

The problem is that, in our impossibility results, we need the identities of the signers to remain hidden against a much stronger distinguisher, as the reduction (which acts as a distinguisher) not only has access to the aggregated signature, but can also observe/program the random oracle while the signature is being generated (and can even rewind the signers). Unsurprisingly, we are not aware of any scheme that would hide the identities of the signers in such a strong (and arguably unusual) adversarial setting.

To circumvent this problem, we instead require the signature scheme to have a special property, which is held naturally by most threshold schemes: we require the existence of a unique secret key sk that can be efficiently reconstructed by corrupting any set of $(t+1)$ parties and then used to run a non-interactive signature algorithm $\mathsf{Sign}(\mathsf{sk}, m)$, which does not require the involvement of any of the parties to generate a valid signature on a message m. Because this secret key is unique and can be reconstructed regardless of which parties were corrupted, and because Sign only requires as input the special secret key and the message to be signed, we can easily argue that a signature generated via Sign leaks no information about the identities of the parties. We also argue that all of the

[12] The trivial scheme has been widely used by cryptocurrencies like Bitcoin and Polkadot.

[13] Ignoring negligible probabilities from $\mathcal{B}$ extracting the DL from the forgery by rewinding or straight-line, e.g., in the AGM, in the usual way for Schnorr signatures.

schemes considered in this work admit an efficient reconstruction procedure for the special secret key and an algorithm Sign. Indeed, key-uniqueness occurs in many threshold schemes that are compatible with standard, single-party signatures, such as BLS, ECDSA, and Schnorr signatures (cf. Table 1).

The Constraint of Consistent Keys and Parameters. Handling rewinding properly in security reductions requires great care, and metareductions – themselves reductions – poses similar challenges, as already indicated above. In our first metareduction, the reduction $\mathcal{B}$ may rewind the adversary $\mathcal{A}$ (simulated by the metareduction), and in our second metareduction, $\mathcal{B}$ must rewind $\mathcal{A}$ at least once. We require that each time $\mathcal{B}$ initializes an instance of $\mathcal{A}$, $\mathcal{B}$ provides the same public parameters and public keys (and hence the same secret key shares, by key-uniqueness). Our metareductions fail when this constraint does not hold. Indeed, after introducing fresh random coins to $\mathcal{B}$ via corruption queries, $\mathcal{M}$ is not guaranteed to receive consistent secret key shares across the two sets of queries, as $\mathcal{B}$ may rewind $\mathcal{A}$ and output different public keys. In this case, $\mathcal{M}$ will receive inconsistent secret key shares and so will fail to recover sk.

A natural question is whether such a constraint is reasonable, or whether a reduction that forks the adversary by providing different sets of keys at key generation[14] could lead to some meaningful advantage. We are not aware of any existing adaptive security reductions in the literature that take this approach.

Our results can (and should) be understood as a guide to design adaptively secure schemes. Indeed, our results indicate that proving threshold signature schemes adaptively secure under NIAs may require specific techniques such as forking at the point of key generation. As such, our results give a foundation for future research in this direction.

More Rewinding Subtleties in Impossibility Result 1. Rewinding the reduction is a common metareduction strategy originating from the work of Coron [40], which rules out tight reductions to NIAs for *unique* signature schemes.[15] For example, in the case of signature schemes, rewinding the reduction allows the adversary (simulated by the metareduction) to obtain an additional signature via a signing query in one execution of the reduction and subsequently use it as a forgery in the other. Such arguments require care for handling how the extra signature affects the reduction's state. This difficulty is compounded if the reduction is allowed to rewind the adversary [28], as is the case in both of our impossibility results.

One way to bypass this difficulty is to instead run two *independent* copies of the reduction(on independent randomness), which cleanly separates the additional signature from the reduction's state. Indeed, this is the approach taken for

[14] I.e., *completes* a first execution of key generation, and then rewinds.

[15] Extended to (unique) threshold BLS under OMDL for fully algebraic reductions for $t_c = t$ adaptive corruptions in [8]. Note that Coron's metareduction implicitly assumes that the public key output by the reduction to the adversary is correct; as in [65], we rectify this error by enforcing an efficient check that the reduction has computed the public key correctly via the VerifyPKs algorithm. (See informal key-uniqueness definition above, Definition 4, and Remark 1.)

Schnorr signatures in [13, 52]; however, these metareductions critically rely on the malleability of the public key and therefore only apply to the version of Schnorr signatures that does not hash the public key. There are known vulnerabilities in the multi-party setting for Schnorr signatures if the public key is not included in the hash [21, 31, 29], and our results capture these so-called "key-prefixed" Schnorr schemes. Additional challenges arise in these works for reductions that program the random oracle. Indeed, [52] only applies only to the non-programmable ROM, and [13] applies only to the programmable ROM.

Our first metareduction similarly rewinds the reduction, using responses from one instance to aid its simulation in the other (with care in how this affects the reduction's state); however, unlike all of the above, *we do not require even one signature to be issued.* Our metareductions even rule out adversaries that do not query the signing oracles, which strengthens our results. Moreover, this key aspect of our metareductions allows us to rule out reductions that entirely control the random oracles for the adversary (if any), *including programming.*

To summarize, our first metareduction eliminates a broad class of security reductions for unique and non-unique threshold signatures, both straight-line and rewinding, including programming. Moreover, all of this holds for the range of $\lfloor t/\ell \rfloor < t_c \leq t$ adaptive corruptions. We provide further comparison with metareductions in the literature in Appendix A.

The Algebraic Group Model (AGM) in Impossibility Result 1. Our first impossibility result not only captures reductions the standard model, but also the random oracle model. A natural question is whether it extends to algebraic adversaries. An algebraic adversary is one that supplies a representation of each group element it outputs relative to all of the group elements it has seen, which is inherently non-black-box [14, 8]. Because the metareduction $\mathcal{M}$ knows the secret key sk when it produces its forgery, $\mathcal{M}$ can simulate an algebraic adversary, as it knows the representation of its forgery. However, an important caveat is that this does not hold if *the secret key contains group elements.* If sk contains group elements, it is possible that the forgeries produced by $\mathcal{M}$ and the real adversary $\mathcal{A}$ are identically distributed, but the algebraic combinations that give the forgeries have different distributions (because $\mathcal{M}$ and $\mathcal{A}$ find sk in different ways). In this case, $\mathcal{A}$ may give a break for its NIA challenge, but $\mathcal{M}$ may fail to do so. Interestingly, this barrier indicates that a key-unique threshold signature scheme could be proven secure under a NIA if its secret key contains group elements. By extension, we conjecture that some "key-unique" threshold verifiable unpredictable functions (VUFs) (essentially unique threshold signatures schemes) with group-element secret keys currently proven adaptively secure under interactive assumptions could actually be proven under non-interactive assumptions.

Impossibility Result 2. With our second impossibility result, we want to rule out the existence of adaptive secure key-unique threshold Schnorr signature schemes under *interactive* assumptions. A natural first attempt to prove this is to adopt the same approach as in our first impossibility result, designing a metareduction that rewinds the reduction $\mathcal{B}$, corrupting at least $t + 1$ parties

overall and reconstructing sk to generate a forgery. However, this approach fails when $\mathcal{B}$ is a reduction to an *interactive* assumption. Consider the case of OMDL, for example. In this case, the reduction $\mathcal{B}$ may be designed in such a way that any corruption query made by the adversary triggers $\mathcal{B}$ asking for a solution of a DL challenge. Hence, if $\mathcal{B}$ is supposed to contradict the hardness of $(t+1)$-OMDL, the metareduction (by rewinding $\mathcal{B}$) would make $\mathcal{B}$ ask (with high probability) for the solution of at least $t+1$ DL challenges, rendering $\mathcal{B}$ an invalid reduction. Thus, our second metareduction must take a different strategy.

In particular, we show that given a reduction $\mathcal{B}$ (formally defined in Theorem 4) against q-OMDL for any q, we can construct an efficient metareduction $\mathcal{M}$ against $(q+1)$-OMDL as follows.

First, we define one particular (inefficient) adversary $\mathcal{A}$, which $\mathcal{M}$ will efficiently simulate to $\mathcal{B}$. $\mathcal{A}$ is executed on coins, public parameters, public keys, and the (single) random oracle response given by $\mathcal{B}$. This RO response is for the query $c^* = \mathsf{H}(R^*, \mathsf{PK}, m^*)$ for $\mathcal{A}$'s forgery $(m^*, \sigma^* = (R^*, z^*))$, where R^* is an arbitrary random group element and m^* is an arbitrary message. $\mathcal{A}$ then randomly selects a set of $t_c = t$ parties to corrupt, using a random function evaluated on the above inputs given by $\mathcal{B}$. $\mathcal{A}$ queries $\mathcal{O}^{\mathsf{Corrupt}}$ to $\mathcal{B}$ to obtain the secret keys of the parties in this set, and brute-forces the secret key of an arbitrary $t+1^{st}$ party. $\mathcal{A}$ then reconstructs the secret key sk and computes a Schnorr signature (Sign) directly on m^* to produce a valid forgery.

We then show that $\mathcal{M}$ can simulate $\mathcal{A}$ efficiently. $\mathcal{M}$ takes as input $q+1$ OMDL challenges $((\mathbb{G}, p, g), (X_0, \ldots, X_q))$, where g is a generator of a group $\mathbb{G}$ of order p. $\mathcal{M}$ then runs $\mathcal{B}((\mathbb{G}, p, g), (X_0, \ldots, X_{q-1}); \rho)$ on the first q OMDL challenges and random coins ρ. $\mathcal{B}$ has access to a discrete logarithm solution oracle $\mathcal{O}^{\mathsf{dlsol}'}$ in the OMDL game, which is simulated by $\mathcal{M}$ to $\mathcal{B}$ using its own DL solution oracle $\mathcal{O}^{\mathsf{dlsol}}$. To simulate $\mathcal{A}$, $\mathcal{M}$ must produce a valid forgery at the end of $\mathcal{B}$'s first execution of $\mathcal{A}$ with the ability to corrupt only $t_c = t$ parties. To do so, $\mathcal{M}$ keeps one OMDL challenge, namely X_q, and one DL solution query in reserve for itself (i.e., $\mathcal{B}$ is not given X_q when it is initialized). When $\mathcal{B}$ rewinds $\mathcal{A}$, it either (1) changes the coins for $\mathcal{A}$, or (2) programs a different response for $\mathcal{A}$'s single query to H (on its forgery), or both. In either case, $\mathcal{M}$ can corrupt a different set of parties in its second execution, cor_2 vs. cor_1, as the value of $\mathcal{M}$'s PRF changes (as does the output of the real adversary's random function). $\mathcal{M}$ will have learned at least $t+1$ distinct secret key shares (except with probability $1/\binom{n}{t}$, when $\mathsf{cor}_1 = \mathsf{cor}_2$) and so can solve for sk. $\mathcal{M}$ then uses its one reserved DL solution query to produce a forgery for $\mathcal{B}$'s first execution of $\mathcal{A}$. Thus, $\mathcal{M}$ makes one more DL solution query than $\mathcal{B}$, which is bounded by $q-1$. $\mathcal{B}$ may rewind $\mathcal{A}$ more than once, and $\mathcal{M}$ can forge easily in each instance using sk. We then generalize this result to $\lfloor t/2 \rfloor < t_c \leq t$ with more involved combinatorial arguments. The full proof is given in Section 6.

It is natural to ask at this point whether we can extend this result to a smaller corruption threshold, such as $t/3$. However, there exist threshold schemes that are adaptively secure for $t_c \leq \lfloor t/2 \rfloor$ [43, 41]. We recall that our argument crucially relies on the assumption that the reduction rewinds the adversary at least once.

This is what ultimately allows the metareduction to collect sufficiently many secret key shares and reconstruct the secret key. For our argument to work for, say $t_c > t/3$, we would need to assume that the reduction rewinds the adversary at least two times. This can be generalized for any $t_c > t/\ell$, assuming that the reduction rewinds the adversary at least $\ell - 1$ times. So, while in principle we could extend our argument to generic t_c, assuming that the reduction needs to rewind the adversary more than once is a strong assumption. Indeed, the security proofs for the schemes proposed in [43, 41] admit a reduction that rewinds the adversary only once.

Technical Challenges and Insights. In addition to the challenges of defining the requirements of key-uniqueness and consistent public keys and parameters as in our first impossibility result, our second metareduction takes a different strategy, introducing new subtleties.

Constraints on the Reduction in Impossibility Result 2. The constraints in our second impossibility result – key-unique threshold Schnorr signatures proven using rewinding in the ROM – capture a natural class of reductions for these schemes [43, 20, 44, 17, 80, 38, 41, 84], as discussed above. However, let us close with a few final observations in light of recent work building on our techniques and results.

Subsequent work [4] uses a similar proof to rule out a black-box, fully algebraic reduction to full (i.e., $t_c = t$) adaptive security of a threshold Schnorr scheme ms-FROST under AOMDL in the ROM. It eliminates straight-line and rewinding (fully algebraic) reductions for this construction – which is not key unique.

What about eliminating straight-line reductions for key-unique schemes? Our arguments (for the case $t_c > \lfloor t/2 \rfloor$) would not apply for schemes that admit a straight-line reduction. Hence, our result indicates that the only way to prove the adaptive security of a key-unique threshold Schnorr scheme under relevant interactive computational assumptions is to design a straight-line reduction (which arguably limits quite significantly the power of a reduction).

On this point, recall that one defining feature of our metareductions is the fact that they define key-only adversaries. This is precisely the strategy exploited in follow-up work [45], which gives a plausible attack based on public key shares alone. It shows that the full adaptive security (and slightly below) of threshold Schnorr schemes with a Shamir secret-shared key[16] cannot be proven without an assumption that implies the hardness of some instances of a problem P they define. However, this is not necessarily an *additional* assumption in case the hardness of DL or other group assumptions implies hardness of P, currently a fascinating open problem.

[16] Our metareductions work for generalized secret sharing (e.g., multiple field elements or packed secret sharing); that is, all schemes with a shared secret key that is enough for signing and can be obtained from any $t + 1$ shares.

2 Preliminaries

2.1 General Notation

Let $\kappa \in \mathbb{N}$ denote the security parameter and 1^κ its unary representation. A function $f : \mathbb{N} \to \mathbb{R}$ is called *negligible* if for all $c \in \mathbb{R}, c > 0$, there exists $k_0 \in \mathbb{N}$ such that $|f(k)| < \frac{1}{k^c}$ for all $k \in \mathbb{N}, k \geq k_0$. For a non-empty set S, let $x \leftarrow\!\!\$\, S$ denote sampling an element of S uniformly at random and assigning it to x. We use $[n]$ to represent the set $\{1, \ldots, n\}$ and $[0..n]$ to represent the set $\{0, \ldots, n\}$.

Let PPT denote probabilistic polynomial time. Algorithms are randomized unless explicitly noted otherwise. Let $y \leftarrow A(x; \omega)$ denote running algorithm A on input x and randomness ω and assigning its output to y. Let $y \leftarrow\!\!\$\, A(x)$ denote $y \leftarrow A(x; \omega)$ for a uniformly random ω. The set of values that have non-zero probability of being output by A on input x is denoted by $[A(x)]$.

We define non-interactive computational and decisional assumptions (NIAs) in Appendix C and the (A)OMDL and AOMCDH assumptions in Appendix G.

Polynomial Interpolation. A polynomial $f(x) = a_0 + a_1 x + a_2 x^2 + \ldots + a_t x^t$ of degree t over a field $\mathbb{F}$ can be interpolated by $t+1$ points. Let $\eta \subseteq [n]$ be the list of $t+1$ distinct indices corresponding to the x-coordinates $x_i \in \mathbb{F}, i \in \eta$, of these points. Then the Lagrange polynomial $L_i(x)$ has the form $L_i(x) = \prod_{j \in \eta; j \neq i} \frac{x - x_j}{x_i - x_j}$. Given a set of $t+1$ points $(x_i, f(x_i))_{i \in [\eta]}$, any point $f(x_\ell)$ on the polynomial f can be determined by Lagrange interpolation, $f(x_\ell) = \sum_{k \in \eta} f(x_k) \cdot L_k(x_\ell)$.

2.2 Threshold Secret Sharing

We next define a threshold secret-sharing scheme. We do so because our results apply to threshold signature schemes with a recovery algorithm, which can output the secret key given a sufficient number of secret key shares.

Definition 1. *An* $(n, t+1)$**-threshold secret-sharing scheme** *is a tuple of efficient algorithms* $\mathcal{SS} = (\mathcal{SS}.\mathsf{Setup}, \mathcal{SS}.\mathsf{Share}, \mathcal{SS}.\mathsf{Recover})$*, defined as follows:*

- $\mathcal{SS}.\mathsf{Setup}(1^\kappa) \to \mathsf{par}$*: On input the security parameter, this algorithm outputs public parameters* par *(which are given implicitly as input to all other algorithms).*
- $\mathcal{SS}.\mathsf{Share}(s, n, t+1) \to (\{(1, \bar{x}_1), \ldots, (n, \bar{x}_n)\})$*: A probabilistic algorithm that takes as input the secret* s*, the number of participants* n*, and the threshold* $t+1$*, where* $0 < t+1 \leq n$*, and outputs a list of shares* $\{(1, \bar{x}_1), \ldots, (n, \bar{x}_n)\}$*.*
- $\mathcal{SS}.\mathsf{Recover}(t+1, \{(i, \bar{x}_i)\}_{i \in \mathcal{S}}) \to s/\perp$*: A deterministic algorithm that takes as input a threshold* $t+1$ *and a set of shares* $\{(i, \bar{x}_i)\}_{i \in \mathcal{S}}$ *and outputs* $\perp$ *if* $\mathcal{S} \not\subseteq [n]$ *or* $|\mathcal{S}| < t+1$*. Otherwise, it recovers* s *using the set of shares.*

A threshold secret-sharing scheme is *correct* if and only if for all security parameters $\kappa \in \mathbb{N}$, all $\mathsf{par} \in [\mathcal{SS}.\mathsf{Setup}(1^\kappa)]$, all $(\{(i, \bar{x}_i)\}_{i \in [n]}) \in [\mathcal{SS}.\mathsf{Share}(s, n, t+1)]$, all $\mathcal{S} \subseteq [n]$ such that $|\mathcal{S}| \geq t+1$, it holds that $\mathcal{SS}.\mathsf{Recover}(t+1, \{(i, \bar{x}_i)\}_{i \in \mathcal{S}}) = s$. It is *secure* if every set of $t+1$ shares output by $\mathcal{SS}.\mathsf{Share}(s, n, t+1)$ reveals nothing about s [26].

The canonical example is Shamir secret-sharing [82], which shares a single field-element secret. However, our results apply to generalized secret-sharing schemes, e.g., packed secret-sharing schemes or a multiple field-element secret.

2.3 Metareductions

In this work, we demonstrate impossibility results against a class of reductions, using the metareduction technique [27, 50, 52]. For an in-depth discussion of how our metareduction techniques compare to others in the literature, see Appendix A. A metareduction demonstrates the impossibility of a security reduction by defining a particular adversary for which the reduction fails.

A reduction demonstrates the security of a primitive $\mathcal{Q}$ against an adversary $\mathcal{A}$ by defining an efficient reduction algorithm $\mathcal{B}$ against a primitive Δ, and then showing that $\mathcal{B}^{\mathcal{A}}$ (the reduction $\mathcal{B}$ using $\mathcal{A}$ as a subroutine) can win against the challenger for Δ. If Δ is assumed to be hard to break, then the reduction $\mathcal{B}^{\mathcal{A}}$ shows that $\mathcal{Q}$ is likewise hard to break. Importantly, $\mathcal{B}$ must be able to succeed with *any* instantiation of $\mathcal{A}$ as defined.

To show the *impossibility* of such a reduction $\mathcal{B}^{\mathcal{A}}$, the metareduction technique gives a proof by contradiction, showing that if the reduction succeeds with a particular adversary $\mathcal{A}^{\dagger}$, then the metareduction can use $\mathcal{B}$ as a subroutine to break some primitive (while simulating $\mathcal{A}^{\dagger}$ to $\mathcal{B}$). If this primitive is assumed to be hard to break, then by contradiction, such a reduction $\mathcal{B}$ cannot exist.

More specifically, the metareduction technique shows that at least one particular (possibly inefficient) adversary instance $\mathcal{A}^{\dagger}$ exists that can win against its challenger for $\mathcal{Q}$, but where $\mathcal{B}^{\mathcal{A}^{\dagger}}$ fails to win against its challenger for Δ. By defining an efficient algorithm $\mathcal{M}$ that can simulate $\mathcal{A}^{\dagger}$ to $\mathcal{B}$ in such a way that when $\mathcal{B}$ successfully outputs a valid solution to Δ, then $\mathcal{M}$ can likewise break a primitive Δ' (where Δ' could equal Δ). However, because Δ' is assumed to be hard to break, the existence of such a metareduction $\mathcal{M}$ is a contradiction, thus establishing the impossibility of $\mathcal{B}^{\mathcal{A}^{\dagger}}$.

Fully Black-Box Reductions. Our results show the impossibility of a class of reductions that are *fully black-box*, which we define next.

Definition 2 (Fully Black-Box Reduction [79, 11]). *Let $\mathcal{Q}$ be a primitive, and let $\mathcal{A}$ be any (possibly inefficient) adversary which breaks any instance q of $\mathcal{Q}$. A* **fully black-box reduction** *$\mathcal{B}$ is an efficient algorithm that transforms any $\mathcal{A}$ into an algorithm $\mathcal{B}^{\mathcal{A},\delta}$ that breaks instance δ of a primitive Δ. The reduction $\mathcal{B}$ treats both the adversary $\mathcal{A}$ and Δ as black-box.*

3 Key-Unique Threshold Signatures

We next specify the syntax of a threshold signature scheme defined with respect to a threshold secret-sharing scheme (Definition 1). To simplify notation used throughout the paper, the trusted key generation algorithm also outputs public key shares; however, the public key shares may be empty (see Section 3.1).

Trusted key generation may be replaced by a distributed key generation (DKG) protocol, where the secret of which parties hold shares is virtual and not held by any party. We then introduce the notion of a *key-unique* threshold signature scheme, which is the subject of this work.

Definition 3. *A* **threshold signature scheme** $\mathsf{TS}_{\mathcal{SS}}$ *defined with respect to a threshold secret-sharing scheme* $\mathcal{SS}$ *and whose signing protocol consists of* r *rounds is a tuple of efficient algorithms* $\mathsf{TS}_{\mathcal{SS}} = (\mathsf{Setup}, \mathsf{KeyGen}, (\mathsf{Sign}_1, \ldots, \mathsf{Sign}_r), \mathsf{Combine}, \mathsf{Verify})$*, defined as follows:*

- $\mathsf{Setup}(1^\kappa) \to \mathsf{par}$*: Takes as input the security parameter and outputs public parameters* par *(which are given implicitly as input to all other algorithms).*
- $\mathsf{KeyGen}(n, t+1) \to (\mathsf{PK}, \{\mathsf{PK}_i\}_{i\in[n]}, \{\mathsf{sk}_i\}_{i\in[n]})$*: A probabilistic algorithm that takes as input the number of signers* n *and the threshold* $t+1$*. The algorithm runs a threshold secret-sharing scheme* $\mathcal{SS}$*, outputting secret shares* $\mathsf{sk}_i \in \mathcal{D}_{\mathsf{sk}}$*, for* $i \in [n]$*, of a secret* $\mathsf{sk} \in \mathcal{D}_{\mathsf{sk}}$*.*[17] *Additionally, the algorithm outputs a public key* $\mathsf{PK} \in \mathcal{D}_{\mathsf{PK}}$ *and public key shares* $\mathsf{PK}_i \in \mathcal{D}_{\mathsf{PK}}$*, for* $i \in [n]$*.* $\mathcal{D}_{\mathsf{sk}}$ *is the domains of secret keys and secret key shares, and* $\mathcal{D}_{\mathsf{PK}}$ *is the domain of public keys and public key shares for* $\mathsf{TS}_{\mathcal{SS}}$*. Note* $\mathcal{D}_{\mathsf{sk}}, \mathcal{D}_{\mathsf{PK}}$ *are implicitly parameterized by* par*. Each signer* i *is sent its* sk_i *privately.*
- $(\mathsf{Sign}_1, \ldots, \mathsf{Sign}_r) \to \{\mathsf{pm}_{1,i}, \ldots, \mathsf{pm}_{r,i}\}_{i\in\mathcal{S}}$*: A set of probabilistic algorithms where each subsequent algorithm represents a single stage in an interactive signing protocol, performed by each signing party in a signing set* $\mathcal{S} \subseteq [n], |\mathcal{S}| \geq t+1$*, with respect to a message* m*, defined as follows:* $(\mathsf{pm}_{1,i}, \mathsf{st}_{1,i}) \leftarrow \mathsf{Sign}_1(i, \mathsf{sk}_i, \mathcal{S}, m); (\mathsf{pm}_{2,i}, \mathsf{st}_{2,i}) \leftarrow \mathsf{Sign}_2(\mathsf{st}_{1,i}, \mathcal{PM}_1); \ldots; \mathsf{pm}_{r,i} \leftarrow \mathsf{Sign}_r(\mathsf{st}_{r-1,i}, \mathcal{PM}_{r-1})$*, where* $\mathsf{pm}_{j,i}$ *is the protocol message sent by party* $i \in \mathcal{S}$ *in round* j*,* $\mathsf{st}_{j,i}$ *is the state of party* i *in round* j*, and* $\mathcal{PM}_j = \{\mathsf{pm}_{j,i}\}_{i\in\mathcal{S}}$ *is a set of protocol messages sent in round* j*. The signing set* $\mathcal{S}$ *and message* m *are shown here as inputs to the first round, but they may be deferred to subsequent rounds, depending on the scheme.*
- $\mathsf{Combine}(\mathcal{S}, m, \mathcal{PM}_1, \mathcal{PM}_2, \ldots, \mathcal{PM}_r) \to (m, \sigma)$*: A deterministic algorithm that takes as input the signing set* $\mathcal{S}$*, the message* m*, and a set of protocol messages* $\mathcal{PM}_1, \mathcal{PM}_2, \ldots, \mathcal{PM}_r$ *from all* r *signing rounds and outputs a signature* σ*. (Note that this algorithm is separate from the signing rounds and may be performed by one of the signers or an external party.)*
- $\mathsf{Verify}(\mathsf{PK}, m, \sigma) \to 0/1$*: A deterministic algorithm that takes as input the public key* PK*, a message* m*, and a purported signature* σ *and outputs* 1 *(accept) if the signature verifies; else, it outputs* 0 *(reject).*

Correctness of $\mathsf{TS}_{\mathcal{SS}}$ [10]. A threshold signature scheme is *correct* if and only if for all security parameters $\kappa \in \mathbb{N}$, all $\mathsf{par} \in [\mathsf{Setup}(1^\kappa)]$, all $(\mathsf{PK}, \{(\mathsf{PK}_i, \mathsf{sk}_i)\}_{i\in[n]}) \in [\mathsf{KeyGen}(n, t+1)]$, all $\mathcal{S} \subseteq [n]$ such that $|\mathcal{S}| \geq t+1$, and all messages $m \in \{0,1\}^\kappa$,

[17] Our definition can be extended to allow secret key shares and the overall secret key to be in their own domains $\mathcal{D}_{\mathsf{sk}_i}$.

$\mathsf{TSignHon}(\mathsf{PK}, \{\mathsf{sk}_i\}_{i\in\mathcal{S}}, \mathcal{S}, m)$

return $\perp$ **if** $\mathcal{S} \not\subseteq [n] \wedge |S| < t+1$
for $k \in \mathcal{S}$ **do** $(\mathsf{pm}_{1,k}, \mathsf{st}_{1,k}) \leftarrow \mathsf{Sign}_1(k, \mathsf{sk}_k, \mathcal{S}, m)$
for $k \in \mathcal{S}$ **do** $(\mathsf{pm}_{2,k}, \mathsf{st}_{2,k}) \leftarrow \mathsf{Sign}_2(\mathsf{st}_{1,k}, \{\mathsf{pm}_{1,i}\}_{i\in\mathcal{S}})$
...
for $k \in \mathcal{S}$ **do** $\mathsf{pm}_{r,k} \leftarrow \mathsf{Sign}_r(\mathsf{st}_{r-1,k}, \{\mathsf{pm}_{r-1,i}\}_{i\in\mathcal{S}})$
$\sigma \leftarrow \mathsf{Combine}(\mathcal{S}, m, \{\mathsf{pm}_{1,i}, \ldots, \mathsf{pm}_{r,i}\}_{i\in\mathcal{S}})$
return σ

Fig. 1: An algorithm $\mathsf{TSignHon}$ for honestly generating a threshold signature for $\mathsf{TS}_{\mathcal{SS}}$ with r signing rounds.

it holds that: $\Pr[\mathsf{Verify}(\mathsf{PK}, m, \sigma) = 1 \mid \sigma \leftarrow \mathsf{TSignHon}(\mathsf{PK}, \{\mathsf{sk}_i\}_{i\in[\mathcal{S}]}, \mathcal{S}, m)] = 1$, where the algorithm $\mathsf{TSignHon}$ is as defined in Fig. 1, which models an honest execution of the signing protocol.

3.1 Key-Unique Threshold Signatures

The impossibility results presented in this work apply to key-unique threshold signature schemes. Intuitively, a key-unique threshold signature scheme outputs public keys that have a unique correspondence to secret keys, and allows for efficient verification that public keys are well-formed. We provide a formal definition of key-uniqueness and then give examples of key-unique and non-key-unique schemes.

Definition 4. *A* **key-unique threshold signature scheme** $\mathsf{KeyUTS}_{\mathcal{SS},f,\mathsf{VerifyPKs}}$ *is defined with respect to the following:*

1. *An efficiently computable map* $f(i,\cdot) : \mathcal{D}_{\mathsf{sk}} \to \mathcal{D}_{\mathsf{PK}}$ *that is injective for all indices* $i \in [0..n]$ *and takes as input a secret key, and outputs a public key in the codomain* $\mathcal{D}_{\mathsf{PK}}$ *of public keys for* $\mathsf{KeyUTS}_{\mathcal{SS},f,\mathsf{VerifyPKs}}$*. The function* f *is in the standard model and is publicly known as it is a parameter.*
2. *An efficient algorithm* $\mathsf{VerifyPKs}(\mathsf{PK}, \{\mathsf{PK}_i\}_{i\in[n]}) \to \{0,1\}$ *that accepts as input a threshold public key* PK *and* n *public key shares* $\{\mathsf{PK}_i\}_{i\in[n]}$ *and outputs* 1 *if an honest execution of* KeyGen *returns* $(\mathsf{PK}, \{\mathsf{PK}_i\}_{i\in[n]})$*, indicating that the public key shares are valid with respect to* PK*. Otherwise, it outputs* 0*.*
3. *An efficient algorithm* $\mathsf{Sign}(\mathsf{sk}, m) \to \sigma$ *that takes as input a secret key* sk *and a message* m *and returns a signature* σ*.*

Furthermore, we require that the following conditions hold for $\mathsf{KeyUTS}_{\mathcal{SS},f,\mathsf{VerifyPKs}}$*:*

(i) For every secret key share $(i, \mathsf{sk}_i) \in [n] \times \mathcal{D}_{\mathsf{sk}}$*, the public key share* PK_i *is generated as* $\mathsf{PK}_i = f(i, \mathsf{sk}_i)$*.*

(ii) The threshold public key is generated similarly as $\mathsf{PK} = f(0, \mathsf{sk})$*.*

(iii) For every choice of $\{\mathsf{PK}, \{\mathsf{PK}_i\}_{i\in[n]}, \mathcal{S}, \{\mathsf{sk}_i\}_{i\in\mathcal{S}}\}$ *such that* $\mathcal{S} \subseteq [n]$ *with* $|\mathcal{S}| \geq t+1$ *and* $\mathsf{VerifyPKs}(\mathsf{PK}, \{\mathsf{PK}_i\}_{i\in[n]}) \to 1$*, then for any message* $m \in \{0,1\}^\kappa$*, it holds that:* $\Pr[\mathsf{Verify}(\mathsf{PK}, m, \mathsf{Sign}(\mathsf{sk}, m)) = 1 \ \wedge \ f(0, \mathsf{sk}) = \mathsf{PK} \mid \mathsf{sk} \leftarrow \mathcal{SS}.\mathsf{Recover}(t+1, \{(i, \mathsf{sk}_i)\}_{i\in\mathcal{S}})] = 1$.

Correctness of $\mathsf{KeyUTS}_{\mathcal{SS},f,\mathsf{VerifyPKs}}$**.** A key-unique threshold signature scheme is *correct* if and only if in addition to the correctness requirements of $\mathsf{TS}_{\mathcal{SS}}$ (Fig. 1), it holds that for all $(\mathsf{PK}, \{\mathsf{PK}_i\}_{i\in[n]}, \{\mathsf{sk}_i\}_{i\in[n]}) \in [\mathsf{KeyGen}(n, t+1)]$, then $1 \leftarrow \mathsf{VerifyPKs}(\mathsf{PK}, \{\mathsf{PK}_i\}_{i\in\mathcal{S}})$, and conditions (*i*, *ii*, *iii*) in Definition 4 hold.

Remark 1 (On the Requirement for Condition (iii)). Key to our metareductions' success is the requirement that, given a set of public key shares $\{\mathsf{PK}_i\}_{i\in[n]}$ and a subset $\mathcal{S} \subseteq [n]$ of at least $t+1$ secret key shares sk_i, it is possible to recover exactly one secret key sk. Moreover, this secret key can be used to generate a signature, by running an efficient algorithm Sign, which will verify correctly according to the verification procedure defined for the threshold scheme. Requiring the existence of an algorithm Sign and the recoverability of sk is not overly prohibitive. Many threshold signature schemes that are compatible with single-party signature schemes satisfy this property, such as BLS, ECDSA, Schnorr, etc. (cf. Table 1). Note that single-party signing for $\mathrm{Twinkle}_1$ is ECVRF [61].

The reason for $\mathsf{VerifyPKs}$ is that, while KeyGen defines a trusted dealer algorithm (or a DKG that realizes this functionality [26], see below), the reduction may not act as a trusted dealer, and instead might output invalid public key shares. In such a setting, our metareduction will fail to simulate a real adversary. Therefore, we define an algorithm $\mathsf{VerifyPKs}$ to make explicit that an adversary in the unforgeability game against $\mathsf{KeyUTS}_{\mathcal{SS},f,\mathsf{VerifyPKs}}$ should be able to efficiently distinguish if such a case occurs, and abort when $\mathsf{VerifyPKs}$ returns failure.

We now give examples of key-unique and non-key-unique schemes. Let $\mathsf{GroupGen}$ be a polynomial-time algorithm that on input the security parameter 1^κ, outputs a tuple $(\mathbb{G}, p, g)$, where g is a generator of $\mathbb{G}$ of order p.

Example 1(Perfectly Binding Commitment with Secret Reconstruction.) For $\mathsf{GroupGen}$, let $f : [0..n] \times \mathbb{Z}_p \to \mathbb{G}$ be defined by the mapping $(i, \mathsf{sk}_i) \mapsto g^{\mathsf{sk}_i}$ for all $i \in [1..n]$, and $(0, \mathsf{sk}) \mapsto g^{\mathsf{sk}}$. Suppose that KeyGen is defined such that there exists a degree $t+1$ interpolation polynomial $p(x)$ (Section 2.1) such that:

$$p(x) = \mathsf{sk} \cdot L_0(x) + \sum_{i\in[n]} L_i(x) \cdot \mathsf{sk}_i$$

where $p(i)$ is equal to sk_i. Then $\mathsf{VerifyPKs}$ can be defined to check that the public keys satisfy the relation:

$$\mathsf{PK}^{L_{0,k}} \cdot \prod_{i\in[n]} \mathsf{PK}_i^{L_{i,k}} = 1_{\mathbb{G}}, \text{ for all } n+1 \geq k \geq t+1$$

where $1_{\mathbb{G}}$ is the identity element of $\mathbb{G}$, and $L_{j,i}$ is the coefficient of the i^{th} term x^i of the j^{th} Lagrange polynomial $L_j(x)$ for the coalition $\{0\} \cup [n]$. In other words, $\mathsf{VerifyPKs}$ can be defined to check that $p(x)$ is a degree $t+1$ polynomial whose

constant term is sk. The $\mathcal{SS}.\mathsf{Recover}(t+1, \{(i, \mathsf{sk}_i)\}_{i \in \mathcal{S}})$ algorithm can then be defined to interpolate to recover sk. Given that sk_i are points on $p(x)$, that $p(x)$ is a degree $t+1$ polynomial, and $|\mathcal{S}| \geq t+1$, interpolation is guaranteed to succeed.

The KeyGen defined in Example 1 is the trusted key generation algorithm defined by Shamir secret-sharing of a secret sk, returning secret key shares $\{\mathsf{sk}_i\}_{i \in [n]}$, public key shares $\{\mathsf{PK}_i = g^{\mathsf{sk}_i}\}_{i \in [n]}$, and a threshold public key $\mathsf{PK} = g^{\mathsf{sk}}$. This is a strong form of key generation to rule out, and indeed our results extend to schemes employing any distributed key generation (DKG) protocol that realizes this functionality [[67],Fig. 6,] as well as any DKG realizing a relaxation of this functionality that allows PK to be biased [[67],Fig. 8].[18]

Example 2 (No Public Key Shares.) For $\mathsf{GroupGen}$, let $f' : [0..n] \times \mathbb{Z}_p \to \mathbb{G}$ be defined by:

$$f'(i, x) = \begin{cases} g^x, & \text{if } i = 0 \\ \perp, & \text{otherwise} \end{cases}$$

A threshold signature scheme defined with respect to this f' is *not* key unique.[19] Note that this f' implies that $\mathsf{KeyGen}(n, t+1) \to (\mathsf{PK}, \perp, \{\mathsf{sk}_i\}_{i \in [n]})$, i.e., only the threshold public key can be output to all participants; the partial public keys must remain hidden. This f' is used in [66, 36, 4, 12] to construct various threshold signature schemes achieving full adaptive security (Appendix B.2).

Example 3 (Collision Resistance.) For H a computationally collision-resistant hash function, let $f'' : [0..n] \times \{0,1\}^* \to \{0,1\}^*$ be defined by the mapping $(i, y) \mapsto \mathsf{H}(i, y)$ for all $i \in [0..n]$. A threshold signature scheme defined with respect to this f'' is *not* key unique. This is because an adversary that breaks the collision resistance of the hash function can equivocate to find an alternative preimage (i, y') such that $\mathsf{H}(i, y) = \mathsf{H}(i, y')$ but $y \neq y'$. The alternative preimage might not lead to a successful recovery of the secret key.

Example 4 (Pedersen Commitments.) For $\mathsf{GroupGen}$, where an additional generator h of $\mathbb{G}$, for which the discrete logarithm relation to g is unknown, is output, let $f''' : [0..n] \times \mathbb{Z}_p \to \mathbb{G}$ be an algorithm defined by:

$$f'''(i, x) = \begin{cases} g^x, & \text{if } i = 0 \\ g^x h^r, & \text{where } r \leftarrow\!\!\$\ \mathbb{Z}_q, \text{ otherwise} \end{cases}$$

A threshold signature scheme defined with respect to this f''' is *not* key unique. More examples of schemes that are not key unique are given in Appendix B.2 and discussed further in Remark 2.

3.2 Adaptive Security

Most threshold signature schemes in the literature are proven in the *static* security model, which captures a limited adversary that can only corrupt parties at the onset of the protocol. We provide a game-based definition in Fig. 2, inspired by

[18] An adversary $\mathcal{A}$ in the $\hat{\mathcal{F}}_{\mathsf{KeyGen}}$-hybrid world [[67],Fig. 6,] can be run as a subroutine by an adversary $\mathcal{B}$ in the $\mathcal{F}^{\Delta}_{\mathsf{KeyGen}}$-hybrid world [[67],Fig. 8] who sets the bias $\Delta = 0$.

[19] Assuming the public key shares are not revealed elsewhere, e.g., through signing.

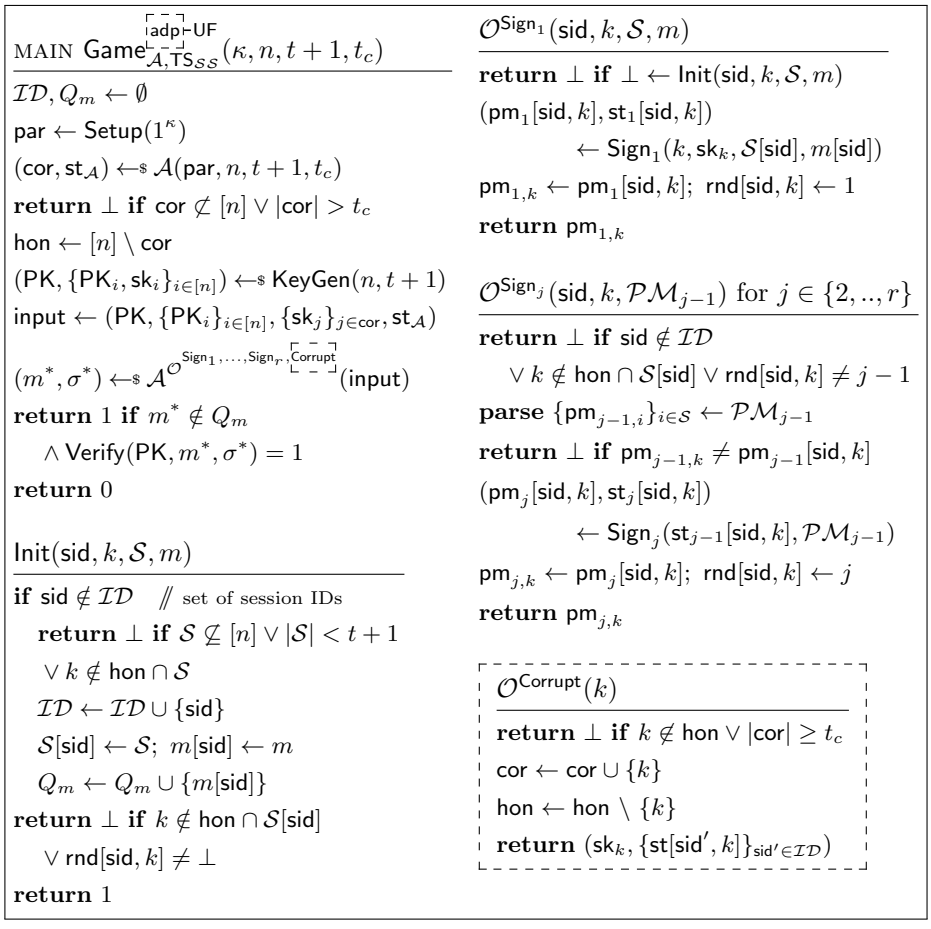

Fig. 2: Static and adaptive unforgeability games for a threshold signature scheme with r signing rounds. The public parameters par are implicitly given as input to all algorithms, and $\mathsf{pm}_1, \mathsf{pm}_2, \ldots, \mathsf{pm}_r$ represent protocol messages defined within the construction. The static game contains all but the dashed boxes, and the adaptive game adds the dashed boxes. t_c specifies the number of signers that the adversary may corrupt, and $t+1$ the threshold.

the definitions in [43, 71]. The static security game contains all but the dashed boxes and is analogous to existential unforgeability under chosen message attack (EUF-CMA) [62] for standard, single-party signature schemes.

In this work, we are interested in *adaptive* security for threshold signatures. We give the adaptive unforgeability experiment in Fig. 2, including the dashed boxes. In the adaptive setting, the adversary is given the additional capability to corrupt honest parties *at any point during the protocol*. More specifically, at any point, the adversary may query $\mathcal{O}^{\mathsf{Corrupt}}$ to corrupt any remaining honest signer $i \in \mathsf{hon}$, up to t_c number of parties in total. When the adversary queries $\mathcal{O}^{\mathsf{Corrupt}}$ on honest party $i \in \mathsf{hon}$, the oracle must output the secret key for party i *as well*

as the internal state of party i across all signing sessions. As such, the adversary learns additional information throughout the game (e.g., the signing state for each corrupted party).

Definition 5 (t_c-Adaptive Security). *Let the advantage of an adversary $\mathcal{A}$ playing the adaptive security game $\mathsf{Game}^{\mathsf{adp\text{-}UF}}_{\mathcal{A},\mathsf{TS}_{\mathcal{SS}}}(\kappa, n, t+1, t_c)$, as defined in Figure 2, be as follows:*

$$\mathit{Adv}^{\mathsf{adp\text{-}UF}}_{\mathcal{A},\mathsf{TS}_{\mathcal{SS}}}(\kappa, n, t+1, t_c) = \Pr[\mathsf{Game}^{\mathsf{adp\text{-}UF}}_{\mathcal{A},\mathsf{TS}_{\mathcal{SS}}}(\kappa, n, t+1, t_c) = 1]$$

A threshold signature scheme $\mathsf{TS}_{\mathcal{SS}}$ is t_c-adaptively secure *if for all PPT adversaries $\mathcal{A}$, for all $n, t \in \mathbb{N}$ polynomial in κ $\mathit{Adv}^{\mathsf{adp\text{-}UF}}_{\mathcal{A},\mathsf{TS}_{\mathcal{SS}}}(\kappa, n, t+1, t_c)$ is negligible in κ, for $t_c \leq t$.*

Definition 6 (Full Adaptive Security). *A threshold signature scheme $\mathsf{TS}_{\mathcal{SS}}$ achieves* full adaptive security *if it is t-adaptively secure.*

4 Impossibility of Full Adaptive Security Under Any Non-Interactive Assumption

We now prove our first impossibility result: that it is impossible to prove the full adaptive security (i.e., $t_c = t$) of any key-unique threshold signature scheme under any non-interactive computational (search) or decisional assumption using a fully black-box reduction. Our result captures both straight-line and rewinding reductions, as long as the reduction runs the adversary on the same public parameters and public keys in each instance (as discussed in Section 1.1.)

Theorem 1. *Assume there exists a fully black-box efficient reduction $\mathcal{B}$ that converts any adversary $\mathcal{A}$ against the t-adaptive unforgeability of a key-unique threshold signature scheme $\mathsf{KeyUTS}_{\mathcal{SS},f,\mathsf{VerifyPKs}}$ into an adversary against a non-interactive assumption NIA, assuming the security of pseudorandom functions. Assume further that, any time $\mathcal{B}$ initializes $\mathcal{A}$, $\mathcal{B}$ provides each instance of $\mathcal{A}$ with the same public parameters and public keys, and that $\mathcal{B}$ has success probability $\mathsf{Adv}^{\mathsf{NIA}}_{\mathcal{B}^{\mathcal{A}}}(\kappa)$ for a given $\mathcal{A}$ and runtime τ. Then, there exists a successful (possibly inefficient) adversary $\mathcal{A}$ against the t-adaptive unforgeability of $\mathsf{KeyUTS}_{\mathcal{SS},f,\mathsf{VerifyPKs}}$ and an efficient metareduction $\mathcal{M}$ that breaks the NIA assumption with non-negligible probability:*

$$\mathsf{Adv}^{\mathsf{NIA}}_{\mathcal{M}^{\mathcal{B}}}(\kappa) \;\geq\; \mathsf{Adv}^{\mathsf{NIA}}_{\mathcal{B}^{\mathcal{A}}}(\kappa)\left(\mathsf{Adv}^{\mathsf{NIA}}_{\mathcal{B}^{\mathcal{A}}}(\kappa) - \tau/\binom{n}{t}\right) - \mathsf{negl}(\kappa)$$

and runtime $\mathsf{Time}_{\mathcal{M}}(\kappa) = 2\tau + \mathsf{poly}(\kappa)$.

Remark 2 (Intuition on Requirement for Key-Uniqueness). Here, we expand on the intuition for key-uniqueness given in Section 1.1. Consider a threshold signature scheme $\mathsf{TS}'_{\mathcal{SS}}$ such that the key generation algorithm outputs public key shares $\{\mathsf{PK}_i\}_{i=1}^n$ such that $\mathsf{PK}_i = g^{\mathsf{sk}_i} h^{\rho_i}$, for $\rho_i \leftarrow\!\!\$\, \mathbb{Z}_p$, and where the discrete logarithm of $h = g^\alpha$ base g is unknown. Our metareduction will fail to simulate

a successful adversary for such a scheme; if the reduction knows the trapdoor α, the reduction can simply pick the corrupted parties' secret shares $\{\mathsf{sk}_i\}_{i\in\mathsf{cor}}$ dynamically *after* the metareduction makes its query to $\mathcal{O}^{\mathsf{Corrupt}}$. Because our metareduction introduces fresh randomness to the reduction when it chooses the set of corrupt parties, the reduction essentially would "fork" at that point, and the metareduction would lose consistency of secret shares output by the reduction across different instances. Hence, even if the metareduction obtains corruption sets $\{\mathsf{sk}_i\}_{i\in\mathsf{cor}_1}, \{\mathsf{sk}_j\}_{j\in\mathsf{cor}_2}$ where $|\mathsf{cor}_1 \cup \mathsf{cor}_2| \geq t+1$, there is no guarantee that these corruption sets could be composed to recover sk. Therefore, our metareduction would fail against a reduction for $\mathsf{TS}'_{\mathcal{SS}}$.

Remark 3 (Intuition on Tightness Loss). The bound in Theorem 1 depends on both n and t. The tightness loss captures the bad events where $\mathcal{M}$ does not receive enough information from $\mathcal{B}$ to simulate $\mathcal{A}$, e.g., the sets of corrupted parties across the different instances of $\mathcal{B}$ do not yield a sufficient number of shares to perform $\mathcal{SS}.\mathsf{Recover}$. In this setting, a reduction exists that could win with $\mathcal{A}$, but not with the metareduction simulating $\mathcal{A}$. The squared term for $\mathsf{Adv}^{\mathsf{NIA}}_{\mathcal{B}^{\mathcal{A}}}$ captures the fact that, while both instances of the reduction do not need to succeed in the NIA game (indeed, the second instance is aborted prematurely), they do both need to simulate correctly. Finally, we need to consider the number of rewinds that $\mathcal{B}$ makes, as $\mathcal{B}$ may answer $\mathcal{A}$'s oracle queries only in one rewinding thread and abort in all others. The number of rewinds is bounded by the total running time of $\mathcal{B}$, which we denote with τ. We analyze this formally in the proof.

Proof (of Theorem 1).

Description of Adversary $\mathcal{A}$. We give a description of one particular (and inefficient) adversary $\mathcal{A}$. Recall that $\mathcal{B}$ initializes any adversary with coins $\rho_{\mathcal{A}}$, and the adversary runs deterministically with respect to $\rho_{\mathcal{A}}$. Below, we describe the code of $\mathcal{A}$, which includes a random function $R : \{0,1\}^{\ell(\kappa)} \to [C]^t$, where $[C]^t := \{\mathsf{cor} \subseteq [n] \mid |\mathsf{cor}| = t\}$, that takes as input a bit string and outputs a set cor such that $\mathsf{cor} \subseteq [n], |\mathsf{cor}| = t$.

1. $\mathcal{B}$ initializes $\mathcal{A}$ on the following values: (1) coins $\rho_{\mathcal{A}}$ and (2) public parameters $\mathsf{pp} = (\mathsf{par}, n, t+1, t_c)$.
2. $\mathcal{A}$ returns an empty initial corruption set $\mathsf{cor} \leftarrow \emptyset$. $\mathcal{B}$ then outputs public keys $\mathsf{keys} = (\mathsf{PK}, \{\mathsf{PK}_i\}_{i=1}^n)$.
3. $\mathcal{A}$ checks that $\mathsf{VerifyPKs}(\mathsf{PK}, \{\mathsf{PK}_i\}_{i\in[n]}) = 1$; if not, $\mathcal{A}$ aborts. We refer to this abort event as $\mathsf{InvalidKeysAbort}$.
4. $\mathcal{A}$ computes a corruption set $\mathsf{cor} \leftarrow R(\rho_{\mathcal{A}}, \mathsf{pp}, \mathsf{keys})$.
5. For all $i \in \mathsf{cor}$, $\mathcal{A}$ queries the corruption oracle $(\mathsf{sk}_i, \{\mathsf{st}[\mathsf{sid}', i]\}_{\mathsf{sid}'\in\mathcal{ID}}) \leftarrow \mathcal{O}^{\mathsf{Corrupt}}(i)$, in the natural order, to $\mathcal{B}$. Recall that $\mathsf{st}[\mathsf{sid}', i]$ is the state of participant i for signing session sid'. If $\mathcal{B}$ does not return t secret keys, $\mathcal{A}$ aborts. We refer to this bad event as $\mathsf{NotAllKeysAbort}$.
6. $\mathcal{A}$ then solves for an arbitrary $\mathsf{sk}_{i'}, i' \notin \mathsf{cor}$, by brute-force search, for $\mathsf{PK}_{i'}$.
7. $\mathcal{A}$ defines cor' to be the set of participant identifiers in $\mathsf{cor} \cup i'$ and runs $\mathsf{sk} \leftarrow \mathcal{SS}.\mathsf{Recover}(t+1, \{(i, \mathsf{sk}_i)\}_{i\in\mathsf{cor}'})$.

8. $\mathcal{A}$ runs $\mathsf{Sign}(\mathsf{sk}, m^*)$ on an arbitrary message m^* to produce a valid signature σ^* under PK. $\mathcal{A}$ outputs (m^*, σ^*).

Description of $\mathcal{M}$. The metareduction $\mathcal{M}$ is initialized by the NIA game as $\mathcal{M}(\mathsf{NIAChal}; \rho')$, with input challenge $\mathsf{NIAChal}$ and coins ρ'. Below, we describe the code of $\mathcal{M}$, which includes random coins ρ that $\mathcal{M}$ will provide as input to $\mathcal{B}$. $\mathcal{M}$ executes two instances of $\mathcal{B}$; we refer to the first instance as $\mathcal{B}_1$, and the second instance as $\mathcal{B}_2$. Both instances are initialized on the same challenge $\mathsf{NIAChal}$ and random coins ρ.

Let $\mathcal{F}$ be the set of PRFs $F : \{0,1\}^\kappa \times \{0,1\}^{\ell(\kappa)} \to [C]^t$ that take as input a PRF secret key and bit string and output a set cor such that $\mathsf{cor} \subseteq [n], |\mathsf{cor}| = t$. F outputs any set cor with probability $1/\binom{n}{t}$. Below, we describe the code of $\mathcal{M}$, which includes two PRFs F_1 and F_2 sampled uniformly at random from $\mathcal{F}$. $\mathcal{M}$ efficiently simulates $\mathcal{A}$ (in a way that is indistinguishable) to $\mathcal{B}_1$ and $\mathcal{B}_2$. Let $\mathcal{A}_1$ and $\mathcal{A}_2$ be $\mathcal{M}$'s simulation of $\mathcal{A}$ to $\mathcal{B}_1$ and $\mathcal{B}_2$, respectively. $\mathcal{M}$'s simulation is as follows:

1. $\mathcal{M}$ executes the first instance of $\mathcal{B}$ as $\mathcal{B}_1^{\mathcal{A}_1}(\mathsf{NIAChal}; \rho)$, where $\mathsf{NIAChal}$ is $\mathcal{M}$'s NIA challenge. $\mathcal{B}_1$ will output parameters $\mathsf{pp} = (\mathsf{par}, n, t+1, t_c)$ and coins $\rho_{\mathcal{A}}$.
2. $\mathcal{M}$ outputs an empty initial corruption set $\mathsf{cor} = \emptyset$. $\mathcal{B}_1$ will then output a threshold public key and public key shares for all participants $\mathsf{keys} = (\mathsf{PK}, \{\mathsf{PK}_i\}_{i=1}^n)$.
3. $\mathcal{M}$ checks that $\mathsf{VerifyPKs}(\mathsf{PK}, \{\mathsf{PK}_i\}_{i\in[n]}) = 1$; if not, $\mathcal{M}$ aborts. This abort event is the same as $\mathsf{InvalidKeysAbort}$.
4. $\mathcal{M}$ computes a corruption set $\mathsf{cor}_1 \leftarrow F_1(\rho_{\mathcal{A}}, \mathsf{pp}, \mathsf{keys})$.
5. For all $i \in \mathsf{cor}_1$, $\mathcal{M}$ queries the corruption oracle $(\mathsf{sk}_i, \{\mathsf{st}[\mathsf{sid}', i]\}_{\mathsf{sid}' \in \mathcal{ID}}) \leftarrow \mathcal{O}^{\mathsf{Corrupt}}(i)$, in the natural order, to $\mathcal{B}_1$, and writes the responses to Q_{st}. If $\mathcal{B}_1$ does not return t secret keys, $\mathcal{M}$ aborts. This abort event is the same as $\mathsf{NotAllKeysAbort}$.
6. $\mathcal{M}$ then executes a second instance of $\mathcal{B}$, as follows:
   (a) $\mathcal{M}$ executes $\mathcal{B}_2^{\mathcal{A}_2}(\mathsf{NIAChal}; \rho)$, providing the same challenge and random tape ρ as in $\mathcal{M}$'s execution of $\mathcal{B}_1$. $\mathcal{B}_2$ will output parameters $(\mathsf{par}, n, t+1, t_c)$ and coins $\rho_{\mathcal{A}}$. Because $\mathcal{B}_2$ is run on the same challenge and random coins ρ, $\mathcal{B}_2$ will output the same parameters and coins $\rho_{\mathcal{A}}$ as in $\mathcal{M}$'s execution of $\mathcal{B}_1$.
   (b) $\mathcal{M}$ outputs an empty initial corruption set $\mathsf{cor} = \emptyset$. $\mathcal{B}_2$ will then output a threshold public key and public key shares for all participants $(\mathsf{PK}, \{\mathsf{PK}_i\}_{i=1}^n)$. Because $\mathcal{B}_2$ is run on the same challenge and random coins ρ, $\mathcal{B}_2$ will output the same threshold public key and public key shares for all participants $(\mathsf{PK}, \{\mathsf{PK}_i\}_{i=1}^n)$ as $\mathcal{B}_1$.
   (c) $\mathcal{M}$ again checks that $\mathsf{VerifyPKs}(\mathsf{PK}, \{\mathsf{PK}_i\}_{i\in[n]}) = 1$; if not, $\mathcal{M}$ aborts. Note that because $\mathsf{VerifyPKs}$ output 1 in the execution of $\mathcal{B}_1$ and $\mathcal{B}_2$ outputs the same public keys $(\mathsf{PK}, \{\mathsf{PK}_i\}_{i\in[n]})$, this abort event will never occur. This abort event is the same as $\mathsf{InvalidKeysAbort}$.
   (d) $\mathcal{M}$ computes a corruption set $\mathsf{cor}_2 \leftarrow F_2(\rho_{\mathcal{A}}, \mathsf{pp}, \mathsf{keys})$.

(e) If $|\mathsf{cor}_1 \cup \mathsf{cor}_2| < t+1$, $\mathcal{M}$ aborts. We refer to this abort event as $\mathsf{BadEvent}$.
(f) For all $i \in \mathsf{cor}_2$, $\mathcal{M}$ queries the corruption oracle $(\mathsf{sk}_i, \{\mathsf{st}[\mathsf{sid}', i]\}_{\mathsf{sid}' \in \mathcal{ID}}) \leftarrow \mathcal{O}^{\mathsf{Corrupt}}(i)$, in the natural order, to $\mathcal{B}_2$, and writes the responses to Q_{st}. If $\mathcal{B}_2$ does not return t secret keys, $\mathcal{M}$ aborts. This abort event is the same as $\mathsf{NotAllKeysAbort}$.
(g) $\mathcal{M}$ aborts the instance of $\mathcal{B}_2$ after $\mathcal{B}_2$ has responded to $\mathcal{M}$'s corruption queries for cor_2.

7. $\mathcal{M}$ defines cor' to be the set of participant identifiers in Q_{st} and runs $\mathsf{sk} \leftarrow \mathcal{SS}.\mathsf{Recover}(t+1, \{(i, \mathsf{sk}_i)\}_{i \in \mathsf{cor}'})$.
8. $\mathcal{M}$ runs $\mathsf{Sign}(\mathsf{sk}, m^*)$ on message m^* to produce a valid signature σ^* under PK. $\mathcal{M}$ outputs (m^*, σ^*) to $\mathcal{B}_1$.

$\mathcal{M}$ is allowed to abort its simulation of $\mathcal{A}$ in a manner that depends on its execution. However, if $\mathcal{M}$ aborts $\mathcal{A}$ (such as if $\mathcal{M}$ terminates one simulation of $\mathcal{A}$ early), $\mathcal{M}$ also aborts all corresponding instances of $\mathcal{B}$ that interact with this particular simulation.

Analysis of $\mathcal{M}$'s Simulation to $\mathcal{B}$. We now show that $\mathcal{M}$'s simulation of $\mathcal{A}$ is convincing, i.e., indistinguishable from the real adversary $\mathcal{A}$. The output behavior of $\mathcal{A}$ and $\mathcal{M}$'s simulation of $\mathcal{A}$ ($\mathcal{A}_1$) is indistinguishable. Indeed, both $\mathcal{A}$ and $\mathcal{M}$, on input coins $\rho_{\mathcal{A}}$ and public parameters pp, output an empty initial corruption set. Then, on input public keys keys, they both run $\mathsf{VerifyPKs}$. If $\mathsf{VerifyPKs}$ returns 0 (i.e., $\mathsf{InvalidKeysAbort}$ occurs), both $\mathcal{A}$ and $\mathcal{M}$ abort. If $\mathsf{VerifyPKs}$ returns 1, $\mathcal{A}$ computes a corruption set using a random function $\mathsf{cor} \leftarrow R(\rho_{\mathcal{A}}, \mathsf{pp}, \mathsf{keys})$, which is computationally indistinguishable from $\mathcal{M}$ computing a corruption set using a pseudorandom function $\mathsf{cor}_1 \leftarrow F_1(\rho_{\mathcal{A}}, \mathsf{pp}, \mathsf{keys})$ by the security of the PRF. Both $\mathcal{A}$ and $\mathcal{M}$ then query $\mathcal{O}^{\mathsf{Corrupt}}$ on their corruption sets and abort if $\mathcal{B}_1$ does not return all t secret keys (i.e., $\mathsf{NotAllKeysAbort}$ occurs). This bad event is analyzed below. If $\mathcal{B}_1$ returns all secret keys, both $\mathcal{A}$ and $\mathcal{M}$ obtain $t+1$ secret keys and reconstruct the secret key sk. Finally, both $\mathcal{A}$ and $\mathcal{M}$ compute the forgery on an arbitrary message m^* by running $\mathsf{Sign}(\mathsf{sk}, m^*)$. Thus, the forgeries are distributed identically. If $\mathcal{A}$ and $\mathcal{M}$ are rewound on the same coins $\rho_{\mathcal{A}}$, the outputs of both remain the same. (Recall that the public parameters and public keys do not change, by assumption.) If $\mathcal{A}$ and $\mathcal{M}$ are rewound on different coins $\rho'_{\mathcal{A}} \neq \rho_{\mathcal{A}}$, the outputs of both remain indistinguishable. Therefore, $\mathcal{A}$ and $\mathcal{M}$ are computationally indistinguishable from $\mathcal{B}_1$'s point of view and thus $\mathsf{Adv}^{\mathsf{NIA}}_{\mathcal{B}_1^{\mathcal{A}_1}}(\kappa) = \mathsf{Adv}^{\mathsf{NIA}}_{\mathcal{B}_1^{\mathcal{A}}}(\kappa) - \mathsf{negl}(\kappa)$.

By the same reasoning, $\mathcal{M}$'s simulation of $\mathcal{A}$ ($\mathcal{A}_2$) to $\mathcal{B}_2$ is indistinguishable up to the point $\mathcal{M}$ corrupts participants in cor_2. $\mathcal{A}$ computes a corruption set using a random function $\mathsf{cor} \leftarrow R(\rho_{\mathcal{A}}, \mathsf{pp}, \mathsf{keys})$, which is computationally indistinguishable from $\mathcal{M}$ computing a corruption set using a pseudorandom function $\mathsf{cor}_2 \leftarrow F_2(\rho_{\mathcal{A}}, \mathsf{pp}, \mathsf{keys})$ by the security of the PRF. $\mathcal{M}$ aborts if $|\mathsf{cor}_1 \cup \mathsf{cor}_2| < t+1$. This $\mathsf{BadEvent}$ occurs when the two sets are equal and therefore $\mathcal{M}$ does not have enough secret keys to recover sk. In this case, $\mathcal{M}$ aborts with probability $1/\binom{n}{t}$. If $\mathcal{B}_2$ returns all t secret keys in cor_2, $\mathcal{M}$ then simply aborts the execution of $\mathcal{B}_2$. $\mathcal{A}$ and $\mathcal{M}$ are computationally indistinguishable to $\mathcal{B}_2$ up

to the point $\mathcal{M}$ aborts the execution of $\mathcal{B}_2$. Since $\mathcal{M}$ aborts the execution of $\mathcal{B}_2$ early, $\mathcal{B}_2$ does not need to output a NIA solution. However, $\mathcal{B}_2$ does need to simulate correctly in order for $\mathcal{M}$ to win,[20] returning all t of the requested secret keys. This is captured in the bounds for Theorem 1.

We first provide a formal analysis considering the case where $\mathcal{B}$ rewinds the adversary only once, and then show a trivial extension to the case where $\mathcal{B}$ makes an arbitrary number of rewinds. Formally, the probability in Theorem 1 is calculated as follows. First, note that the two instances of $\mathcal{B}$ are not independent. (They share the same input $\mathsf{NIAChal}$ and random coins ρ.) For ease of notation, let $\Pr[X^Y]$ denote the probability X^Y wins the NIA game (e.g., $\Pr[\mathcal{B}^{\mathcal{A}}] := \Pr[\mathsf{Game}^{\mathsf{NIA}}_{\mathcal{B},\mathcal{A}}(\kappa) = 1] = \mathsf{Adv}^{\mathsf{NIA}}_{\mathcal{B},\mathcal{A}}(\kappa)$). For a particular NIA instance I:

$$\Pr[\mathcal{M}^{\mathcal{B}} \mid I] \;=\; \Pr[\mathcal{B}_1^{\mathcal{A}_1} \mid I] \left(\Pr[\mathcal{B}_2 \text{ successfully simulates to } \mathcal{M} \mid I] - 1/\binom{n}{t} \right)$$

where I is the input to $\mathcal{B}_1$ and $\mathcal{B}_2$. Because $\Pr[\mathcal{B}_2 \text{ successfully simulates to } \mathcal{M} \mid I] \geq \Pr[\mathcal{B}_2^{\mathcal{A}_2} \mid I]$, we have (omitting the negligible term $\mathsf{negl}(\kappa)$):

$$\Pr[\mathcal{M}^{\mathcal{B}} \mid I] \;\geq\; \Pr[\mathcal{B}^{\mathcal{A}} \mid I] \left(\Pr[\mathcal{B}^{\mathcal{A}} \mid I] - 1/\binom{n}{t} \right)$$

By definition, $\Pr[\mathcal{M}^{\mathcal{B}}, I] = \Pr[\mathcal{M}^{\mathcal{B}} \mid I] \cdot \Pr[I]$, where $\Pr[I]$ denotes the probability that the instance I is sampled. Marginalizing out I, we have:

$$\sum_I \Pr[\mathcal{M}^{\mathcal{B}}, I] = \sum_I \Pr[\mathcal{M}^{\mathcal{B}} \mid I] \cdot \Pr[I]$$

By definition, $\Pr[\mathcal{M}^{\mathcal{B}}] = \sum_I \Pr[\mathcal{M}^{\mathcal{B}}, I]$, so we have:

$$\begin{aligned}
\Pr[\mathcal{M}^{\mathcal{B}}] &\geq \sum_I \Pr[\mathcal{B}^{\mathcal{A}} \mid I] \left(\Pr[\mathcal{B}^{\mathcal{A}} \mid I] - 1/\binom{n}{t} \right) \cdot \Pr[I] \\
&= \sum_I \left(\Pr[\mathcal{B}^{\mathcal{A}} \mid I]^2 \cdot \Pr[I] - 1/\binom{n}{t} \Pr[\mathcal{B}^{\mathcal{A}} \mid I] \cdot \Pr[I] \right) \\
&= \mathbb{E}[\Pr[\mathcal{B}^{\mathcal{A}} \mid I]^2] - 1/\binom{n}{t} \sum_I \Pr[\mathcal{B}^{\mathcal{A}}, I] \\
&\geq \mathbb{E}[\Pr[\mathcal{B}^{\mathcal{A}} \mid I]]^2 - 1/\binom{n}{t} \Pr[\mathcal{B}^{\mathcal{A}}] \;=\; \Pr[\mathcal{B}^{\mathcal{A}}]^2 - 1/\binom{n}{t} \Pr[\mathcal{B}^{\mathcal{A}}]
\end{aligned}$$

where the expectation is taken over the $I \leftarrow\!\!\$\ \mathsf{InstGen}$, where $\mathsf{InstGen}$ is a randomized instance generator algorithm, and the last inequality is due to Jensen's inequality.

For the case where $\mathcal{B}$ rewinds the adversary more than once, we first note that the number of rewinds is bounded by the total running time of $\mathcal{B}$, which

[20] Unless $\mathcal{B}$ outputs a forgery directly, without executing $\mathcal{A}$, in which case $\mathcal{M}$ is trivially successful with greater probability than the bound in Theorem 1.

we denote as τ. Then, the only change in the discussion above is that $\mathcal{B}$ may answer the adversary's corruption queries only in one rewinding thread. Hence, the probability that $\mathsf{BadEvent}$ occurs becomes $\tau/\binom{n}{t}$. This gives the bound in Theorem 1. The runtime of $\mathcal{M}$ consists of two executions of $\mathcal{B}$ and a constant number of operations. Thus, $\mathsf{Time}_{\mathcal{M}}(\kappa) = 2\tau + \mathsf{poly}(\kappa)$. □

5 Generalization to Adaptive Security for $\lfloor t/\ell \rfloor < t_c \leq t$

In this section, we prove a generalization of Theorem 1, which admits any corruption threshold t_c such that $\lfloor t/\ell \rfloor < t_c \leq t$ for any $\ell \in \mathbb{N}$. This result quantifies the extent to which the metareduction in the proof of Theorem 1 succeeds for lower corruption profiles. Recall that in the proof of Theorem 1, the metareduction pseudorandomly chooses two sets, cor_1 and cor_2, of t parties to corrupt and receives their secret keys. The metareduction succeeds in reconstructing the overall secret key if it obtains at least a threshold number of secret keys, i.e., if $|\mathsf{cor}_1 \cup \mathsf{cor}_2| \geq t + 1$.

Let us consider what happens for smaller corruption thresholds $t_c < t$. First, observe that if $t_c \leq \lfloor t/2 \rfloor$, then $|\mathsf{cor}_1 \cup \mathsf{cor}_2|$ will never meet the required threshold $t + 1$ and the metareduction will fail with probability 1 (because the combined set is maximally of size t). Thus, we are first interested in the degree to which the metareduction succeeds for corruption thresholds t_c in the range $\lfloor t/2 \rfloor < t_c \leq t$. Intuitively, as the corruption threshold t_c (and therefore the size of the sets) tends towards t, the metareduction succeeds with higher probability because the set union is more likely to meet or exceed the threshold. Formally, we have the following theorem. Theorem 1 can be seen as a special case, where $t_c = t$.

Theorem 2. *Theorem 1 holds for t_c-adaptive unforgeability for $\lfloor t/2 \rfloor < t_c \leq t$, where the metareduction $\mathcal{M}$ breaks the NIA assumption with non-negligible probability:*

$$\mathsf{Adv}^{\mathsf{NIA}}_{\mathcal{M}^{\mathcal{B}}}(\kappa) \;\geq\; \mathsf{Adv}^{\mathsf{NIA}}_{\mathcal{B}^{\mathcal{A}}}(\kappa) \left(\mathsf{Adv}^{\mathsf{NIA}}_{\mathcal{B}^{\mathcal{A}}}(\kappa) - \tau \sum_{k=t_c}^{t} \frac{\binom{n}{k}\binom{k}{t_c}\binom{t_c}{2t_c-k}}{\binom{n}{t_c}^2} \right) - \mathsf{negl}(\kappa).$$

where τ is the runtime of $\mathcal{B}$.

To prove Theorem 2, we rely on the following lemma with $t_c = t_1 = t_2$. We prove Lemma 1 and Theorem 2 in Appendix F.

Lemma 1. *Let A be a finite set of size n, and let $B_1 \subseteq A, B_2 \subseteq A$ be independent, randomly chosen subsets such that $|B_1| = t_1$ and $|B_2| = t_2$. Let $\Pr[k]$ denote the probability that the union $B_1 \cup B_2$ is of size exactly k for $0 \leq k \leq n$. Then:*

$$\Pr[k] = \begin{cases} \frac{\binom{n}{k}\binom{k}{t_1}\binom{t_1}{t_1+t_2-k}}{\binom{n}{t_1}\binom{n}{t_2}}, & \text{if } \max\{t_1, t_2\} \leq k \leq \min\{n, t_1 + t_2\} \\ 0, & \text{otherwise} \end{cases} \tag{1}$$

Proof Outline of Theorem 2. The metareduction is the same as in the proof of Theorem 1, except the corruption threshold is any $\lfloor t/2 \rfloor < t_c \leq t$. Recall that

the metareduction in Theorem 1 succeeds as long as the sets $\mathsf{cor}_1, \mathsf{cor}_2$ that $\mathcal{M}$ pseudorandomly selects are such that $|\mathsf{cor}_1 \cup \mathsf{cor}_2| \geq t+1$. Otherwise, $\mathcal{M}$ aborts when $|\mathsf{cor}_1 \cup \mathsf{cor}_2| < t+1$.

Let $U = \mathsf{cor}_1 \cup \mathsf{cor}_2$ and $k = |U|$. Because $|\mathsf{cor}_1| = |\mathsf{cor}_1| = t_c$, k is at least t_c. $\mathcal{M}$ aborts when $k < t+1$. The probability of this can be defined by first counting the number of possible ways to sample U from $[n]$, then the number of ways to sample cor_1 from U, then the number of ways to sample cor_2 so that the union of cor_2 with cor_1 is of size exactly k. This is exactly what is defined by Lemma 1. As such, the probability that $\mathcal{M}$ aborts is defined by the union bound of Equation 1, for $t_c \leq k \leq t$, which gives the probability in Theorem 2.

Finally, we give our generalized result for $\lfloor t/\ell \rfloor < t_c \leq t$ for any $\ell \in \mathbb{N}$. It boils down to a similar argument about the corruption sets: the metareduction corrupts t_c parties in each of the ℓ instances of $\mathcal{B}$ and succeeds in recovering sk if $|\mathsf{cor}_1 \cup \cdots \cup \mathsf{cor}_\ell| \geq t+1$. The combinatorial argument is more involved (Lemma 2), and is proven with Theorem 3 in Appendix F.

Theorem 3. *Theorem 1 holds for t_c-adaptive unforgeability for $\lfloor t/\ell \rfloor < t_c \leq t$, $\ell \in \mathbb{N}$, where the metareduction $\mathcal{M}$ breaks the* NIA *assumption with non-negligible probability:*

$$\mathsf{Adv}^{\mathsf{NIA}}_{\mathcal{M}\mathcal{B}}(\kappa) \;\geq\; \mathsf{Adv}^{\mathsf{NIA}}_{\mathcal{B}\mathcal{A}}(\kappa)\overbrace{\left(\mathsf{Adv}^{\mathsf{NIA}}_{\mathcal{B}\mathcal{A}}(\kappa)\left(\cdots\left(\mathsf{Adv}^{\mathsf{NIA}}_{\mathcal{B}\mathcal{A}}(\kappa) - \tau\sum_{k=t_c}^{t}\frac{\binom{n}{k}\sum_{i=0}^{k}(-1)^i\binom{k}{i}\binom{k-i}{t_c}^\ell}{\binom{n}{t_c}^\ell}\right)\right)\right)}^{\ell} - \mathsf{negl}(\kappa)$$

and has runtime $\mathsf{Time}_{\mathcal{M}}(\kappa) = \ell\tau + \mathsf{poly}(\kappa)$, where τ is the runtime of $\mathcal{B}$.

6 Impossibility of $\lfloor t/2 \rfloor < t_c \leq t$ Adaptive Security for Schnorr Under (A)OMDL or AOMCDH Using Rewinding

We now state and prove our second impossibility result: even if we allow the *interactive* assumption OMDL, it is impossible to define a fully black-box adaptive reduction for $\lfloor t/2 \rfloor < t_c \leq t$ in the ROM that rewinds the adversary and either changes the coins or programs the random oracle on at least one point, or both. Similar to our first impossibility result, we require the reduction to run the adversary on consistent public parameters and public keys. We extend this result to the AOMDL and AOMCDH [10] assumptions in Appendix H.

We define $\mathsf{KeyUTS}_{\mathcal{SS},f,\mathsf{VerifyPKs}}.\mathsf{TSchnorr}[\mathsf{H}]$ to be a key-unique threshold signature scheme as in Definition 4, parameterized by a random oracle H, such that signatures output by $\mathsf{KeyUTS}_{\mathcal{SS},f,\mathsf{VerifyPKs}}.\mathsf{TSchnorr}[\mathsf{H}]$ are compatible with the Schnorr verification algorithm as in Definition 8.

Theorem 4. *Assume there exists a fully black-box efficient reduction $\mathcal{B}$ that converts any adversary $\mathcal{A}$ against the t_c-adaptive unforgeability for $\lfloor t/2 \rfloor < t_c \leq t$ of a key-unique threshold signature scheme $\mathsf{KeyUTS}_{\mathcal{SS},f,\mathsf{VerifyPKs}}.\mathsf{TSchnorr}[\mathsf{H}]$ into an adversary against the q-OMDL assumption, assuming the security of*

pseudorandom functions. Assume further that $\mathcal{B}$: (1) runs an execution of $\mathcal{A}$ with coins $\rho_{\mathcal{A}}$ and returns a value x_i for $\mathcal{A}$'s i^{th} query to the random oracle H, and (2) upon receiving the forgery from $\mathcal{A}$, rewinds $\mathcal{A}$ and repeats (1) using $\rho'_{\mathcal{A}}$ and $(x'_1, \ldots, x'_{q_h})$ such that $(\rho'_{\mathcal{A}}, x'_1, ..., x'_{q_h}) \neq (\rho_{\mathcal{A}}, x_1, ..., x_{q_h})$, where q_h is the maximum number of queries $\mathcal{A}$ makes to H in either of its first two executions, and that (3) $\mathcal{B}$ provides all instances of $\mathcal{A}$ with the same public parameters and public keys. Assume $\mathcal{B}$ has success probability $\mathsf{Adv}^{\mathsf{NIA}}_{\mathcal{B},\mathcal{A}}(\kappa)$ for a given $\mathcal{A}$ and runtime τ. Then, there exists a successful (possibly inefficient) adversary $\mathcal{A}$ against the t_c-adaptive unforgeability of $\mathsf{KeyUTS}_{\mathcal{SS},f,\mathsf{VerifyPKs}}.\mathsf{TSchnorr}[\mathsf{H}]$ and an efficient metareduction $\mathcal{M}$ that breaks the $(q+1)$-OMDL assumption with non-negligible probability:

$$\mathsf{Adv}^{(q+1)\text{-omdl}}_{\mathcal{M},\mathcal{B}}(\kappa) \;\geq\; \mathsf{Adv}^{q\text{-omdl}}_{\mathcal{B},\mathcal{A}}(\kappa) - \tau \sum_{k=t_c}^{t} \frac{\binom{n}{k}\binom{k}{t_c}\binom{t_c}{2t_c-k}}{\binom{n}{t_c}^2} - \mathsf{negl}(\kappa)$$

and runtime $\mathsf{Time}_{\mathcal{M}}(\kappa) = \tau + \mathsf{poly}(\kappa)$.

Proof (of Theorem 4).

Description of Adversary $\mathcal{A}$. We give a description of one particular (and inefficient) adversary $\mathcal{A}$. Recall that $\mathcal{B}$ initializes any adversary with coins $\rho_{\mathcal{A}}$, and the adversary runs deterministically with respect to $\rho_{\mathcal{A}}$. Below, we describe the code of $\mathcal{A}$, which includes a random $r^* \leftarrow_{\$} \mathbb{Z}_p$ and a random function $R : \{0,1\}^{\ell(\kappa)} \to [C]^t$, where $[C]^t := \{\mathsf{cor} \subseteq [n] \mid |\mathsf{cor}| = t\}$, that takes as input a bit string and outputs a set cor such that $\mathsf{cor} \subseteq [n], |\mathsf{cor}| = t$.

1. $\mathcal{B}$ initializes $\mathcal{A}$ on the following values: (1) coins $\rho_{\mathcal{A}}$ and (2) public parameters $\mathsf{pp} = (\mathsf{par}, n, t+1, t_c)$.
2. $\mathcal{A}$ returns an empty initial corruption set $\mathsf{cor} \leftarrow \emptyset$. $\mathcal{B}$ then outputs public keys $\mathsf{keys} = (\mathsf{PK}, \{\mathsf{PK}_i\}_{i=1}^n)$.
3. $\mathcal{A}$ checks that $\mathsf{VerifyPKs}(\mathsf{PK}, \{\mathsf{PK}_i\}_{i\in[n]}) = 1$; if not, $\mathcal{A}$ aborts. We refer to this abort event as $\mathsf{InvalidKeysAbort}$.
4. $\mathcal{A}$ computes $c^* = \mathsf{H}(R^*, \mathsf{PK}, m^*)$ on $R^* = g^{r^*}$ and an arbitrary message m^*, then computes a corruption set $\mathsf{cor} \leftarrow R(\rho_{\mathcal{A}}, \mathsf{pp}, \mathsf{keys}, c^*)$.
5. For all $i \in \mathsf{cor}$, $\mathcal{A}$ queries the corruption oracle $(\mathsf{sk}_i, \{\mathsf{st}[\mathsf{sid}', i]\}_{\mathsf{sid}'\in\mathcal{ID}}) \leftarrow \mathcal{O}^{\mathsf{Corrupt}}(i)$, in the natural order, to $\mathcal{B}$. Recall that $\mathsf{st}[\mathsf{sid}', i]$ is the state of participant i for signing session sid'. If $\mathcal{B}$ does not return t secret keys, $\mathcal{A}$ aborts. We refer to this bad event as $\mathsf{NotAllKeysAbort}$.
6. $\mathcal{A}$ then solves for an arbitrary $\mathsf{sk}_{i'}, i' \notin \mathsf{cor}$, by brute-force search, with respect to $\mathsf{PK}_{i'}$.
7. $\mathcal{A}$ defines cor' to be the set of participant identifiers in $\mathsf{cor} \cup i'$ and runs $\mathsf{sk} \leftarrow \mathcal{SS}.\mathsf{Recover}(t+1, \{(i, \mathsf{sk}_i)\}_{i\in\mathsf{cor}'})$.
8. $\mathcal{A}$ computes $z^* = r^* + c^*\mathsf{sk}$ and outputs $(m^*, \sigma^* = (R^*, z^*))$.

Description of $\mathcal{M}$. The metareduction $\mathcal{M}$ is initialized by the q-OMDL game as $\mathcal{M}((\mathbb{G}, p, g), (X_0, \ldots, X_q); \rho')$, with input challenge $((\mathbb{G}, p, g), (X_0, \ldots, X_q))$ and coins ρ'. Below, we describe the code of $\mathcal{M}$, which includes random coins

ρ that $\mathcal{M}$ will provide as input to $\mathcal{B}$. $\mathcal{B}$ is initialized on the challenge $((\mathbb{G}, p, g), (X_0, \ldots, X_{q-1}))$ and random coins ρ.

Let $\mathcal{F}$ be the set of PRFs $F : \{0,1\}^\kappa \times \{0,1\}^{\ell(\kappa)} \to [C]^t$ that take as input a PRF secret key and bit string and output a set cor such that $\mathsf{cor} \subseteq [n], |\mathsf{cor}| = t$. F outputs any set cor with probability $1/\binom{n}{t}$. Below, we describe the code of $\mathcal{M}$, which includes a PRF F sampled uniformly at random from $\mathcal{F}$. $\mathcal{M}$ efficiently simulates $\mathcal{A}$ (in a way that is indistinguishable) to $\mathcal{B}$. Let $\mathcal{A}'$ be $\mathcal{M}$'s simulation of $\mathcal{A}$ to $\mathcal{B}$. $\mathcal{M}$'s simulation is as follows.

$\mathcal{M}$ maintains the following tables and a counter: (1) a table Q_{st} to maintain corrupted parties' states across different executions of $\mathcal{A}$; (2) a table Q_{odl} to maintain the set of DL solution oracle queries and responses; (3) a table Q_{sig} to maintain forgeries from simulated executions of $\mathcal{A}$; and (4) a counter ctr to count the number of DL solution oracle queries that $\mathcal{B}$ makes.

Responding to $\mathcal{O}^{\mathsf{dlsol}'}$ Queries. $\mathcal{M}$ provides the oracle $\mathcal{O}^{\mathsf{dlsol}'}$ to $\mathcal{B}$, simulating the real discrete logarithm solution oracle $\mathcal{O}^{\mathsf{dlsol}}$. When $\mathcal{B}$ queries $\mathcal{O}^{\mathsf{dlsol}'}(Y)$ on some $Y \in \mathbb{G}$, $\mathcal{M}$ does the following:

- If $\mathsf{ctr} = q - 1$ (i.e., $\mathcal{B}$ has reached its allowed number of DL solution queries), $\mathcal{M}$ returns $\perp$.
- Otherwise, $\mathcal{M}$ increments ctr by one and queries its own DL solution oracle $\mathcal{O}^{\mathsf{dlsol}}$ on the same input, receiving $y \leftarrow \mathcal{O}^{\mathsf{dlsol}}(Y)$. It then updates Q_{odl} with the entry $Q_{\mathsf{odl}} \leftarrow \{(Y, y)\}$.
- Finally, $\mathcal{M}$ outputs y to $\mathcal{B}$.

How $\mathcal{M}$ Simulates the Adversary $\mathcal{A}$. We now describe how $\mathcal{M}$ simulates $\mathcal{A}$ in a way that is indistinguishable to $\mathcal{B}$. $\mathcal{M}$ performs the below steps each time $\mathcal{B}$ runs $\mathcal{A}$.

1. $\mathcal{M}$ executes $\mathcal{B}$ as $\mathcal{B}^{\mathcal{A}', \mathcal{O}^{\mathsf{dlsol}'}}((\mathbb{G}, p, g), (X_0, \ldots, X_{q-1}); \rho)$, where $(\mathbb{G}, p, g), (X_0, \ldots, X_{q-1})$ is a subset of $\mathcal{M}$'s $(q+1)$-OMDL challenges. $\mathcal{B}$ will output parameters $\mathsf{pp} = (\mathsf{par}, n, t+1, t_c)$ and coins $\rho_{\mathcal{A}}$.
2. $\mathcal{M}$ outputs an empty initial corruption set $\mathsf{cor} = \emptyset$. $\mathcal{B}$ will then output a threshold public key and public key shares for all participants $\mathsf{keys} = (\mathsf{PK}, \{\mathsf{PK}_i\}_{i=1}^n)$.
3. $\mathcal{M}$ checks that $\mathsf{VerifyPKs}(\mathsf{PK}, \{\mathsf{PK}_i\}_{i \in [n]}) = 1$; if not, $\mathcal{M}$ aborts. This abort event is the same as $\mathsf{InvalidKeysAbort}$.
4. $\mathcal{M}$ sets $R^* \leftarrow X_q$ from the last of its OMDL challenges, computes $c^* = \mathsf{H}(R^*, \mathsf{PK}, m^*)$, and computes a corruption set $\mathsf{cor} \leftarrow F(\rho_{\mathcal{A}}, \mathsf{pp}, \mathsf{keys}, c^*)$.
5. For all $i \in \mathsf{cor}$, $\mathcal{M}$ queries the corruption oracle $(\mathsf{sk}_i, \{\mathsf{st}[\mathsf{sid}', i]\}_{\mathsf{sid}' \in \mathcal{ID}}) \leftarrow \mathcal{O}^{\mathsf{Corrupt}}(i)$, in the natural order, to $\mathcal{B}$, and writes the responses to Q_{st}. If $\mathcal{B}$ does not return t secret keys, $\mathcal{M}$ aborts. This abort event is the same as $\mathsf{NotAllKeysAbort}$.
6. If this is the first time that $\mathcal{B}$ has executed $\mathcal{A}$, $\mathcal{M}$ does the following:
   - Computes $Z^* = R^* \cdot \mathsf{PK}^{c^*}$.
   - Queries $z^* \leftarrow \mathcal{O}^{\mathsf{dlsol}}(Z^*)$.

– Stores (Z^*, z^*) in Q_{odl} and (c^*, z^*) in Q_{sig}.

7. Otherwise, if $\mathcal{M}$ is in its second execution, then $\mathcal{M}$ checks Q_{st} to determine if it has corrupted at least $t+1$ distinct parties between its executions, meaning that Q_{st} holds at least $t+1$ distinct secret key shares. If not, $\mathcal{M}$ aborts. We refer to this abort event as $\mathsf{BadEvent}$. Otherwise, let cor' be the set of participant identifiers in Q_{st}. $\mathcal{M}$ runs $\mathsf{sk} = \mathcal{SS}.\mathsf{Recover}(t+1, \{(i, \mathsf{sk}_i)\}_{i \in \mathsf{cor}'})$.
   (a) $\mathcal{M}$ looks up (c^{**}, z^{**}) in Q_{sig}, where z^{**} is the forgery returned by $\mathcal{M}$ in its first execution, and computes $x_q = z^{**} - c^{**}\mathsf{sk}$.
   (b) $\mathcal{M}$ computes $z^* = x_q + c^*\mathsf{sk}$, where $c^* = \mathsf{H}(R^*, \mathsf{PK}, m^*)$ is for this execution (with the same R^* and m^*).
8. Otherwise, if $\mathcal{M}$ is in any subsequent execution, $\mathcal{M}$ generates z^* using its knowledge of sk directly as $z^* = x_q + c^*\mathsf{sk}$, where $c^* = \mathsf{H}(R^*, \mathsf{PK}, m^*)$ is for this execution (with the same R^* and m^*).
9. $\mathcal{M}$ returns $(m^*, \sigma^* = (R^*, z^*))$.

$\mathcal{M}$ is allowed to abort its simulation of $\mathcal{A}$ in a manner that depends on its execution. However, if $\mathcal{M}$ aborts $\mathcal{A}$, it also aborts the execution of $\mathcal{B}$.

Analysis of $\mathcal{M}$'s Simulation to $\mathcal{B}$. We now show that $\mathcal{M}$'s simulation of $\mathcal{A}$ is convincing, i.e., indistinguishable from the real adversary $\mathcal{A}$.

First, note that $\mathcal{M}$ simply serves as an intermediary for the DL solution oracle in the OMDL game for $\mathcal{B}$, querying its own DL solution oracle on the same inputs provided by $\mathcal{B}$ and relaying the results to $\mathcal{B}$.

The output behavior of $\mathcal{A}$ and $\mathcal{M}$'s simulation of $\mathcal{A}$ ($\mathcal{A}'$) is indistinguishable. The analysis is largely the same as in the proof of Theorem 1, with the following exceptions. Because we assume the rewind is complete, i.e., that $\mathcal{B}$ returns all messages to $\mathcal{A}$ up to the point $\mathcal{A}$ forges, the bad event $\mathsf{NotAllKeysAbort}$ will not occur (as in this event, $\mathcal{B}$ causes $\mathcal{A}$ to terminate early).

For each execution of $\mathcal{A}$, $\mathcal{A}$ computes the corruption set $\mathsf{cor} \leftarrow R(\rho_{\mathcal{A}}, \mathsf{pp}, \mathsf{keys}, c^*)$ using a random function R, where its random oracle query is $c^* = \mathsf{H}(R^* = g^{r^*}, \mathsf{PK}, m^*)$. This is computationally indistinguishable from $\mathcal{M}$ computing the corruption set $\mathsf{cor} \leftarrow F(\rho_{\mathcal{A}}, \mathsf{pp}, \mathsf{keys}, c^*)$ using a pseudorandom function F, where its random oracle query is $c^* = \mathsf{H}(R^* = X_q, \mathsf{PK}, m^*)$ for uniformly random X_q, by the security of the PRF. Hence, on its second execution, $\mathcal{A}$'s corruption set changes, based on either $\rho_{\mathcal{A}}$ changing or c^* changing, and $\mathcal{M}$'s corruption set does as well. In the event of $\mathsf{BadEvent}$, $\mathcal{M}$ aborts when it obtains two corruption sets such that $|\mathsf{cor}_1 \cup \mathsf{cor}_2| < t+1$, across the first two executions of $\mathcal{A}$, which occurs with probability $\binom{n}{t}$. Else, $\mathsf{cor}_1 \neq \mathsf{cor}_2$ and hence $|\mathsf{cor}_1 \cup \mathsf{cor}_2| \geq t+1$.

With the exception of the abort event described above, both $\mathcal{A}$ and $\mathcal{M}$ obtain $t+1$ secret keys and reconstruct the secret key sk, which passes $f(0, \mathsf{sk}) = \mathsf{PK}$. Finally, both $\mathcal{A}$ and $\mathcal{M}$ compute the forgery on an arbitrary message m^* by directly computing a Schnorr signature on m^* using sk. Thus, the forgeries are distributed identically. Therefore, $\mathcal{A}$ and $\mathcal{M}$ are computationally indistinguishable from $\mathcal{B}$'s point of view and thus $\mathsf{Adv}^{\mathsf{NIA}}_{\mathcal{B},\mathcal{A}'}(\kappa) = \mathsf{Adv}^{\mathsf{NIA}}_{\mathcal{B},\mathcal{A}}(\kappa) - \mathsf{negl}(\kappa)$.

If $\mathcal{B}$ returns a q-OMDL solution $(x_0, \ldots, x_{q-1})$ with non-negligible probability, having made up to $q-1$ DL solution queries, then $\mathcal{M}$ returns a $(q+1)$-OMDL solution $(x_0, \ldots, x_q)$ with non-negligible probability defined in Theorem 4, having

made one more DL solution query than $\mathcal{B}$, but strictly less than $q+1$. The runtime of $\mathcal{M}$ consists of one execution of $\mathcal{B}$ and a constant number of operations. Thus, $\mathsf{Time}_{\mathcal{M}}(\kappa) = \tau + \mathsf{poly}(\kappa)$. □

Acknowledgements. We would like to thank Alistair Stewart and the anonymous reviewers, whose valuable feedback has improved the quality of this paper. In particular, we are thankful for feedback on the definition of key-uniqueness and on reductions that rewind the adversary. We would also like to thank Jonathan Katz for his helpful discussion.

References

[1] M. Abe and S. Fehr. "Adaptively Secure Feldman VSS and Applications to Universally-Composable Threshold Cryptography". In: *Advances in Cryptology - CRYPTO 2004, 24th Annual International CryptologyConference, Santa Barbara, California, USA, August 15-19, 2004, Proceedings*. Ed. by M. K. Franklin. Vol. 3152. Lecture Notes in Computer Science. Springer, 2004, pp. 317–334. DOI: 10.1007/978-3-540-28628-8_20. URL: https://doi.org/10.1007/978-3-540-28628-8_20.

[2] J. F. Almansa, I. Damgård, and J. B. Nielsen. "Simplified Threshold RSA with Adaptive and Proactive Security". In: *EUROCRYPT 2006, 25th Annual International Conference on the Theory and Applications of Cryptographic Techniques, St. Petersburg, Russia, May 28 - June 1, 2006*. Ed. by S. Vaudenay. Vol. 4004. LNCS. Springer, 2006, pp. 593–611.

[3] M. H. Au, W. Susilo, and Y. Mu. "Constant-Size Dynamic k-TAA". In: *Security and Cryptography for Networks, 5th International Conference, SCN 2006, Maiori, Italy, September 6-8, 2006, Proceedings*. Ed. by R. D. Prisco and M. Yung. Vol. 4116. LNCS. Springer, 2006, pp. 111–125. DOI: 10.1007/11832072_8. URL: https://doi.org/10.1007/11832072_8.

[4] R. Bacho, Y. Chen, J. Loss, S. Tessaro, and C. Zhu. "Adaptively Secure Partially Non-Interactive Threshold Schnorr Signatures in the AGM". In: *IACR Cryptol. ePrint Arch.* (2025), p. 1953. URL: https://eprint.iacr.org/2025/1953.

[5] R. Bacho, S. Das, J. Loss, and L. Ren. "Adaptively Secure Three-Round Threshold Schnorr Signatures from DDH". In: *Advances in Cryptology - CRYPTO 2025 - 45th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 17-21, 2025, Proceedings, Part VI*. Ed. by Y. T. Kalai and S. F. Kamara. Vol. 16005. Lecture Notes in Computer Science. Springer, 2025, pp. 390–422. DOI: 10.1007/978-3-032-01887-8_13. URL: https://doi.org/10.1007/978-3-032-01887-8_13.

[6] R. Bacho, S. Das, J. Loss, and L. Ren. "Glacius: Threshold Schnorr Signatures from DDH with Full Adaptive Security". In: *Advances in Cryptology - EUROCRYPT 2025 - 44th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Madrid, Spain, May 4-8, 2025, Proceedings, Part II*. Ed. by S. Fehr and P. Fouque. Vol. 15602. Lecture Notes in Computer Science. Springer, 2025, pp. 304–334. DOI: 10.1007/978-3-031-91124-8_11. URL: https://doi.org/10.1007/978-3-031-91124-8_11.

[7] R. Bacho, C. Lenzen, J. Loss, S. Ochsenreither, and D. Papachristoudis. "GRand-Line: Adaptively Secure DKG and Randomness Beacon with (Log-)Quadratic Communication Complexity". In: *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security, CCS 2024, Salt Lake City, UT, USA, October 14-18, 2024*. Ed. by B. Luo, X. Liao, J. Xu, E. Kirda, and D. Lie. ACM, 2024, pp. 941–955. DOI: 10.1145/3658644.3690287. URL: https://doi.org/10.1145/3658644.3690287.

[8] R. Bacho and J. Loss. "On the Adaptive Security of the Threshold BLS Signature Scheme". In: *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security, CCS 2022, Los Angeles, CA, USA, November 7-11, 2022*. Ed. by H. Yin, A. Stavrou, C. Cremers, and E. Shi. ACM, 2022, pp. 193–207. DOI: 10.1145/3548606.3560656. URL: https://doi.org/10.1145/3548606.3560656.

[9] R. Bacho, J. Loss, G. Stern, and B. Wagner. "HARTS: High-Threshold, Adaptively Secure, and Robust Threshold Schnorr Signatures". In: *Advances in Cryptology - ASIACRYPT 2024 - 30th International Conference on the Theory and Application of Cryptology and Information Security, Kolkata, India, December 9-13, 2024, Proceedings, Part III*. Ed. by K. Chung and Y. Sasaki. Vol. 15486. Lecture Notes in Computer Science. Springer, 2024, pp. 104–140. DOI: 10.1007/978-981-96-0891-1_4. URL: https://doi.org/10.1007/978-981-96-0891-1_4.

[10] R. Bacho, J. Loss, S. Tessaro, B. Wagner, and C. Zhu. "Twinkle: Threshold Signatures from DDH with Full Adaptive Security". In: *Advances in Cryptology - EUROCRYPT 2024 - 43rd Annual International Conference on the Theory and Applications of Cryptographic Techniques, Zurich, Switzerland, May 26-30, 2024, Proceedings, Part I*. Ed. by M. Joye and G. Leander. Vol. 14651. Lecture Notes in Computer Science. Springer, 2024, pp. 429–459. DOI: 10.1007/978-3-031-58716-0_15. URL: https://doi.org/10.1007/978-3-031-58716-0_15.

[11] P. Baecher, C. Brzuska, and M. Fischlin. "Notions of Black-Box Reductions, Revisited". In: *Advances in Cryptology - ASIACRYPT 2013 - 19th International Conference on the Theory and Application of Cryptology and Information Security, Bengaluru, India, December 1-5, 2013, Proceedings, Part I*. Ed. by K. Sako and P. Sarkar. Vol. 8269. Lecture Notes in Computer Science. Springer, 2013, pp. 296–315. DOI: 10.1007/978-3-642-42033-7_16. URL: https://doi.org/10.1007/978-3-642-42033-7_16.

[12] R. Baecker, P. Gerhart, D. L. Calsi, L. Russo, D. Schröder, and A. Yerukhimovich. "Fully Adaptive FROST in the Algebraic Group Model From Falsifiable Assumptions". In: *IACR Cryptol. ePrint Arch.* (2025), p. 1950. URL: https://eprint.iacr.org/2025/1950.

[13] F. Baldimtsi and A. Lysyanskaya. "On the Security of One-Witness Blind Signature Schemes". In: *ASIACRYPT 2013 - 19th International Conference on the Theory and Application of Cryptology and Information Security, Bengaluru, India, December 1-5, 2013*. Ed. by K. Sako and P. Sarkar. Vol. 8270. LNCS. Springer, 2013, pp. 82–99. DOI: 10.1007/978-3-642-42045-0_5. URL: https://doi.org/10.1007/978-3-642-42045-0_5.

[14] B. Bauer, G. Fuchsbauer, and J. Loss. "A Classification of Computational Assumptions in the Algebraic Group Model". In: *Advances in Cryptology - CRYPTO 2020 - 40th Annual International Cryptology Conference, CRYPTO 2020, Santa Barbara, CA, USA, August 17-21, 2020, Proceedings, Part II*. Ed. by D. Micciancio and T. Ristenpart. Vol. 12171. Lecture Notes in Computer Science.

Springer, 2020, pp. 121–151. DOI: 10.1007/978-3-030-56880-1_5. URL: https://doi.org/10.1007/978-3-030-56880-1_5.

[15] B. Bauer, G. Fuchsbauer, and A. Plouviez. "The One-More Discrete Logarithm Assumption in the Generic Group Model". In: *ASIACRYPT 2021, Singapore, December 6-10, 2021*. Ed. by M. Tibouchi and H. Wang. Vol. 13093. LNCS. Springer, 2021, pp. 587–617.

[16] C. Baum et al. *Cryptographers' feedback on the EU digital identity's ARF*. 2024. URL: https://github.com/user-attachments/files/15904122/cryptographers-feedback.pdf.

[17] M. Bellare, E. C. Crites, C. Komlo, M. Maller, S. Tessaro, and C. Zhu. "Better than Advertised Security for Non-interactive Threshold Signatures". In: *Advances in Cryptology - CRYPTO 2022 - 42nd Annual International Cryptology Conference, CRYPTO 2022, Santa Barbara, CA, USA, August 15-18, 2022, Proceedings, Part IV*. Ed. by Y. Dodis and T. Shrimpton. Vol. 13510. Lecture Notes in Computer Science. Springer, 2022, pp. 517–550. DOI: 10.1007/978-3-031-15985-5_18. URL: https://doi.org/10.1007/978-3-031-15985-5_18.

[18] M. Bellare, D. Hofheinz, and S. Yilek. "Possibility and Impossibility Results for Encryption and Commitment Secure under Selective Opening". In: *Advances in Cryptology - EUROCRYPT 2009, 28th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Cologne, Germany, April 26-30, 2009. Proceedings*. Ed. by A. Joux. Vol. 5479. Lecture Notes in Computer Science. Springer, 2009, pp. 1–35. DOI: 10.1007/978-3-642-01001-9_1. URL: https://doi.org/10.1007/978-3-642-01001-9_1.

[19] M. Bellare, C. Namprempre, D. Pointcheval, and M. Semanko. "The One-More-RSA-Inversion Problems and the Security of Chaum's Blind Signature Scheme". In: *J. Cryptol.* 16.3 (2003), pp. 185–215. DOI: 10.1007/S00145-002-0120-1. URL: https://doi.org/10.1007/s00145-002-0120-1.

[20] F. Benhamouda, S. Halevi, H. Krawczyk, Y. Ma, and T. Rabin. "SPRINT: High-Throughput Robust Distributed Schnorr Signatures". In: *Advances in Cryptology - EUROCRYPT 2024 - 43rd Annual International Conference on the Theory and Applications of Cryptographic Techniques, Zurich, Switzerland, May 26-30, 2024, Proceedings, Part V*. Ed. by M. Joye and G. Leander. Vol. 14655. Lecture Notes in Computer Science. Springer, 2024, pp. 62–91. DOI: 10.1007/978-3-031-58740-5_3. URL: https://doi.org/10.1007/978-3-031-58740-5_3.

[21] D. J. Bernstein. "Multi-user Schnorr security, revisited". In: *IACR Cryptol. ePrint Arch.* (2015), p. 996. URL: http://eprint.iacr.org/2015/996.

[22] A. Boldyreva. "Threshold Signatures, Multisignatures and Blind Signatures Based on the Gap-Diffie-Hellman-Group Signature Scheme". In: *Public Key Cryptography - PKC 2003, 6th International Workshop on Theory and Practice in Public Key Cryptography, Miami, FL, USA, January 6-8, 2003, Proceedings*. Ed. by Y. Desmedt. Vol. 2567. Lecture Notes in Computer Science. Springer, 2003, pp. 31–46. DOI: 10.1007/3-540-36288-6_3. URL: https://doi.org/10.1007/3-540-36288-6_3.

[23] D. Boneh and X. Boyen. "Efficient Selective-ID Secure Identity-Based Encryption Without Random Oracles". In: *Advances in Cryptology - EUROCRYPT 2004, International Conference on the Theory and Applications of Cryptographic Techniques, Interlaken, Switzerland, May 2-6, 2004, Proceedings*. Ed. by C. Cachin and J. Camenisch. Vol. 3027. Lecture Notes in Computer Science. Springer, 2004, pp. 223–238. DOI: 10.1007/978-3-540-24676-3_14. URL: https://doi.org/10.1007/978-3-540-24676-3_14.

[24] D. Boneh and X. Boyen. "Short Signatures Without Random Oracles and the SDH Assumption in Bilinear Groups". In: *J. Cryptol.* 21.2 (2008), pp. 149–177. DOI: 10.1007/S00145-007-9005-7. URL: https://doi.org/10.1007/s00145-007-9005-7.

[25] D. Boneh, X. Boyen, and H. Shacham. "Short Group Signatures". In: *CRYPTO 2004, 24th Annual International CryptologyConference, Santa Barbara, California, USA, August 15-19, 2004*. Ed. by M. K. Franklin. Vol. 3152. LNCS. Springer, 2004, pp. 41–55. DOI: 10.1007/978-3-540-28628-8_3. URL: https://doi.org/10.1007/978-3-540-28628-8_3.

[26] D. Boneh and V. Shoup. *A Graduate Course in Applied Cryptography*. 2020. URL: http://toc.cryptobook.us/book.pdf.

[27] D. Boneh and R. Venkatesan. "Breaking RSA May Not Be Equivalent to Factoring". In: *EUROCRYPT '98, International Conference on the Theory and Application of Cryptographic Techniques, Espoo, Finland, May 31 - June 4, 1998*. Ed. by K. Nyberg. Vol. 1403. LNCS. Springer, 1998, pp. 59–71. DOI: 10.1007/BFb0054117. URL: https://doi.org/10.1007/BFb0054117.

[28] Z. Brakerski and S. Medina. "Limits on Adaptive Security for Attribute-Based Encryption". In: *Theory of Cryptography - 22nd International Conference, TCC 2024, Milan, Italy, December 2-6, 2024, Proceedings, Part III*. Ed. by E. Boyle and M. Mahmoody. Vol. 15366. Lecture Notes in Computer Science. Springer, 2024, pp. 91–123. DOI: 10.1007/978-3-031-78020-2_4. URL: https://doi.org/10.1007/978-3-031-78020-2_4.

[29] L. Brandão and M. Davidson. *Notes on Threshold EdDSA/Schnorr Signatures*. https://nvlpubs.nist.gov/nistpubs/ir/2022/NIST.IR.8214B.ipd.pdf. 2022.

[30] L. Brandão and R. Peralta. *NIST First Call for Multi-Party Threshold Schemes*. https://nvlpubs.nist.gov/nistpubs/ir/2025/NIST.IR.8214C.2pd.pdf. 2025.

[31] J. Brendel, C. Cremers, D. Jackson, and M. Zhao. "The Provable Security of Ed25519: Theory and Practice". In: *42nd IEEE Symposium on Security and Privacy, SP 2021, San Francisco, CA, USA, 24-27 May 2021*. IEEE, 2021, pp. 1659–1676. DOI: 10.1109/SP40001.2021.00042. URL: https://doi.org/10.1109/SP40001.2021.00042.

[32] R. Canetti, R. Gennaro, S. Goldfeder, N. Makriyannis, and U. Peled. "UC Non-Interactive, Proactive, Threshold ECDSA with Identifiable Aborts". In: *CCS '20: 2020 ACM SIGSAC Conference on Computer and Communications Security, Virtual Event, USA, November 9-13, 2020*. Ed. by J. Ligatti, X. Ou, J. Katz, and G. Vigna. ACM, 2020, pp. 1769–1787. DOI: 10.1145/3372297.3423367. URL: https://doi.org/10.1145/3372297.3423367.

[33] R. Canetti, R. Gennaro, S. Jarecki, H. Krawczyk, and T. Rabin. "Adaptive Security for Threshold Cryptosystems". In: *Advances in Cryptology - CRYPTO '99, 19th Annual International Cryptology Conference, Santa Barbara, California, USA, August 15-19, 1999, Proceedings*. Ed. by M. J. Wiener. Vol. 1666. Lecture Notes in Computer Science. Springer, 1999, pp. 98–115. DOI: 10.1007/3-540-48405-1_7. URL: https://doi.org/10.1007/3-540-48405-1_7.

[34] D. Chaum and T. P. Pedersen. "Wallet Databases with Observers". In: *Advances in Cryptology - CRYPTO '92, 12th Annual International Cryptology Conference, Santa Barbara, California, USA, August 16-20, 1992, Proceedings*. Ed. by E. F. Brickell. Vol. 740. Lecture Notes in Computer Science. Springer, 1992, pp. 89–105.

DOI: 10.1007/3-540-48071-4_7. URL: https://doi.org/10.1007/3-540-48071-4_7.

[35] Y. Chen. "Dazzle: Improved Adaptive Threshold Signatures from DDH". In: *Public-Key Cryptography - PKC 2025 - 28th IACR International Conference on Practice and Theory of Public-Key Cryptography, Røros, Norway, May 12-15, 2025, Proceedings, Part III*. Ed. by T. Jager and J. Pan. Vol. 15676. Lecture Notes in Computer Science. Springer, 2025, pp. 233–261. DOI: 10.1007/978-3-031-91826-1_8. URL: https://doi.org/10.1007/978-3-031-91826-1_8.

[36] Y. Chen. "Round-Efficient Adaptively Secure Threshold Signatures with Rewinding". In: *IACR Commun. Cryptol.* 2.2 (2025), p. 16. DOI: 10.62056/AH893Z10K. URL: https://doi.org/10.62056/ah893z10k.

[37] G. Cho, G. Fuchsbauer, A. O'Neill, and M. Sefranek. "Schnorr Signatures are Tightly Secure in the ROM Under a Non-interactive Assumption". In: *Advances in Cryptology - CRYPTO 2025 - 45th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 17-21, 2025, Proceedings, Part VI*. Ed. by Y. T. Kalai and S. F. Kamara. Vol. 16005. Lecture Notes in Computer Science. Springer, 2025, pp. 223–255. DOI: 10.1007/978-3-032-01887-8_8. URL: https://doi.org/10.1007/978-3-032-01887-8_8.

[38] H. Chu, P. Gerhart, T. Ruffing, and D. Schröder. "Practical Schnorr Threshold Signatures Without the Algebraic Group Model". In: *CRYPTO 2023 - 43rd Annual International Cryptology Conference, CRYPTO 2023, Santa Barbara, CA, USA, August 20-24, 2023*. Ed. by H. Handschuh and A. Lysyanskaya. Vol. 14081. LNCS. Springer, 2023, pp. 743–773. DOI: 10.1007/978-3-031-38557-5_24. URL: https://doi.org/10.1007/978-3-031-38557-5_24.

[39] D. Connolly, C. Komlo, I. Goldberg, and C. Wood. *Two-Round Threshold Schnorr Signatures with FROST*. 2024. URL: https://datatracker.ietf.org/doc/draft-irtf-cfrg-frost/.

[40] J. Coron. "Optimal Security Proofs for PSS and Other Signature Schemes". In: *Advances in Cryptology - EUROCRYPT 2002, International Conference on the Theory and Applications of Cryptographic Techniques, Amsterdam, The Netherlands, April 28 - May 2, 2002, Proceedings*. Ed. by L. R. Knudsen. Vol. 2332. Lecture Notes in Computer Science. Springer, 2002, pp. 272–287. DOI: 10.1007/3-540-46035-7_18. URL: https://doi.org/10.1007/3-540-46035-7_18.

[41] E. C. Crites, J. Katz, C. Komlo, S. Tessaro, and C. Zhu. "On the Adaptive Security of FROST". In: *Advances in Cryptology - CRYPTO 2025 - 45th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 17-21, 2025, Proceedings, Part VI*. Ed. by Y. T. Kalai and S. F. Kamara. Vol. 16005. Lecture Notes in Computer Science. Springer, 2025, pp. 480–511. DOI: 10.1007/978-3-032-01887-8_16. URL: https://doi.org/10.1007/978-3-032-01887-8_16.

[42] E. C. Crites, M. Kohlweiss, B. Preneel, M. Sedaghat, and D. Slamanig. "Threshold Structure-Preserving Signatures". In: *Advances in Cryptology - ASIACRYPT 2023 - 29th International Conference on the Theory and Application of Cryptology and Information Security, Guangzhou, China, December 4-8, 2023, Proceedings, Part II*. Ed. by J. Guo and R. Steinfeld. Vol. 14439. Lecture Notes in Computer Science. Springer, 2023, pp. 348–382. DOI: 10.1007/978-981-99-8724-5_11. URL: https://doi.org/10.1007/978-981-99-8724-5_11.

[43] E. C. Crites, C. Komlo, and M. Maller. "Fully Adaptive Schnorr Threshold Signatures". In: *CRYPTO 2023 - 43rd Annual International Cryptology Conference, CRYPTO 2023, Santa Barbara, CA, USA, August 20-24, 2023*. Ed. by H.

Handschuh and A. Lysyanskaya. Vol. 14081. LNCS. Springer, 2023, pp. 678–709. DOI: 10.1007/978-3-031-38557-5_22. URL: https://doi.org/10.1007/978-3-031-38557-5_22.

[44] E. C. Crites, C. Komlo, and M. Maller. "How to Prove Schnorr Assuming Schnorr: Security of Multi- and Threshold Signatures". In: *IACR Cryptol. ePrint Arch.* (2021), p. 1375. URL: https://eprint.iacr.org/2021/1375.

[45] E. C. Crites and A. Stewart. "A Plausible Attack on the Adaptive Security of Threshold Schnorr Signatures". In: *Advances in Cryptology - CRYPTO 2025 - 45th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 17-21, 2025, Proceedings, Part VI*. Ed. by Y. T. Kalai and S. F. Kamara. Vol. 16005. Lecture Notes in Computer Science. Springer, 2025, pp. 457–479. DOI: 10.1007/978-3-032-01887-8_15. URL: https://doi.org/10.1007/978-3-032-01887-8_15.

[46] S. Das and L. Ren. "Adaptively Secure BLS Threshold Signatures from DDH and co-CDH". In: *Advances in Cryptology - CRYPTO 2024 - 44th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 18-22, 2024, Proceedings, Part VII*. Ed. by L. Reyzin and D. Stebila. Vol. 14926. Lecture Notes in Computer Science. Springer, 2024, pp. 251–284. DOI: 10.1007/978-3-031-68394-7_9. URL: https://doi.org/10.1007/978-3-031-68394-7_9.

[47] Y. Dodis and A. Yampolskiy. "A Verifiable Random Function with Short Proofs and Keys". In: *Public Key Cryptography - PKC 2005, 8th International Workshop on Theory and Practice in Public Key Cryptography, Les Diablerets, Switzerland, January 23-26, 2005, Proceedings*. Ed. by S. Vaudenay. Vol. 3386. Lecture Notes in Computer Science. Springer, 2005, pp. 416–431. DOI: 10.1007/978-3-540-30580-4_28. URL: https://doi.org/10.1007/978-3-540-30580-4_28.

[48] J. Doerner, Y. Kondi, E. Lee, A. Shelat, and L. Tyner. "Threshold BBS+ Signatures for Distributed Anonymous Credential Issuance". In: *44th IEEE Symposium on Security and Privacy, SP 2023, San Francisco, CA, USA, May 21-25, 2023*. IEEE, 2023, pp. 773–789. DOI: 10.1109/SP46215.2023.10179470. URL: https://doi.org/10.1109/SP46215.2023.10179470.

[49] A. Fiat and A. Shamir. "How to Prove Yourself: Practical Solutions to Identification and Signature Problems". In: *Advances in Cryptology - CRYPTO '86, Santa Barbara, California, USA, 1986, Proceedings*. Ed. by A. M. Odlyzko. Vol. 263. Lecture Notes in Computer Science. Springer, 1986, pp. 186–194. DOI: 10.1007/3-540-47721-7_12. URL: https://doi.org/10.1007/3-540-47721-7_12.

[50] M. Fischlin. "Black-Box Reductions and Separations in Cryptography". In: *AFRICACRYPT 2012 - 5th International Conference on Cryptology in Africa, Ifrance, Morocco, July 10-12*. Ed. by A. Mitrokotsa and S. Vaudenay. Vol. 7374. LNCS. Springer, 2012, pp. 413–422. DOI: 10.1007/978-3-642-31410-0_26. URL: https://doi.org/10.1007/978-3-642-31410-0_26.

[51] M. Fischlin. "Communication-Efficient Non-interactive Proofs of Knowledge with Online Extractors". In: *Advances in Cryptology - CRYPTO 2005: 25th Annual International Cryptology Conference, Santa Barbara, California, USA, August 14-18, 2005, Proceedings*. Ed. by V. Shoup. Vol. 3621. Lecture Notes in Computer Science. Springer, 2005, pp. 152–168. DOI: 10.1007/11535218_10. URL: https://doi.org/10.1007/11535218_10.

[52] M. Fischlin and N. Fleischhacker. "Limitations of the Meta-reduction Technique: The Case of Schnorr Signatures". In: *EUROCRYPT 2013, 32nd Annual International Conference on the Theory and Applications of Cryptographic Techniques, Athens, Greece, May 26-30, 2013*. Ed. by T. Johansson and P. Q. Nguyen. Vol. 7881.

LNCS. Springer, 2013, pp. 444–460. DOI: 10.1007/978-3-642-38348-9_27. URL: https://doi.org/10.1007/978-3-642-38348-9_27.

[53] Y. Frankel, P. D. MacKenzie, and M. Yung. "Adaptively secure distributed public-key systems". In: *Theor. Comput. Sci.* 287.2 (2002), pp. 535–561. DOI: 10.1016/S0304-3975(01)00260-2. URL: https://doi.org/10.1016/S0304-3975(01)00260-2.

[54] Y. Frankel, P. D. MacKenzie, and M. Yung. "Adaptively-Secure Optimal-Resilience Proactive RSA". In: *ASIACRYPT '99, International Conference on the Theory and Applications of Cryptology and Information Security, Singapore, November 14-18, 1999*. Ed. by K. Lam, E. Okamoto, and C. Xing. Vol. 1716. LNCS. Springer, 1999, pp. 180–194. DOI: 10.1007/978-3-540-48000-6_15. URL: https://doi.org/10.1007/978-3-540-48000-6_15.

[55] G. Fuchsbauer, E. Kiltz, and J. Loss. "The Algebraic Group Model and its Applications". In: *Advances in Cryptology - CRYPTO 2018 - 38th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 19-23, 2018, Proceedings, Part II*. Ed. by H. Shacham and A. Boldyreva. Vol. 10992. Lecture Notes in Computer Science. Springer, 2018, pp. 33–62. DOI: 10.1007/978-3-319-96881-0_2. URL: https://doi.org/10.1007/978-3-319-96881-0_2.

[56] G. Fuchsbauer, M. Konstantinov, K. Pietrzak, and V. Rao. "Adaptive Security of Constrained PRFs". In: *Advances in Cryptology - ASIACRYPT 2014 - 20th International Conference on the Theory and Application of Cryptology and Information Security, Kaoshiung, Taiwan, R.O.C., December 7-11, 2014, Proceedings, Part II*. Ed. by P. Sarkar and T. Iwata. Vol. 8874. Lecture Notes in Computer Science. Springer, 2014, pp. 82–101. DOI: 10.1007/978-3-662-45608-8_5. URL: https://doi.org/10.1007/978-3-662-45608-8_5.

[57] F. Garillot, Y. Kondi, P. Mohassel, and V. Nikolaenko. "Threshold Schnorr with Stateless Deterministic Signing from Standard Assumptions". In: *CRYPTO 2021 - 41st Annual International Cryptology Conference, CRYPTO 2021, Virtual Event, August 16-20, 2021, Proceedings, Part I*. Ed. by T. Malkin and C. Peikert. Vol. 12825. LNCS. Springer, 2021, pp. 127–156. DOI: 10.1007/978-3-030-84242-0_6. URL: https://doi.org/10.1007/978-3-030-84242-0_6.

[58] R. Gennaro and S. Goldfeder. "Fast Multiparty Threshold ECDSA with Fast Trustless Setup". In: *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security, CCS 2018, Toronto, ON, Canada, October 15-19, 2018*. Ed. by D. Lie, M. Mannan, M. Backes, and X. Wang. ACM, 2018, pp. 1179–1194. DOI: 10.1145/3243734.3243859. URL: https://doi.org/10.1145/3243734.3243859.

[59] C. Gentry and S. Halevi. "Hierarchical Identity Based Encryption with Polynomially Many Levels". In: *Theory of Cryptography, 6th Theory of Cryptography Conference, TCC 2009, San Francisco, CA, USA, March 15-17, 2009. Proceedings*. Ed. by O. Reingold. Vol. 5444. Lecture Notes in Computer Science. Springer, 2009, pp. 437–456. DOI: 10.1007/978-3-642-00457-5_26. URL: https://doi.org/10.1007/978-3-642-00457-5_26.

[60] P. Gerhart, D. L. Calsi, L. Russo, and D. Schröder. "Fully-Adaptive Two-Round Threshold Schnorr Signatures from DDH". In: *To appear in EUROCRYPT 2026.* (2025), p. 1478. URL: https://eprint.iacr.org/2025/1478.

[61] S. Goldberg, L. Reyzin, D. Papadopoulos, and J. Vcelak. *Verifiable Random Functions (VRFs)*. 2023. URL: https://datatracker.ietf.org/doc/rfc9381/.

[62] S. Goldwasser, S. Micali, and R. L. Rivest. "A Digital Signature Scheme Secure Against Adaptive Chosen-Message Attacks". In: *SIAM J. Comput.* 17.2 (1988), pp. 281–308. DOI: 10.1137/0217017. URL: https://doi.org/10.1137/0217017.

[63] D. Hofheinz, T. Jager, and E. Knapp. "Waters Signatures with Optimal Security Reduction". In: *Public Key Cryptography - PKC 2012 - 15th International Conference on Practice and Theory in Public Key Cryptography, Darmstadt, Germany, May 21-23, 2012. Proceedings*. Ed. by M. Fischlin, J. Buchmann, and M. Manulis. Vol. 7293. Lecture Notes in Computer Science. Springer, 2012, pp. 66–83. DOI: 10.1007/978-3-642-30057-8_5. URL: https://doi.org/10.1007/978-3-642-30057-8_5.

[64] S. Jarecki and A. Lysyanskaya. "Adaptively Secure Threshold Cryptography: Introducing Concurrency, Removing Erasures". In: *Advances in Cryptology - EUROCRYPT 2000, International Conference on the Theory and Application of Cryptographic Techniques, Bruges, Belgium, May 14-18, 2000, Proceeding*. Ed. by B. Preneel. Vol. 1807. Lecture Notes in Computer Science. Springer, 2000, pp. 221–242. DOI: 10.1007/3-540-45539-6_16. URL: https://doi.org/10.1007/3-540-45539-6_16.

[65] S. A. Kakvi and E. Kiltz. "Optimal Security Proofs for Full Domain Hash, Revisited". In: *Advances in Cryptology - EUROCRYPT 2012 - 31st Annual International Conference on the Theory and Applications of Cryptographic Techniques, Cambridge, UK, April 15-19, 2012. Proceedings*. Ed. by D. Pointcheval and T. Johansson. Vol. 7237. Lecture Notes in Computer Science. Springer, 2012, pp. 537–553. DOI: 10.1007/978-3-642-29011-4_32. URL: https://doi.org/10.1007/978-3-642-29011-4_32.

[66] S. Katsumata, M. Reichle, and K. Takemure. "Adaptively Secure 5 Round Threshold Signatures from MLWE/MSIS and DL with Rewinding". In: *Advances in Cryptology - CRYPTO 2024 - 44th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 18-22, 2024, Proceedings, Part VII*. Ed. by L. Reyzin and D. Stebila. Vol. 14926. Lecture Notes in Computer Science. Springer, 2024, pp. 459–491. DOI: 10.1007/978-3-031-68394-7_15. URL: https://doi.org/10.1007/978-3-031-68394-7_15.

[67] J. Katz. "Round-Optimal, Fully Secure Distributed Key Generation". In: *Advances in Cryptology - CRYPTO 2024 - 44th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 18-22, 2024, Proceedings, Part VII*. Ed. by L. Reyzin and D. Stebila. Vol. 14926. Lecture Notes in Computer Science. Springer, 2024, pp. 285–316. DOI: 10.1007/978-3-031-68394-7_10. URL: https://doi.org/10.1007/978-3-031-68394-7_10.

[68] C. Komlo and I. Goldberg. "Arctic: Lightweight and Stateless Threshold Schnorr Signatures". In: *Public-Key Cryptography - PKC 2025 - 28th IACR International Conference on Practice and Theory of Public-Key Cryptography, Røros, Norway, May 12-15, 2025, Proceedings, Part V*. Ed. by T. Jager and J. Pan. Vol. 15678. Lecture Notes in Computer Science. Springer, 2025, pp. 234–267. DOI: 10.1007/978-3-031-91832-2_8. URL: https://doi.org/10.1007/978-3-031-91832-2_8.

[69] C. Komlo and I. Goldberg. "FROST: Flexible Round-Optimized Schnorr Threshold Signatures". In: *SAC 2020, Halifax, NS, Canada (Virtual Event), October 21-23, 2020*. Ed. by O. Dunkelman, M. J. J. Jr., and C. O'Flynn. Vol. 12804. LNCS. Springer, 2020, pp. 34–65. DOI: 10.1007/978-3-030-81652-0_2.

[70] A. B. Lewko and B. Waters. "Why Proving HIBE Systems Secure Is Difficult". In: *Advances in Cryptology - EUROCRYPT 2014 - 33rd Annual International Con-*

ference on the Theory and Applications of Cryptographic Techniques, Copenhagen, Denmark, May 11-15, 2014. Proceedings. Ed. by P. Q. Nguyen and E. Oswald. Vol. 8441. Lecture Notes in Computer Science. Springer, 2014, pp. 58–76. DOI: 10.1007/978-3-642-55220-5_4. URL: https://doi.org/10.1007/978-3-642-55220-5_4.

[71] B. Libert, M. Joye, and M. Yung. "Born and raised distributively: Fully distributed non-interactive adaptively-secure threshold signatures with short shares". In: *Theor. Comput. Sci.* 645 (2016), pp. 1–24. DOI: 10.1016/J.TCS.2016.02.031. URL: https://doi.org/10.1016/j.tcs.2016.02.031.

[72] Y. Lindell. "Simple Three-Round Multiparty Schnorr Signing with Full Simulatability". In: *IACR Commun. Cryptol.* 1.1 (2024), p. 25. DOI: 10.62056/A36C0L5VT. URL: https://doi.org/10.62056/a36c0l5vt.

[73] T. Looker, V. Kalos, A. Whitehead, and M. Lodder. *The BBS Signature Scheme*. 2026. URL: https://datatracker.ietf.org/doc/draft-irtf-cfrg-bbs-signatures/.

[74] A. Lysyanskaya and C. Peikert. "Adaptive Security in the Threshold Setting: From Cryptosystems to Signature Schemes". In: *Advances in Cryptology - ASIACRYPT 2001, 7th International Conference on the Theory and Application of Cryptology and Information Security, Gold Coast, Australia, December 9-13, 2001, Proceedings*. Ed. by C. Boyd. Vol. 2248. Lecture Notes in Computer Science. Springer, 2001, pp. 331–350. DOI: 10.1007/3-540-45682-1_20. URL: https://doi.org/10.1007/3-540-45682-1_20.

[75] N. Makriyannis. "On the Classic Protocol for MPC Schnorr Signatures". In: *IACR Cryptol. ePrint Arch.* (2022), p. 1332. URL: https://eprint.iacr.org/2022/1332.

[76] J. Nick, T. Ruffing, and Y. Seurin. "MuSig2: Simple Two-Round Schnorr Multisignatures". In: *Advances in Cryptology - CRYPTO 2021 - 41st Annual International Cryptology Conference, CRYPTO 2021, Virtual Event, August 16-20, 2021, Proceedings, Part I*. Ed. by T. Malkin and C. Peikert. Vol. 12825. Lecture Notes in Computer Science. Springer, 2021, pp. 189–221. DOI: 10.1007/978-3-030-84242-0_8. URL: https://doi.org/10.1007/978-3-030-84242-0_8.

[77] P. Paillier and D. Vergnaud. "Discrete-Log-Based Signatures May Not Be Equivalent to Discrete Log". In: *ASIACRYPT 2005, 11th International Conference on the Theory and Application of Cryptology and Information Security, Chennai, India, December 4-8, 2005*. Ed. by B. K. Roy. Vol. 3788. LNCS. Springer, 2005, pp. 1–20. DOI: 10.1007/11593447_1. URL: https://doi.org/10.1007/11593447_1.

[78] D. Pointcheval and J. Stern. "Security Arguments for Digital Signatures and Blind Signatures". In: *J. Cryptol.* 13.3 (2000), pp. 361–396. DOI: 10.1007/S001450010003. URL: https://doi.org/10.1007/s001450010003.

[79] O. Reingold, L. Trevisan, and S. P. Vadhan. "Notions of Reducibility between Cryptographic Primitives". In: *Theory of Cryptography, First Theory of Cryptography Conference, TCC 2004, Cambridge, MA, USA, February 19-21, 2004, Proceedings*. Ed. by M. Naor. Vol. 2951. Lecture Notes in Computer Science. Springer, 2004, pp. 1–20. DOI: 10.1007/978-3-540-24638-1_1. URL: https://doi.org/10.1007/978-3-540-24638-1_1.

[80] T. Ruffing, V. Ronge, E. Jin, J. Schneider-Bensch, and D. Schröder. "ROAST: Robust Asynchronous Schnorr Threshold Signatures". In: *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security, CCS 2022, Los Angeles, CA, USA, November 7-11, 2022*. Ed. by H. Yin, A. Stavrou, C.

Cremers, and E. Shi. ACM, 2022, pp. 2551–2564. DOI: 10.1145/3548606.3560583. URL: https://doi.org/10.1145/3548606.3560583.

[81] C. Schnorr. "Efficient Signature Generation by Smart Cards". In: *J. Cryptol.* 4.3 (1991), pp. 161–174. DOI: 10.1007/BF00196725. URL: https://doi.org/10.1007/BF00196725.

[82] A. Shamir. "How to Share a Secret". In: *Commun. ACM* 22.11 (1979), pp. 612–613.

[83] D. R. Stinson and R. Strobl. "Provably Secure Distributed Schnorr Signatures and a (t, n) Threshold Scheme for Implicit Certificates". In: *Information Security and Privacy, 6th Australasian Conference, ACISP 2001, Sydney, Australia, July 11-13, 2001, Proceedings*. Ed. by V. Varadharajan and Y. Mu. Vol. 2119. Lecture Notes in Computer Science. Springer, 2001, pp. 417–434. DOI: 10.1007/3-540-47719-5_33. URL: https://doi.org/10.1007/3-540-47719-5_33.

[84] S.-Y. Tan, W. Wang, and R. Chow. *From TS-SUF-2 to TS-SUF-4: Practical Security Enhancements for FROST2 Threshold Signatures*. Cryptology ePrint Archive, Paper 2026/075. 2026. URL: https://eprint.iacr.org/2026/075.

[85] S. Tessaro and C. Zhu. "Revisiting BBS Signatures". In: *Advances in Cryptology - EUROCRYPT 2023 - 42nd Annual International Conference on the Theory and Applications of Cryptographic Techniques, Lyon, France, April 23-27, 2023, Proceedings, Part V*. Ed. by C. Hazay and M. Stam. Vol. 14008. Lecture Notes in Computer Science. Springer, 2023, pp. 691–721. DOI: 10.1007/978-3-031-30589-4_24. URL: https://doi.org/10.1007/978-3-031-30589-4_24.

[86] Z. Wang, H. Qian, and Z. Li. "Adaptively Secure Threshold Signature Scheme in the Standard Model". In: *Informatica* 20.4 (2009), pp. 591–612. URL: http://content.iospress.com/articles/informatica/info20-4-09.

A Metareduction Techniques

We now survey other related impossibility results in the literature, and discuss how our techniques and results compare.

The seminal work of Coron [40] rules out tight reductions to non-interactive *computational* assumptions for *unique* signature schemes, which was later extended to (non-unique) re-randomizable signatures by Hofheinz et al. [63]. Coron's metareduction was found to contain a flaw: it is implicitly assumed that the public key output by the reduction to the adversary is correct; however, a reduction may output an incorrect key that is computationally indistinguishable from a correct one. Kakvi and Kiltz [65] rectify this by enforcing an efficient check that the reduction has computed the public key correctly. We similarly require this efficient check for our metareduction to succeed; hence the requirement that key-unique schemes have an efficient verification algorithm (see Definition 4).

In the same vein, Lewko and Waters [70] prove the impossibility of fully black-box reductions to non-interactive *decisional* assumptions for hierarchical identity-based encryption (HIBE) and attribute-based encryption (ABE) schemes that satisfy a certain "checkability" property for ciphertexts and keys. Our key-uniqueness property similarly requires an efficient checkability property for keys. However, there are (HIBE) constructions (e.g., Gentry and Halevi [59]) that are not checkable under their definition and *can* be proven secure under

non-interactive computational assumptions. Our checkability property rules out constructions under non-interactive *computational* and *decisional* assumptions (as well as some interactive assumptions in our second main result). They provide a "simple" metareduction, which forbids the reduction from rewinding the adversary or supplying its randomness. Our metareduction allows the reduction to rewind the adversary *and* supply its randomness, as long as on each rewind the reduction provides the same public parameters and public keys. Therefore, [70] does not imply our first result. Whereas Coron [40] and Hofheinz et al. [63] show the necessity of polynomial loss of security (in the number of signature queries), [70] shows the necessity of a drastic exponential loss of security (as a function of the hierarchy depth) for HIBE and ABE schemes. Very recently, Brakerski and Medina [28] resurrected the work of [70], extending its ABE results to rewinding reductions.

Bellare et al. [18] give impossibility results for semantically secure public key encryption and commitment schemes under selective opening attacks. They show that non-interactive or perfectly binding commitment schemes cannot be proven semantically secure with respect to selective opening attacks under standard (non-interactive) computational assumptions. Our work only requires that public keys are generated via an injective map, as opposed to a function that preserves semantic security.

Fuchsbauer et al. [56] investigate the adaptive security of constrained pseudorandom functions (PRFs), constructing a metareduction that shows that any "simple" reduction of adaptive security from a non-interactive decisional assumption must incur exponential security loss, following [70]. Since PRFs are not a public-key primitive, a new checkability property is required.

While our property of key-uniqueness follows in spirit from the variants of the checkability property defined above, all of them (with the exception of [18]) result in impossibility results for decisional assumptions only, and place other restrictions (like unique [40] or re-randomizable [63] signatures only). Therefore, our metareduction approach does not follow from [70, 28], and requires different techniques. Moreover, one of the main goals of this work is to define a "recipe" for how to construct adaptively secure threshold signature schemes, which [40, 18, 56, 70, 28] do not provide.

Bacho and Loss [8] extend the results of Coron [40] to a unique *threshold* signature (BLS) under the *interactive* computational assumption OMDL, ruling out a natural class of tight black-box reductions. We similarly demonstrate the necessity of interactive assumptions (OMDL) and a straight-line reduction for our second main result. However, our metareduction is for threshold Schnorr signatures and our OMDL result does not assume an algebraic reduction. Thus, we require different techniques than those employed in [8]. Moreover, our results extend beyond ruling out full adaptive security (t out of $t+1$) to any $t/\ell < t_c \leq t$ adaptive corruptions.

In all of the above [70, 56, 8, 28], the metareduction works by rewinding the reduction, as in the original metareduction of Coron [40]. For example, in the case of signature schemes, rewinding the reduction allows the adversary (simulated by

the metareduction) to obtain an additional signature via a signing query in one execution of the reduction and subsequently use it as a forgery in the other. Such arguments require care for handling how the extra signature affects the reduction's state. This difficulty is compounded if the reduction is allowed to rewind the adversary [28]. An observation, first made by Baldimtsi and Lysyanskaya [13] and followed by Fischlin and Fleischhacker [52], is to, instead, run two *independent* copies of the reduction (on independent randomness). This cleanly separates the additional signature from the reduction's state and removes the aforementioned barrier. This technique is applied to single-party Schnorr signatures in [52], and to blind Schnorr signatures in [13]. Similar to [63], the results of [52] only apply to the version of Schnorr signatures where the challenge is computed as $\mathsf{H}(R, m)$, where m is the message, R is the nonce, and H is modeled as a random oracle, and not to the version of Schnorr signatures where H additionally takes as input the public key (so-called "key-prefixed" Schnorr). [13] similarly relies on the non-key-prefixed of the Schnorr signature scheme. There are known vulnerabilities in the multi-party setting for Schnorr signatures if the public key is not included in the hash [21, 31, 29], and our results capture key-prefixed Schnorr schemes. Our metareduction similarly executes two instances of the reduction, introducing randomness across each instance and using responses from one instance to aid its simulation in the other. However, the strategy in [52] requires the inputs to the two instances to be *different*, and that at least one signing query be made. In fact, perhaps the most interesting deviation from this technique is the fact that, unlike [13] and [52], *we do not require even one signature to be issued.* Our results hold based on the reduction's issuance of the public key shares alone. As a result, we require no restrictions on the reduction's ability to program any random oracles for the adversary. In contrast, [52] only applies only to the non-programmable ROM, and [13] applies only to the programmable ROM.

In summary, our metareduction techniques depart substantially from the approach used for HIBE/ABE in [70, 28]. Our metareduction allows the reduction to rewind the adversary *and* supply its randomness, as long as it provides consistent public parameters and public keys in each instance. Uniquely, our metareductions do not require a signature in order to work, and instead show a strong result ruling out adversaries that receive public key shares only. Taken together, our metareductions eliminate a broad class of security reductions for unique and non-unique threshold signatures, both straight-line and rewinding, in various idealized models, under both computational and decisional assumptions, and even allow the reduction to freely program any random oracles for the adversary. Moreover, all of this holds for the range of $\lfloor t/\ell \rfloor < t_c \leq t$ adaptive corruptions.

In subsequent work, Bacho et al. [4] use a similar proof to ours to rule out a black-box, fully algebraic reduction to full adaptive security of a threshold Schnorr scheme ms-FROST under AOMDL in the ROM. It eliminates straight-line and rewinding (*fully algebraic*) reductions for this construction – which is not key unique. We also note that our strategy is exploited in a recent, follow-up work

giving a plausible attack based on public key shares alone [45], which we discuss in more detail in the introduction.

B Application to Existing Schemes

B.1 Key-Unique Schemes

Key-unique schemes have a shared secret key that is enough for issuing a signature and can be obtained from any $t+1$ shares (Section 2.2). Note that this does not just include schemes with a single field-element secret, but also where secrets consist of multiple field elements as well as packed secret-sharing schemes. As discussed in the introduction, key-uniqueness is often leveraged in practice to achieve non-interactive identifiable abort, without the need for additional zero-knowledge proofs. There has also been recent motivation to prove the (static, $t_c = t$) security of threshold signatures with respect to non-interactive assumptions, such as DL [43, 72, 75, 20, 68], in light of the NIST call [30].

We describe a range of key-unique threshold signature schemes in the literature, to which our first impossibility result applies. We also highlight a few approaches for achieving adaptive security.

Bacho and Loss [8] show the adaptive security of (Type I) threshold BLS for up to $t < n/2$ corruptions in the algebraic group model (AGM) and ROM under the OMDL assumption. The static security of threshold BLS [22] holds under the gap Diffie-Hellman assumption (GDH) in the ROM.

The threshold ECDSA scheme of Gennaro and Goldfeder [58] is statically secure assuming the security of single-party ECDSA signatures, the Strong RSA and DDH assumptions, and the security of a non-malleable equivocable commitment scheme. The security of the threshold ECDSA scheme by Canetti et al. [32] relies on the security of ECDSA, the Strong RSA and DDH assumptions, the semantic security of Paillier encryption, and the global random oracle model. Both schemes have been proven adaptively secure with $\binom{n}{t_c}$ tightness loss using the standard guessing strategy [33] discussed in the introduction.

Doerner et al. [48] construct a threshold scheme for BBS+ signatures [3, 25]. They prove that their scheme UC-realizes a single-party BBS+ functionality in the $(\mathcal{F}_{\mathsf{com}}, \mathcal{F}_{\mathsf{zk}}^{R_{\mathrm{DL}\wedge\phi}}, \mathcal{F}_{\mathsf{DLKeyGen}}, \mathcal{F}_{\mathsf{zero}}, \mathcal{F}_{\mathsf{mul2P}})$-hybrid model in the presence of a static adversary, where $\mathcal{F}_{\mathsf{com}}$ is a commitment functionality, $\mathcal{F}_{\mathsf{ZK}}^{R_{\mathrm{DL}\wedge\phi}}$ is a zero-knowledge discrete logarithm functionality, $\mathcal{F}_{\mathsf{DLKeyGen}}$ is a discrete logarithm key generation functionality, $\mathcal{F}_{\mathsf{Zero}}$ is a functionality that produces secret sharings of zero in a particular group, and $\mathcal{F}_{\mathsf{Mul2P}}$ is a two-party multiplication functionality. The q-SDH assumption underpins BBS+ signatures [3, 25, 85].

Twinkle [10] is a family of three-round threshold signature schemes. The first construction, which we call Twinkle$_1$, is a distributed Chaum-Pedersen proof of equality of discrete logarithms $\mathsf{PK} = g^{\mathsf{sk}}$ and $\mathsf{H}(m)^{\mathsf{sk}}$ [34] (vs. a proof of knowledge of the discrete logarithm of $\mathsf{PK} = g^{\mathsf{sk}}$, as in Schnorr signatures). Thus, signatures are Chaum-Pedersen outputs $(\mathsf{H}(m)^{\mathsf{sk}}, c, z)$, where (c, z) is a Schnorr signature, as in ECVRF [61]. This scheme is proven fully adaptively secure under an interactive, one-more variant of CDH (AOMCDH) in the ROM.

We now specifically discuss key-unique threshold signature schemes whose verification is compatible with single-party Schnorr signatures, to which all of our impossibility results apply.

FROST [69, 17, 41] is a two-round threshold Schnorr signature scheme that is currently undergoing standardization efforts [39]. FROST2 [44, 17, 41], FROST2+/# [84], and FROST3 [80, 38, 41] are optimized variants of FROST. All of them have been proven statically secure in the ROM under (A)OMDL. FROST/2/3 have been proven adaptively secure for up to $t/2$ corruptions in the ROM under AOMDL, and fully adaptively secure in the AGM and ROM under AOMDL and the low-dimensional vector representation (LDVR) assumption, introduced in [41].

Sparkle+ [43] is a three-round threshold Schnorr signature scheme that is claimed to achieve full t adaptive corruptions under AOMDL in the AGM and ROM.[21] When the adversary is limited to making fewer than $t/2$ corruptions, the adaptive security of Sparkle+ holds under AOMDL in the ROM only, by a reduction that rewinds.

Lindell [72] defines a three-round threshold Schnorr signature scheme and proves that it UC-realizes a Schnorr signing functionality in the $(\mathcal{F}_{\mathsf{mcom}}, \mathcal{F}_{\mathsf{zk}})$-hybrid model in the presence of a static adversary, where $\mathcal{F}_{\mathsf{mcom}}$ is a multiparty broadcast commitment functionality, and $\mathcal{F}_{\mathsf{zk}}$ is a zero-knowledge functionality.

Classic S. [75] is a three-round threshold Schnorr signature scheme, which is proven to UC-realize the threshold signature functionality in [32] in the global random oracle model, assuming the hardness of DL. Classic S. is proven adaptively secure with $\binom{n}{t_c}$ tightness loss using the standard guessing strategy.

Stinson and Strobl [83] introduce a robust, seven-round threshold Schnorr scheme and prove static security assuming the security of single-party Schnorr signatures.

There has been a recent line of work on robust threshold Schnorr schemes. ROAST [80] is an $\mathcal{O}(n)$-round scheme proven statically secure under OMDL in the ROM. SPRINT [20] is a 3-broadcast-round scheme shown to be statically secure under DL in the ROM. HARTS [9] is an $\mathcal{O}(1)$-round scheme that is claimed to achieve full t adaptive corruptions under OMDL in the AGM and ROM, assuming secure erasure of secret state for its zero-knowledge proofs.[22]

The above discussion relates primarily to randomized schemes, with the exception of BLS. However, our results likewise apply to deterministic threshold Schnorr schemes [57, 68]. The three-round protocol by Garillot et al. [57] is proven to UC-realize an n-party Schnorr signing functionality in the $(\mathcal{F}_{\mathsf{F}\cdot\mathbb{G}}, \mathcal{F}_{\mathsf{com-zk}}^{R_{\mathrm{DL}}})$-hybrid model in the presence of an adversary statically corrupting up to $n-1$ parties, where $\mathcal{F}_{\mathsf{F}\cdot\mathbb{G}}$ is a deterministic nonce derivation functionality, and $\mathcal{F}_{\mathsf{com-zk}}^{R_{\mathrm{DL}}}$ is a committed ZKPoK for discrete logarithm functionality. Arctic [68], a two-round scheme, is proven statically secure under the DL assumption in the ROM.

[21] Later work [45] shows that the full adaptive security of Sparkle+ cannot hold without assuming the hardness of a search problem P defined in [45].

[22] Later work [45] shows that the full adaptive security of HARTS cannot hold without assuming the hardness of a search problem P defined in [45].

Thus, we see that, for key-unique schemes, adaptive security may be achieved by reducing to stronger assumptions, such as interactive assumptions.

B.2 Non-Key-Unique Schemes

We now highlight a variety of non-key-unique, adaptively secure threshold signature schemes, categorized by the techniques used to prove their security.

No Secret Reconstruction. As discussed in the introduction, secret reconstruction is integral to the definition of key-uniqueness. While the trivial threshold Schnorr signature scheme has public keys that are perfectly binding commitments to secret keys, the lack of secret reconstruction means the trivial scheme bypasses our impossibility results.

Non-Perfectly Binding Commitments. Libert et al. [71] provide the first fully adaptively secure, non-interactive, and unique threshold signature scheme in the ROM. The construction is based on the threshold BLS scheme of [22] and proven adaptively secure under the symmetric external Diffie-Hellman assumption (SXDH). The proof of adaptive security relies on a specific distributed key generation (DKG) protocol that results in public keys of the form $\mathsf{PK}_i = (g_1^{f(i)} g_2^{g(i)}, g_1^{h(i)} g_2^{k(i)})$, where f, g, h, k are independent polynomials of degree t. Public keys of this form do not commit the signer to a secret key, which enables the reduction to effectively simulate signatures with adaptive corruption. Indeed, public keys of this form are not key unique. Public keys consist of two group elements, and signatures are twice the size of BLS signatures and not compatible standard BLS signature verification.

Das and Ren [46] prove the full adaptive security of (Type III) threshold BLS using the Single Inconsistent Player (SIP) technique (discussed below). Public keys are of the form $\mathsf{PK}_i = g^{s(i)} h^{r(i)} v^{u(i)}$, where s, r, u are independent polynomials of degree t and the constant term in both r and u is zero. This allows the reduction to equivocate on the secret keys. The partial signatures are not compatible with single-party BLS verification and are augmented by appending a non-interactive zero-knowledge (NIZK) proof of well-formedness in order to identify misbehaving parties. Glacius [6] is a five-round threshold Schnorr signature scheme that is proven fully adaptively secure under DDH in the ROM using similar techniques to [46]. Gargos [5] reduces the rounds to three. Gerhart et al. [60] further reduce the rounds to two, by deriving the nonce in a deterministic fashion. However, the security of [60] relies on participants refreshing their secret key shares after a fixed number of signing rounds; failure to do so will result in a key-exposure attack.

Zero S. [75] is a three-round threshold Schnorr signature scheme, which is proven to UC-realize the threshold signature functionality in [32] in the global random oracle model, assuming the hardness of the DL problem. Zero S. relies on Pedersen commitments, Fischlin proofs of knowledge [51], and an interactive protocol to identify misbehaving parties. Since public key shares for each party are Pedersen commitments, Zero S. is not key unique. Moreover, adaptive corruption

of honest signers' key shares can be simulated, but not the signers' internal signing states, and therefore secure erasures are required.

Another threshold signature scheme in the Twinkle [10] family is a three-round, matrix construction, which we call Twinkle_2, that is proven fully adaptively secure under the DDH assumption in the ROM. Public keys are formed as $(g^{x_1}\hat{g}^{x_2}, h^{x_1}\hat{h}^{x_2}) \in \mathbb{G}^2$, for generators $g, \hat{g}, h, \hat{h}$ of $\mathbb{G}$ and secret key $(x_1, x_2) \in \mathbb{Z}_p^2$. While this mapping is injective with high probability if the generators are sampled uniformly at random, $g, \hat{g}, h, \hat{h}$ are swapped out with a DDH tuple in the proof, breaking the injectivity. Dazzle [35] is a similar scheme but reduces the number of group elements in the signature by one.

Requiring Secrecy of Public Key Shares. Katsumata et al. [66] construct a five-round threshold Schnorr signature scheme Crackle. They then reduce Crackle by one round by assuming that a (non-repeating) unique session identifier is broadcast to all signers. They call their four-round scheme Snap. Both schemes are proven fully adaptively secure under the DL assumption in the ROM, using rewinding. An important caveat is that if the partial public keys are revealed, the adaptive simulation fails. (Static security still holds.) Thus, Crackle and Snap sidestep our impossibility results by requiring secrecy of the public key shares. However, neither scheme allows for identifying misbehaving parties using public key shares. In the same work, the authors construct two fully adaptive lattice-based threshold signature schemes.

The same technique is employed by Chen [36] to construct a threshold Schnorr scheme based on FROST, called FROST-Mask, which is three rounds or two rounds with the requirement of consistent session identifiers, as well as a two-round protocol, HBTS-Mask, which does not require consistent session identifiers. FROST-Mask achieves full adaptive security under the AOMDL assumption in the ROM only. ms-FROST, by Bacho et al. [4], removes the need for a session identifier in two-round FROST-Mask, while (provably) requiring the AGM instead. FaFROST, by Baecker et al. [12], is the same scheme, but additionally achieves identifiable abort. These schemes all rely on the secrecy of public key shares, so our impossibility results do not apply.

Non-Key-Unique Key Resharing. A range of classical adaptively secure threshold signature schemes [33, 54, 53, 64, 74, 1, 2] leverage an approach called Single Inconsistent Player (SIP), in which the reduction can simulate the internal state of all honest players except one. If the adversary corrupts this player, the reduction rewinds the adversary and picks a different inconsistent player. The SIP strategy taken by Almansa et al. [2] for threshold RSA signatures is to linearly share the full secret and Shamir secret share the partial secret keys; consequently, the secret sharing of the partial secret keys is not key unique. The simulation can adaptively choose the secret shares of all simulated parties (except for one) at the time when the adversary makes its corruption requests. A similar strategy is used to prove the full adaptive security of threshold Waters signatures [86] for $t_c < n/2$ under CDH in the standard model. As mentioned above, it was also recently used for the threshold BLS scheme in [46].

MAIN $\mathsf{Game}^{\mathsf{NIA}}_{\mathcal{A}}(\kappa)$
$(\mathsf{NIAChal}, \mathsf{NIAPriv}) \leftarrow\!\!\$\ \mathsf{InstGen}(1^\kappa)$
$\mathsf{NIASol} \leftarrow\!\!\$\ \mathcal{A}(\mathsf{NIAChal})$
return $\mathsf{Verify}(\mathsf{NIAChal}, \mathsf{NIAPriv}, \mathsf{NIASol})$

Fig. 3: Non-Interactive Assumption NIA. The adversary is given a challenge NIAChal and returns a solution NIASol.

Thus, we see that adaptive security may be achieved using one, or a combination, of the above techniques. Indeed, our results directly explain recent fully adaptive threshold schemes [46, 10, 6, 66, 35, 5, 35, 60, 4, 12] and the assumptions under which they are proven.

C Non-Interactive Assumptions

Our first impossibility result holds under the class of cryptographic assumptions that are non-interactive and computational (search) or decisional. We provide here a definition of these assumptions and give examples, underscoring a range of assumptions under which key-unique threshold signatures cannot be proven adaptively secure (Theorems 1 to 3).

A non-interactive computational or decisional assumption (NIA) generates a challenge NIAChal and private information NIAPriv, and gives NIAChal to an adversary. A non-interactive assumption is said to hold if any PPT adversary's advantage in producing a valid solution NIASol to the challenge is negligible in the security parameter κ. We consider non-interactive assumptions for which there is an efficient verification algorithm Verify that can decide if the adversary's solution is valid.

Definition 7 (Non-Interactive Computational or Decisional Assumption (NIA)). *A* non-interactive assumption $\mathsf{NIA} = (\mathsf{InstGen}, \mathsf{Verify})$ *consists of the following polynomial-time algorithms:*

- $\mathsf{InstGen}(1^\kappa) \rightarrow (\mathsf{NIAChal}, \mathsf{NIAPriv})$*: On input the security parameter, this randomized algorithm outputs a public challenge instance* NIAChal *and private information* NIAPriv.
- $\mathsf{Verify}(\mathsf{NIAChal}, \mathsf{NIAPriv}, \mathsf{NIASol}) \rightarrow 0/1$*: On input the public challenge instance, private information* NIAPriv*, and a solution* NIASol*, this deterministic algorithm returns* 1 *if the solution is valid and* 0 *otherwise.*

Let the advantage of an adversary $\mathcal{A}$ against the non-interactive assumption NIA*, defined by game* $\mathsf{Game}^{\mathsf{NIA}}_{\mathcal{A}}$ *(Figure 3), be as follows:*

$$\mathsf{Adv}^{\mathsf{NIA}}_{\mathcal{A}}(\kappa) = \Pr[\mathsf{Game}^{\mathsf{NIA}}_{\mathcal{A}}(\kappa) = 1]$$

A non-interactive assumption holds if for all PPT adversaries $\mathcal{A}$, $\mathsf{Adv}^{\mathsf{NIA}}_{\mathcal{A}}(\kappa)$ is negligible.

Assumption	NIAChal	NIAPriv	NIASol	Verify
DL	$(\mathcal{G}, g^x)$	$x \leftarrow\!\!\$\ \mathbb{Z}_p$	x'	$x' = x$
CDH	$(\mathcal{G}, g^x, g^y)$	$(x,y) \leftarrow\!\!\$\ \mathbb{Z}_p^2$	Z	$Z = g^{xy}$
CDL	$(\mathcal{G}, f, g^x)$	$x \leftarrow\!\!\$\ \mathbb{Z}_p$	(R, z)	$f(R) \neq 0$ $g^z = R \cdot g^{xf(R)}$
q-SDH	$(\mathcal{BG}_{\mathrm{III}}, \{g^{x^i}\}_{i=1}^q, \hat{g}^x)$	$x \leftarrow\!\!\$\ \mathbb{Z}_p$	(c, Z)	$c \in \mathbb{Z}_p \backslash \{-x\}$ $\wedge\ Z = g^{\frac{1}{x+c}}$
q-BDHI	$(\mathcal{BG}_{\mathrm{I}}, \{g^{x^i}\}_{i=1}^q)$	$x \leftarrow\!\!\$\ \mathbb{Z}_p$	Z	$Z = e(g,g)^{1/x}$
DDH	$(\mathcal{BG}_{\mathrm{III}}, g^r, g^s,$ $g^{(1-b)\cdot u + b \cdot rs})$	$b \leftarrow\!\!\$\ \{0,1\};$ $r, s, u \leftarrow\!\!\$\ \mathbb{Z}_p$	b^*	$b^* = b$

Table 2: Examples of Non-Interactive Computational and Decisional Assumptions (NIA). DL is the Discrete Logarithm assumption. CDH is the Computational Diffie-Hellman assumption. CDL is the Circular DL assumption, which takes an additional parameter function $f : \mathbb{G} \to \mathbb{Z}_p$ as input, and was recently introduced to prove the tight security of Schnorr signatures and Sparkle+ in the ROM only [37]. q-SDH is the q-Strong Diffie-Hellman assumption [24], which underpins the BBS+ signature scheme [3, 25]. q-BDHI is the q-Bilinear Diffie-Hellman Inversion assumption [23, 47]. DDH is the Decisional Diffie-Hellman assumption. $\mathcal{G} = (\mathbb{G}, p, g)$ for cyclic group generator $(\mathbb{G}, p, g) \leftarrow \mathsf{GroupGen}(1^\kappa)$. $\mathcal{BG}_{\mathrm{I}} = (\mathbb{G}, p, g, e)$ for bilinear group generator $(\mathbb{G}, p, g, e) \leftarrow \mathsf{GroupGen}(1^\kappa)$ and similarly for $\mathcal{BG}_{\mathrm{III}} = (\mathbb{G}, \hat{\mathbb{G}}, p, g, \hat{g}, e)$.

Examples of Non-Interactive Assumptions. We give several examples of NIAs in Table 2. These include both elliptic curve and pairing-based assumptions, under which we broadly eliminate the possibility of proving key-unique threshold signatures adaptively secure for $\lfloor t/\ell \rfloor < t_c \leq t$ (Theorems 1 to 3). This list is not exhaustive.

D Definition of Schnorr Signatures

Definition 8 (Schnorr Signatures [81]). *The Schnorr signature scheme is defined with respect to a hash function* H *and consists of efficient algorithms* $(\mathsf{Setup}, \mathsf{KeyGen}, \mathsf{Sign}, \mathsf{Verify})$*, defined as follows:*

- $\mathsf{Setup}(1^\kappa) \to \mathsf{par}$*: On input the security parameter, run* $(\mathbb{G}, p, g) \leftarrow \mathsf{GroupGen}(1^\kappa)$ *and select a hash function* $\mathsf{H} : \{0,1\}^* \to \mathbb{Z}_p$*. Output public parameters* $\mathsf{par} = ((\mathbb{G}, p, g), \mathsf{H})$ *(which are given implicitly as input to all other algorithms).*
- $\mathsf{KeyGen}() \to (\mathsf{PK}, \mathsf{sk})$*: Sample a secret key* $\mathsf{sk} \leftarrow\!\!\$\ \mathbb{Z}_p$ *and compute a public key* $\mathsf{PK} \leftarrow g^{\mathsf{sk}}$*. Output* $(\mathsf{PK}, \mathsf{sk})$*.*
- $\mathsf{Sign}(\mathsf{sk}, m) \to \sigma$*: On input message* m*, the signer samples a nonce* $r \leftarrow\!\!\$\ \mathbb{Z}_p$ *and computes a nonce commitment* $R \leftarrow g^r$*. The signer then computes*

the challenge $c \leftarrow \mathsf{H}(R, \mathsf{PK}, m)$ *and the response* $z \leftarrow r + c\ \mathsf{sk}$. *Output the signature* $\sigma = (R, z)$.

- $\mathsf{Verify}(\mathsf{PK}, m, \sigma) \to 0/1$: *On input the public key* PK, *a message* m, *and a purported signature* $\sigma = (R, z)$, *the verifier computes* $c \leftarrow \mathsf{H}(R, \mathsf{PK}, m)$ *and accepts if* $g^z = R \cdot \mathsf{PK}^c$.

A Schnorr signature is a Sigma protocol zero-knowledge proof of knowledge of the discrete logarithm of the public key, made non-interactive and bound to the message m by the Fiat-Shamir transform [49]. Schnorr signatures are secure under the DL assumption in the ROM [78] and the AGM [55]. They are also tightly secure under the CDL assumption in the ROM [37].

E Additional Details on Static Security

In the static unforgeability game, the challenger accepts as input a security parameter κ, the total number of parties n, a threshold $t+1$, and a corruption threshold t_c.

The challenger generates public parameters par and initializes the adversary with $(\mathsf{par}, n, t+1, t_c)$. The adversary chooses a set of corrupt parties cor, and the challenger sets the honest parties $\mathsf{hon} \leftarrow [n] \setminus \mathsf{cor}$. The challenger then runs KeyGen to derive the threshold public key PK representing all n signers, the individual public key shares $\{\mathsf{PK}_i\}_{i\in[n]}$, and the secret key shares $\{\mathsf{sk}_i\}_{i\in[n]}$. It returns PK, $\{\mathsf{PK}_i\}_{i\in[n]}$, and the set of corrupt signing shares $\{\mathsf{sk}_j\}_{j\in\mathsf{cor}}$ to the adversary.

After key generation has concluded, the adversary can then query signing oracles $\mathcal{O}^{\mathsf{Sign}_1}, \ldots, \mathcal{O}^{\mathsf{Sign}_r}$ for honest signers to engage in subsequent signing rounds, across different sessions, indicated by session identifier sid. The adversary wins if it can produce a valid forgery σ^* with respect to the threshold public key PK on a message m^* that has not been previously queried. We model the adversary as *rushing*, meaning it may wait to produce its outputs after having seen honest protocol messages in each round. The adversary is allowed to be *concurrent*, meaning it may open simultaneous signing sessions at once or choose not to complete a signing session.

F Proofs of Lemmas 1 and 2 and Theorems 2 and 3

Proof (of Lemma 1). For Lemma 1, we must calculate:

$$\Pr[k] = \frac{\#\text{ of ways of choosing } B_1, B_2 \text{ s.t. } |B_1 \cup B_2| = k}{\#\text{ of ways of choosing } B_1, B_2} \tag{2}$$

There are $\binom{n}{k}$ ways to pick a set $U := B_1 \cup B_2$ of size k. We can first pick B_1 arbitrarily from U. There are $\binom{k}{t_1}$ ways to do this. We have fewer degrees of freedom for selecting B_2. We must pick B_2 in such a way that it has t_2 elements and its union with B_1 has exactly k elements. Since $|B_1 \cup B_2| =$

$|B_1| + |B_2| - |B_1 \cap B_2|$ (by the inclusion-exclusion principle for two sets), and we must have that $|B_1 \cup B_2| = k$, $|B_1| = t_1$, and $|B_2| = t_2$, then $|B_1 \cap B_2| = t_1 + t_2 - k$. There are $\binom{t_1}{t_1+t_2-k}$ ways to choose these intersected elements from B_1. Finally, for the denominator, there are $\binom{n}{t_1}$ ways to choose B_1 of size t_1 and $\binom{n}{t_2}$ ways to choose B_2 of size t_2. U must be at least as large as $\max\{t_1, t_2\}$ and cannot exceed n. This gives Eq. (1). □

In the case where B_1 and B_2 have the same maximal cardinality t_c (which maximizes the metareduction's success probability in Theorem 2), it follows that:

$$\Pr[k] = \begin{cases} \frac{\binom{n}{k}\binom{k}{t_c}\binom{t_c}{2t_c-k}}{\binom{n}{t_c}^2}, & \text{if } t_c \leq k \leq \min\{n, 2t_c\} \\ 0, & \text{otherwise} \end{cases} \tag{3}$$

With this lemma in hand, we can now prove Theorem 2.

Proof (of Theorem 2). The metareduction $\mathcal{M}$ works very similarly to the one described in the proof of Theorem 1, with the following differences:

- For each of the two instances of $\mathcal{B}$, $\mathcal{B}_1$ and $\mathcal{B}_2$, $\mathcal{M}$ generates PRFs $F_1 : \{0,1\}^\kappa \times \{0,1\}^{\ell(\kappa)} \to [C]^{t_c}$ and $F_2 : \{0,1\}^\kappa \times \{0,1\}^{\ell(\kappa)} \to [C]^{t_c}$ (i.e., for t_c instead of t).
- The real adversary $\mathcal{A}$ solves for $t+1-t_c$ arbitrary $\mathsf{sk}_{i'}$'s, $i' \notin \mathsf{cor}$, by brute-force search, for $\mathsf{PK}_{i'}$'s.

To compute the probability of $\mathcal{M}$ succeeding, we need to compute: 1) the probability that $\mathsf{cor}_1, \mathsf{cor}_2$ satisfy $|\mathsf{cor}_1 \cup \mathsf{cor}_2| \geq t+1$, and 2) the probability that the oracle $\mathcal{O}^{\mathsf{Corrupt}}(i)$ answers all of the queries made by $\mathcal{M}$ in both instances of $\mathcal{B}$.

1) corresponds to determining the following probability. Let A be a finite set of size n, and let $B_i \subseteq A$ for $i \in \{1,2\}$ be independent, randomly chosen subsets such that $|B_i| = t_c$. We want to compute the probability that the union $B_1 \cup B_2$ is of size at least $t+1$. In Lemma 1, we show that this probability is:

$$\sum_{k=t+1}^{\min\{n,2t_c\}} \frac{\binom{n}{k}\binom{k}{t_c}\binom{t_c}{2t_c-k}}{\binom{n}{t_c}^2} \tag{4}$$

For 2), to determine the probability that the corruption oracle answers all of the queries that $\mathcal{M}$ makes, we need to compute the probability that neither of the instances of $\mathcal{B}$ aborts. To compute this probability, we use the analysis of Theorem 1, showing that we can marginalize out the NIA instance I in:

$$\Pr[\mathcal{M}^{\mathcal{B}} \mid I] \;=\; \Pr[\mathcal{B}_1^{\mathcal{A}_1} \mid I] \left(\Pr[\mathcal{B}_2 \text{ successfully simulates to } \mathcal{M} \mid I] - \frac{\binom{n}{k}\binom{k}{t_c}\binom{t_c}{2t_c-k}}{\binom{n}{t_c}^2} \right)$$

The analysis is the same as in the proof of Theorem 1, which gives the bound in Theorem 2, except that we need to consider the case where the reduction rewinds

more than once. The number of rewinds is bounded by the running time of $\mathcal{B}$, which we denote with τ. This affects the combinatorial argument, as now the reduction has more opportunities to abort the interaction with the adversary (up to τ times); hence up to τ times, the reduction may not answer the corruption queries of made by the adversary (i.e., the metareduction). □

For values of t_c equal to or below $\lfloor t/2 \rfloor$, the metareduction should fail with probability 1. In this case, $2t_c < t + 1 \leq n$, so $\min\{n, 2t_c\} = 2t_c$. Thus,

$$\Pr[\mathsf{BadEvent}^{\dagger}] = \sum_{k=t_c}^{t} \Pr[k] = \sum_{k=t_c}^{2t_c} \Pr[k] + \sum_{k=2t_c+1}^{t} \Pr[k] = 1 + 0 = 1 \tag{5}$$

since $\sum_{k=t_c}^{\min\{n,2t_c\}} \Pr[k] = 1$ and $\Pr[k] = 0$ for all $k \notin [t_c, 2t_c]$ (Eq. (3)). Indeed, the metareduction fails with probability 1.

The probability of the $\mathsf{BadEvent}^{\dagger}$ here should reduce to the probability of the $\mathsf{BadEvent}^{\dagger}$ in Theorem 1 when $t_c = t$. The $\mathsf{BadEvent}^{\dagger}$, where $|B_1| = |B_2| = t_c$ and $B_1 \cup B_2 = t_c$ (i.e., they are identical), occurs with probability:

$$\Pr[\mathsf{BadEvent}^{\dagger}] = \sum_{k=t_c}^{t} \Pr[k] = \Pr[t_c] = \frac{\binom{n}{t_c}\binom{t_c}{t_c}\binom{t_c}{2t_c - t_c}}{\binom{n}{t_c}^2} = \frac{1}{\binom{n}{t_c}} \tag{6}$$

This is indeed the result of Theorem 1. For more intuition on the result of Theorem 2, we provide concrete, worked examples for small numbers of parties in Appendix I.

We now give an alternative (albeit less intuitive, but equivalent) expression for Lemma 1, whose proofs lends itself to more straightforward generalization in Lemma 2.

Proof (of Lemma 1 - Alternative). There are $\binom{n}{k}$ ways to pick a set of size k from a set of size n. Here, for simplicity, we identify the sets B_1 and B_2 with indices in the set $\{1, \ldots, k\}$. Let $\mathcal{I}$ be the set of all pairs (I_1, I_2) such that $I_1 \subseteq \{1, \ldots, k\}, I_2 \subseteq \{1, \ldots, k\}$ and $|I_1| = t_1$ and $|I_2| = t_2$. Then $|\mathcal{I}| = \binom{k}{t_1}\binom{k}{t_2}$.

From $\mathcal{I}$, we want to find only pairs (I_1, I_2) such that $I_1 \cup I_2 = k$. So, let us find all pairs (I_1, I_2) such that $I_1 \cup I_2 < k$ and exclude them. Let A_j be the set of all pairs (I_1, I_2) such that the index $j \in \{1, \ldots, k\}$ is not in either I_1 or I_2. The set of all pairs (I_1, I_2) such that $I_1 \cup I_2 < k$ is then $\mathcal{I} \setminus \bigcup_{j=1}^{k} A_j$. By the inclusion-exclusion principle:

$$|\mathcal{I} \setminus \bigcup_{j=1}^{k} A_j| = \sum_{J \subseteq \{1,\ldots,k\}} (-1)^{|J|} |\bigcap_{j \in J} A_j|$$

Now, I_1, I_2 can be picked freely from $k - |J|$ elements of $\{1, \ldots, k\}$. Let $i = |J|$. Since there are $\binom{k}{i}$ ways to pick an exclusion set of size $|J| = i$ from a set of size

k, and $\binom{k-i}{t_1}, \binom{k-i}{t_2}$ ways to pick I_1 and I_2, respectively, from the remaining $k-i$ indices, we have:

$$\Pr[k] = \begin{cases} \frac{\binom{n}{k}\sum_{i=0}^{k}(-1)^i\binom{k}{i}\binom{k-i}{t_1}\binom{k-i}{t_2}}{\binom{n}{t_1}\binom{n}{t_2}}, & \text{if } \max\{t_1,t_2\} \le k \le \min\{n, t_1+t_2\} \\ 0, & \text{otherwise} \end{cases} \quad (7)$$

□

Note that this is equivalent (among many others) to the expression derived directly in Eq. (1).

We now generalize Lemma 1 in order to prove Theorem 3.

Lemma 2. *Let A be a finite set of size n, and let $B_1 \subseteq A, B_2 \subseteq A, \ldots, B_\ell \subseteq A$ be independent, randomly chosen subsets such that $|B_1| = t_1, |B_2| = t_2, \ldots, |B_\ell| = t_\ell$. Let $\Pr[k]$ denote the probability that the union $B_1 \cup B_2 \cup \cdots \cup B_\ell$ is of size exactly k for $0 \le k \le n$. Then:*

$$\Pr[k] = \begin{cases} \frac{\binom{n}{k}\sum_{i=0}^{k}(-1)^i\binom{k}{i}\prod_{j=1}^{\ell}\binom{k-i}{t_j}}{\prod_{j=1}^{\ell}\binom{n}{t_j}}, & \textit{if } \max\{t_1,t_2,\ldots,t_\ell\} \le k \le \min\{n, \sum_{i=1}^{\ell} t_i\} \\ 0, & \textit{otherwise} \end{cases} \quad (8)$$

Proof (of Lemma 2). For Lemma 2, we must calculate:

$$\Pr[k] = \frac{\#\text{ of ways of choosing } B_1, B_2, \ldots, B_\ell \text{ s.t. } |B_1 \cup B_2 \cup \cdots \cup B_\ell| = k}{\#\text{ of ways of choosing } B_1, B_2, \ldots, B_\ell} \quad (9)$$

There are $\binom{n}{k}$ ways to pick a set of size k from a set of size n. Let $\mathcal{I}$ be the set of all tuples $(I_1, I_2, \ldots, I_\ell)$ such that $I_1 \subseteq \{1,\ldots,k\}, I_2 \subseteq \{1,\ldots,k\}, \ldots, I_\ell \subseteq \{1,\ldots,k\}$ and $|I_1| = t_1, |I_2| = t_2, \ldots, |I_\ell| = t_\ell$. Then $|\mathcal{I}| = \binom{k}{t_1}\binom{k}{t_2}\cdots\binom{k}{t_\ell}$.

From $\mathcal{I}$, we want to find only tuples $(I_1, I_2, \ldots, I_\ell)$ such that $I_1 \cup I_2 \cup \cdots \cup I_\ell = k$. So, let us find all tuples $(I_1, I_2, \ldots, I_\ell)$ such that $I_1 \cup I_2 \cup \cdots \cup I_\ell < k$ and exclude them. Let A_j be the set of all tuples $(I_1, I_2, \ldots, I_\ell)$ such that the index $j \in \{1,\ldots,k\}$ is not in any of the sets $I_1, I_2, \ldots, I_\ell$. The set of all tuples $(I_1, I_2, \ldots, I_\ell)$ such that $I_1 \cup I_2 \cup \cdots \cup I_\ell < k$ is then $\mathcal{I} \setminus \bigcup_{j=1}^{k} A_j$. By the inclusion-exclusion principle:

$$|\mathcal{I} \setminus \bigcup_{j=1}^{k} A_j| = \sum_{J \subseteq \{1,\ldots,k\}} (-1)^{|J|} |\bigcap_{j \in J} A_j|$$

Now, $I_1, I_2, \ldots, I_\ell$ can be picked freely from $k - |J|$ elements of $\{1,\ldots,k\}$. Let $i = |J|$. Since there are $\binom{k}{i}$ ways to pick an exclusion set of size $|J| = i$ from a set of size k, and $\binom{k-i}{t_1}, \binom{k-i}{t_2}, \ldots, \binom{k-i}{t_\ell}$ ways to pick $I_1, I_2, \ldots, I_\ell$, respectively, from the remaining $k-i$ indices, this gives Eq. (8). □

In the case where $I_1, I_2, \ldots, I_\ell$ have the same maximal cardinality t_c (which maximizes the metareduction's success probability in Theorem 3), it follows that:

$$\Pr[k] = \begin{cases} \frac{\binom{n}{k}\sum_{i=0}^{k}(-1)^i\binom{k}{i}\binom{k-i}{t_c}^\ell}{\binom{n}{t_c}^\ell}, & \text{if } t_c \leq k \leq \min\{n, \ell t_c\} \\ 0, & \text{otherwise} \end{cases} \tag{10}$$

We are now ready to prove Theorem 3.

Proof Outline of Theorem 3. The metareduction $\mathcal{M}$ works very similarly to the one described in the proof of Theorem 1, with the following differences:

- $\mathcal{M}$ runs ℓ instances of $\mathcal{B}$ (instead of only two instances as in Theorem 1), which we denote by $(\mathcal{B}_1, \ldots, \mathcal{B}_\ell)$. For each of these instances, $\mathcal{M}$ generates a different PRF $F_i : \{0,1\}^\kappa \times \{0,1\}^{\ell(\kappa)} \to [C]^{t_c}$ for $i \in \{1, \ldots, \ell\}$.
- Each instance of $\mathcal{B}$ is run by $\mathcal{M}$ exactly as described in the proof of Theorem 1 with the only difference being that, in step 6(d), $\mathcal{M}$ picks a corruption set by computing $\mathsf{cor}_i \leftarrow F_i(\rho_{\mathcal{A}}, \mathsf{pp}, \mathsf{keys})$, and in step 6(e), it checks whether $|\mathsf{cor}_1 \cup \cdots \cup \mathsf{cor}_\ell| < t + 1$. If the condition holds, then $\mathcal{M}$ aborts; else, it continues.
- The real adversary $\mathcal{A}$ solves for $t+1-t_c$ arbitrary $\mathsf{sk}_{i'}$'s, $i' \notin \mathsf{cor}$, by brute-force search, for $\mathsf{PK}_{i'}$'s.

To compute the probability of $\mathcal{M}$ succeeding, we need to compute: 1) the probability that $\mathsf{cor}_1, \ldots, \mathsf{cor}_\ell$ satisfy $|\mathsf{cor}_1 \cup \cdots \cup \mathsf{cor}_\ell| \geq t + 1$, and 2) the probability that the oracle $\mathcal{O}^{\mathsf{Corrupt}}(i)$ answers all of the queries made by $\mathcal{M}$ in all of the ℓ instances of $\mathcal{B}$.

1) corresponds to the probability in Lemma 2, namely:

$$\sum_{k=t+1}^{\min\{n,\ell t_c\}} \frac{\binom{n}{k}\sum_{i=0}^{k}(-1)^i\binom{k}{i}\binom{k-i}{t_c}^\ell}{\binom{n}{t_c}^\ell} \tag{11}$$

For 2), to determine the probability that the corruption oracle answers all of the queries that $\mathcal{M}$ makes, we need to compute the probability that none of the instances of $\mathcal{B}$ aborts. To compute this probability, we extend the analysis of Theorem 1, showing that we can marginalize out the NIA instance I:

$$\Pr[\mathcal{M}^{\mathcal{B}} \mid I] = \Pr[\mathcal{B}_1^{\mathcal{A}_1} \mid I]\Bigg(\Pr[\mathcal{B}_2 \text{ successfully simulates to } \mathcal{M} \mid I]\Bigg(\cdots$$
$$\Bigg(\Pr[\mathcal{B}_\ell \text{ successfully simulates to } \mathcal{M} \mid I] - \sum_{k=t_c}^{t} \frac{\binom{n}{k}\sum_{i=0}^{k}(-1)^i\binom{k}{i}\binom{k-i}{t_c}^\ell}{\binom{n}{t_c}^\ell}\Bigg)\Bigg)\Bigg)$$

where the expectation is taken over the $I \leftarrow\!\!\$\ \mathsf{InstGen}$, where $\mathsf{InstGen}$ is a randomized instance generator algorithm, and the final inequality $\mathbb{E}[\Pr[\mathcal{B}^{\mathcal{A}} \mid I]^\ell] \geq \mathbb{E}[\Pr[\mathcal{B}^{\mathcal{A}} \mid I]]^\ell$ still holds for odd powers $\ell \in \mathbb{N}$ due to Jensen's inequality because $\Pr[\mathcal{B}^{\mathcal{A}} \mid I] \geq 0$. This gives the bound in Theorem 3, except that we need to consider the case where the reduction rewinds more than once. The number of

rewinds is bounded by the running time of $\mathcal{B}$, which we denote with τ. This affects the combinatorial argument, as now the reduction has more opportunities to abort the interaction with the adversary (up to τ times); hence up to τ times, the reduction may not answer the corruption queries of made by the adversary (i.e., the metareduction).

G Interactive Computational Assumptions

We recall the definition of an interactive computational assumption. We then review the one-more discrete logarithm (OMDL) assumption, the *algebraic* one-more discrete logarithm (AOMDL) assumption, and the algebraic one-more computational Diffie-Hellman (AOMCDH) assumption. We give impossibility results for these interactive assumptions in Section 6 and Appendix H.

Interactive Computational Assumptions. An interactive computational assumption has the same challenge-solution interface as a non-interactive assumption (NIA) (Appendix C), but allows the adversary to interact with additional oracles. Thus, the adversary may learn additional information beyond its own available resources. As in non-interactive assumptions, we consider interactive assumptions for which there is an efficient verification algorithm Verify that can decide if the adversary's solution is valid.

If, in addition, the oracle algorithm runs in polynomial time, we say that the interactive computational assumption is *falsifiable*. Else, it is a *non-falsifiable* assumption. This distinction is relevant to the impossibility results in Appendix H.

OMDL and AOMDL. OMDL [19] is an interactive computational assumption, with the same instance generation and verification steps as the discrete logarithm assumption. However, OMDL additionally defines a discrete logarithm solution oracle $\mathcal{O}^{\mathsf{dlsol}}$, which accepts an arbitrary group element $X \in \mathbb{G}$, and outputs a solution x such that $X = g^x$. Because X is an arbitrary group element, the challenger is not guaranteed to run in polynomial time. For this reason, we say that OMDL is an unfalsifiable assumption (i.e., despite the verification algorithm Verify running in polynomial time).

AOMDL, on the other hand, additionally requires that the adversary provides an algebraic representation $(\hat{\beta}, \{\beta_i\}_{i=0}^{q})$ of X with respect to all AOMDL challenges $(X_0, \ldots, X_q)$ and the generator g when querying $\mathcal{O}^{\mathsf{dlsol}}$. For this reason, we can guarantee that the challenger runs in polynomial time, so we say that AOMDL is a falsifiable (and therefore weaker) assumption.

Assumption 1 (OMDL Assumption) [19] *Let* $\mathsf{GroupGen}$ *be a group generator that outputs* $(\mathbb{G}, p, g)$*. Let the advantage of an adversary* $\mathcal{A}$ *playing the one-more discrete logarithm game* $\mathsf{Game}_{\mathcal{A}}^{(q+1)\text{-}\mathsf{omdl}}$*, as defined in Figure 4, be as follows:*

$$\mathsf{Adv}_{\mathcal{A}}^{(q+1)\text{-}omdl}(\kappa) = \Pr[\mathsf{Game}_{\mathcal{A}}^{(q+1)\text{-}\mathsf{omdl}}(\kappa) = 1]$$

The one-more discrete logarithm assumption holds for $\mathsf{GroupGen}$ *if for all PPT adversaries* $\mathcal{A}$*,* $\mathsf{Adv}_{\mathcal{A}}^{(q+1)\text{-}omdl}(\kappa)$ *is negligible.*

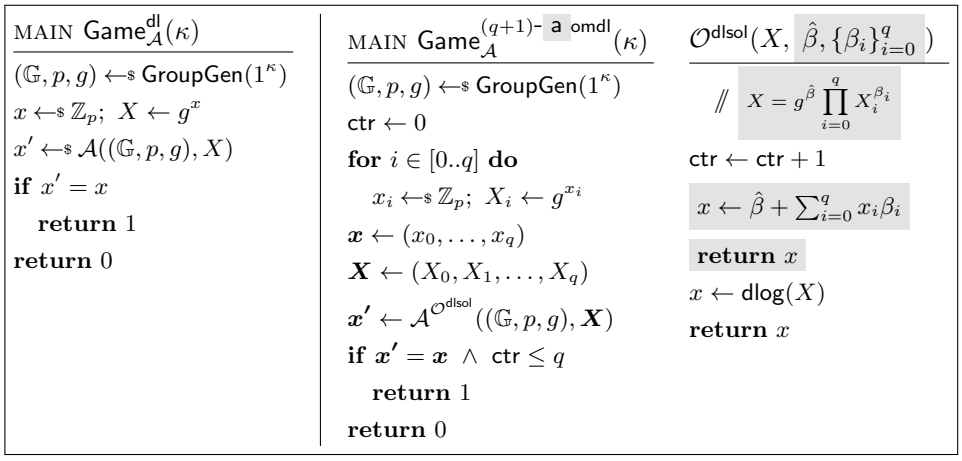

Fig. 4: The Discrete Logarithm (DL), One-More Discrete Logarithm (OMDL), and Algebraic One-More Discrete Logarithm (AOMDL) games. $\mathbb{G}$ is a cyclic group with prime order p and generator g. dlog is an algorithm that finds the discrete logarithm relation of the input X and g. The differences between the OMDL and AOMDL games are highlighted in gray; the key distinction is that in the AOMDL game, the adversary can only query for linear combinations of its challenge group elements. In the OMDL game, the challenger must return the discrete logarithm of an arbitrary group element.

Assumption 2 (AOMDL Assumption) [76] *Let* $\mathsf{GroupGen}$ *be a group generator that outputs* $(\mathbb{G}, p, g)$*. Let the advantage of an adversary* $\mathcal{A}$ *playing the algebraic one-more discrete logarithm game* $\mathsf{Game}^{(q+1)\text{-}\mathsf{aomdl}}_{\mathcal{A}}$*, as defined in Figure 4, be as follows:*

$$\mathsf{Adv}^{(q+1)\text{-}aomdl}_{\mathcal{A}}(\kappa) = \Pr[\mathsf{Game}^{(q+1)\text{-}\mathsf{aomdl}}_{\mathcal{A}}(\kappa) = 1]$$

The algebraic one-more discrete logarithm assumption holds for $\mathsf{GroupGen}$ *if for all PPT adversaries* $\mathcal{A}$*,* $\mathsf{Adv}^{(q+1)\text{-}aomdl}_{\mathcal{A}}(\kappa)$ *is negligible.*

AOMCDH. AOMCDH [10], like AOMDL, consists of challenges of the form $(X_0, \ldots, X_q)$, where $X_i = g^{x_i}$ for all $i \in [0..q]$. However, rather than producing discrete logarithms solutions $(x_0, \ldots, x_q)$, the adversary is asked to compute CDH outputs $(h^{x_0}, \ldots, h^{x_q})$ with respect to the second generator h. Like AOMDL, the adversary is given q-time access to a discrete logarithm solution oracle $\mathcal{O}^{\mathsf{dlsol}}$; however, unlike AOMDL, this oracle is agnostic to the generators, g and h. Note that no non-algebraic version of this assumption exists; the OMCDH assumption in [15] is defined with respect to a CDH oracle, not a DL oracle. While in principle we could define a corresponding OMCDH notion with a DL oracle (base g, for our proof), no threshold schemes have been proven under such an assumption.

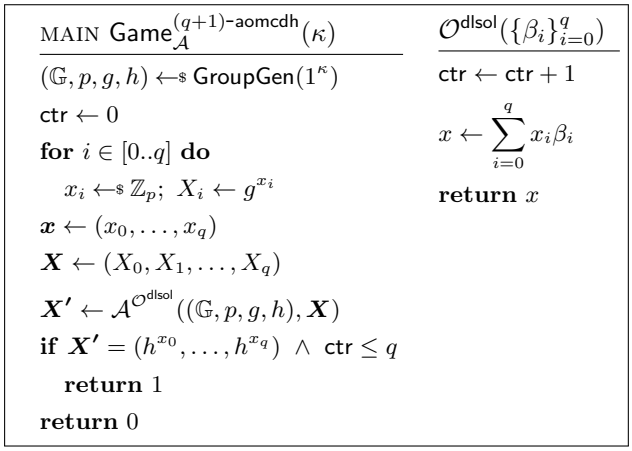

Fig. 5: The Algebraic One-More Computational Diffie-Hellman (AOMCDH) game. $\mathbb{G}$ is a cyclic group with prime order p and generators g and h.

Assumption 3 (AOMCDH Assumption) [10] *Let* GroupGen *be a group generator that outputs* $(\mathbb{G}, p, g, h)$*. Let the advantage of an adversary* $\mathcal{A}$ *playing the algebraic one-more computational Diffie-Hellman game* $\mathsf{Game}_{\mathcal{A}}^{(q+1)\text{-aomcdh}}$*, as defined in Figure 5, be as follows:*

$$\mathit{Adv}_{\mathcal{A}}^{(q+1)\text{-}aomcdh}(\kappa) = \Pr[\mathsf{Game}_{\mathcal{A}}^{(q+1)\text{-aomcdh}}(\kappa) = 1]$$

The algebraic one-more computational Diffie-Hellman assumption holds for GroupGen *if for all PPT adversaries* $\mathcal{A}$*,* $\mathit{Adv}_{\mathcal{A}}^{(q+1)\text{-}aomcdh}(\kappa)$ *is negligible.*

H Extension of Second Impossibility Result to AOMDL and AOMCDH

We now extend the results of Section 6 to the AOMDL and AOMCDH assumptions defined in Appendix G.

Recall from the proof of Theorem 4 that, to produce a forgery in its first execution, $\mathcal{M}$ first sets R^* to be one of its challenges. $\mathcal{M}$ then queries the discrete logarithm solution oracle on $Z^* = R^* \cdot \mathsf{PK}^{c^*}$, where PK is defined by the reduction $\mathcal{B}$. However, in order to make this query in the *AOMDL* game, $\mathcal{M}$ must also provide the representation of Z^* with respect to all of its challenges and the generator g. Since R^* is one of the challenges, $\mathcal{M}$ knows its representation; however, PK is output by $\mathcal{B}$, so $\mathcal{M}$ has no means of deriving its representation directly. As such, we require $\mathcal{B}$ to be *key algebraic*, a notion we now define.

Definition 9. *A* **key-algebraic** *reduction* $\mathcal{B}$ *is one that outputs an algebraic representation* $\boldsymbol{a} = (a_0, a_1, a_2, \ldots)$ *of the public key* $\mathsf{PK} = g^{a_0} \prod X_i^{a_i}$*, where* X_i *are group elements the reduction has seen thus far and* g *is a generator.*

While we could require that $\mathcal{B}$ be fully algebraic, as in [8, 4], we only need $\mathcal{B}$ to output an algebraic representation of PK (as well as group elements it queries to the discrete logarithm solution oracle, already captured by the AOMDL game). Note that due to the requirement of a key-algebraic reduction, our metareduction is not black-box [14], as it assumes an extractor algorithm for algebraic coefficients.

We now state and prove our impossibility result for AOMDL.

Theorem 5. *Theorem 4 holds for a reduction $\mathcal{B}$ against the q-AOMDL assumption, which must additionally be key algebraic, and a metareduction $\mathcal{M}$ against the $(q+1)$-AOMDL assumption.*

Proof. The proof of Theorem 5 follows from the proof of Theorem 4, with the following differences. First, when the reduction $\mathcal{B}$ outputs public keys $(\mathsf{PK}, \{\mathsf{PK}_i\}_{i\in[n]})$ to the adversary $\mathcal{A}$, it additionally outputs an algebraic representation $\boldsymbol{a} = (a, a_0, \ldots, a_{q-1})$ of the public key PK with respect to the generator g and $\mathcal{B}$'s AOMDL challenges $(X_0, \ldots, X_{q-1})$.

Second, when $\mathcal{M}$ queries its DL solution oracle on its forgery $Z^* = R^* \cdot \mathsf{PK}^{c^*}$, $\mathcal{M}$ additionally provides the algebraic representation $\boldsymbol{a'} = c^*\boldsymbol{a}||1$ of Z^*, recalling that $\mathcal{M}$ sets R^* to be its last AOMDL challenge X_q (hence, the exponent on X_q is 1).

Concretely, the differences are as follows.

Responding to $\mathcal{O}^{\mathsf{dlsol}'}$ Queries. $\mathcal{M}$ provides the oracle $\mathcal{O}^{\mathsf{dlsol}'}$ to $\mathcal{B}$, simulating the real discrete logarithm solution oracle $\mathcal{O}^{\mathsf{dlsol}}$. When $\mathcal{B}$ queries $\mathcal{O}^{\mathsf{dlsol}'}(Y, \hat{\beta}, \{\beta_i\}_{i=0}^{q-1})$ on some $Y \in \mathbb{G}$ and its algebraic representation $\hat{\beta}, \{\beta_i\}_{i=0}^{q-1}$ with respect to the group generator g and its AOMDL challenges $(X_0, \ldots, X_{q-1})$, $\mathcal{M}$ does the following:

- If $Y \neq g^{\hat{\beta}} \prod_{i=0}^{q-1} X_i^{\beta_i}$, $\mathcal{M}$ returns $\perp$.
- If $\mathsf{ctr} = q-1$ (i.e., $\mathcal{B}$ has reached its allowed number of DL solution queries), $\mathcal{M}$ returns $\perp$.
- Otherwise, $\mathcal{M}$ increments ctr by one and queries its own DL solution oracle $\mathcal{O}^{\mathsf{dlsol}}$ on input $(Y, \hat{\beta}, \{\beta_i\}_{i=0}^{q})$ with $\beta_q = 0$, receiving $y \leftarrow \mathcal{O}^{\mathsf{dlsol}}(Y, \hat{\beta}, \{\beta_i\}_{i=0}^{q})$. It then updates Q_{odl} with the entry $Q_{\mathsf{odl}} \leftarrow \{((Y, \hat{\beta}, \{\beta_i\}_{i=0}^{q}), y)\}$.
- Finally, $\mathcal{M}$ outputs y to $\mathcal{B}$.

How $\mathcal{M}$ Simulates the Adversary $\mathcal{A}$.

Step 2: $\mathcal{M}$ outputs an empty initial corruption set $\mathsf{cor} = \emptyset$. $\mathcal{B}$ will then output a threshold public key and public key shares for all participants $(\mathsf{PK}, \{\mathsf{PK}_i\}_{i=1}^{n})$ as well as a an algebraic representation $\boldsymbol{a} = (a, a_0, \ldots, a_{q-1})$ for PK. $\mathcal{M}$ checks that $\mathsf{PK} = g^a \prod_{i=0}^{q-1} X_i^{a_i}$ and returns $\perp$ if not.

Step 6: If this is the first time that $\mathcal{B}$ has executed $\mathcal{A}$, $\mathcal{M}$ does the following:

- Computes $Z^* = R^* \cdot \mathsf{PK}^{c^*}$.

– Defines $\boldsymbol{a}' = c^*\boldsymbol{a} \mid\mid 1$. Recall that $|\boldsymbol{a}| = q$, and R^* is the AOMDL challenge X_q.
– Queries $z^* \leftarrow \mathcal{O}^{\mathsf{dlsol}}(Z^*, \boldsymbol{a}')$.
– Stores $((Z^*, \boldsymbol{a}'), z^*)$ in Q_{odl} and (c^*, z^*) in Q_{sig}.

The rest of the proof is the same as the proof of Theorem 4. □

Finally, we show that the same holds for AOMCDH. Note that in the AOMCDH assumption as defined in [10], no generators are included in the $\mathcal{O}^{\mathsf{dlsol}}$ oracle, so key-algebraic here must be with respect to AOMCDH challenges only.

Theorem 6. *Theorem 4 holds for a reduction $\mathcal{B}$ against the q-AOMCDH assumption, which must additionally be key algebraic, and a metareduction $\mathcal{M}$ against the $(q+1)$-AOMCDH assumption.*

Proof. The proof of Theorem 6 follows from the proof of Theorem 4, with the following differences. First, when the reduction $\mathcal{B}$ outputs public keys $(\mathsf{PK}, \{\mathsf{PK}_i\}_{i\in[n]})$ to the adversary $\mathcal{A}$, it additionally outputs an algebraic representation $\boldsymbol{a} = (a_0, \ldots, a_{q-1})$ of the public key PK with respect to $\mathcal{B}$'s AOMCDH challenges $(X_0, \ldots, X_{q-1})$.

Second, when $\mathcal{M}$ queries its DL solution oracle for its forgery $Z^* = R^* \cdot \mathsf{PK}^{c^*}$, $\mathcal{M}$ provides the algebraic representation $\boldsymbol{a}' = c^*\boldsymbol{a}||1$ of Z^*, recalling that $\mathcal{M}$ sets R^* to be its last AOMCDH challenge X_q (hence, the exponent on X_q is 1).

Concretely, the differences are as follows.

Responding to $\mathcal{O}^{\mathsf{dlsol}'}$ Queries. $\mathcal{M}$ provides the oracle $\mathcal{O}^{\mathsf{dlsol}'}$ to $\mathcal{B}$, simulating the real discrete logarithm solution oracle $\mathcal{O}^{\mathsf{dlsol}}$. When $\mathcal{B}$ queries $\mathcal{O}^{\mathsf{dlsol}'}(\{\beta_i\}_{i=0}^{q-1})$ with respect to its AOMCDH challenges $(X_0, \ldots, X_{q-1})$, $\mathcal{M}$ does the following:

– If $\mathsf{ctr} = q-1$ (i.e., $\mathcal{B}$ has reached its allowed number of DL solution queries), $\mathcal{M}$ returns $\perp$.
– Otherwise, $\mathcal{M}$ increments ctr by one and queries its own DL solution oracle $\mathcal{O}^{\mathsf{dlsol}}$ on input $(\{\beta_i\}_{i=0}^{q})$ with $\beta_q = 0$, receiving $y \leftarrow \mathcal{O}^{\mathsf{dlsol}}(\{\beta_i\}_{i=0}^{q})$. It then updates Q_{odl} with the entry $Q_{\mathsf{odl}} \leftarrow \{(\{\beta_i\}_{i=0}^{q}, y)\}$.
– Finally, $\mathcal{M}$ outputs y to $\mathcal{B}$.

How $\mathcal{M}$ Simulates the Adversary $\mathcal{A}$.

Step 2: $\mathcal{M}$ outputs an empty initial corruption set $\mathsf{cor} = \emptyset$. $\mathcal{B}$ will then output a threshold public key and public key shares for all participants $(\mathsf{PK}, \{\mathsf{PK}_i\}_{i=1}^{n})$ as well as a an algebraic representation $\boldsymbol{a} = (a_0, \ldots, a_{q-1})$ for PK. $\mathcal{M}$ checks that $\mathsf{PK} = \prod_{i=0}^{q-1} X_i^{a_i}$ and returns $\perp$ if not.

Step 6: If this is the first time that $\mathcal{B}$ has executed $\mathcal{A}$, $\mathcal{M}$ does the following:

– Computes $Z^* = R^* \cdot \mathsf{PK}^{c^*}$.
– Defines $\boldsymbol{a}' = c^*\boldsymbol{a} \mid\mid 1$. Recall that $|\boldsymbol{a}| = q$, and R^* is the AOMCDH challenge X_q.

– Queries $z^* \leftarrow \mathcal{O}^{\mathsf{dlsol}}(\boldsymbol{a}')$.
– Stores $(\boldsymbol{a}', z^*)$ in Q_{odl} and (c^*, z^*) in Q_{sig}.

The rest of the proof is the same as the proof of Theorem 4, except once $\mathcal{M}$ has obtained x_q, it computes its last CDH challenge as h^{x_q}. □

I Theorem 2 Examples

Here we provide concrete, worked examples to further illustrate the result of Theorem 2 for small numbers of parties.

Example 1. Suppose $n = 6$ and $A = \{1, 2, 3, 4, 5, 6\}$. Let the threshold be $t+1 = 5$. Let $B_1 \subseteq A, B_2 \subseteq A$ be independent, randomly chosen subsets such that $|B_1| = |B_2| = t_c = 3$. (In the metareduction for Theorem 2, $B_1 = \mathsf{cor}_1, B_2 = \mathsf{cor}_2$, and t_c is the corruption threshold.) There are $\binom{n}{t+1} = \binom{6}{5} = 6$ ways to pick the set $U := B_1 \cup B_2$ of size $t + 1$:

$$U = \{1, 2, 3, 4, 5\}, \{1, 2, 3, 4, 6\}, \{1, 2, 3, 5, 6\}, \{1, 2, 4, 5, 6\}, \{1, 3, 4, 5, 6\}, \{2, 3, 4, 5, 6\}$$

Say $U = \{1, 2, 4, 5, 6\}$. There are $\binom{t+1}{t_c} = \binom{5}{3} = 10$ ways to pick the set B_1, which is a subset of U. Say $B_1 = \{1, 2, 5\}$. Then, we must pick elements of $\{1, 2, 4, 5, 6\}$ such that B_2 has 3 elements and $B_1 \cup B_2 = U$. There are $\binom{t_c}{2t_c-(t+1)} = \binom{3}{6-5} = 3$ ways to pick B_2:

$$B_2 = \{1, 4, 6\}, \{2, 4, 6\}, \{5, 4, 6\}$$

$\Pr[t + 1]$ is the probability that $B_1 \cup B_2$ has exactly $t + 1$ elements.

$$\Pr[t+1] = \frac{\#\text{ of ways of choosing } B_1, B_2 \text{ s.t. } U \text{ has size } t+1}{\#\text{ of ways of choosing } B_1, B_2} \tag{12}$$

There are $\binom{n}{t_c} = \binom{6}{3} = 20$ ways to choose uniformly random B_1, B_2 of size t_c. Thus, we have:

$$\Pr[5] = \frac{\binom{6}{5}\binom{5}{3}\binom{3}{6-5}}{\binom{6}{3}^2} = \frac{9}{20} \tag{13}$$

(Note that $3 \leq 5 \leq \min\{6, 6\}$.) Thus, the chance that two randomly chosen sets of size 3 have a union of exactly size 5 is 45%.

Now, for the purposes of our metareduction, $|B_1 \cup B_2| = t + 1 = 5$ certainly suffices, but the metareduction also succeeds as long as $|B_1 \cup B_2| \geq t + 1$. Thus, in our example, $k = 6$ also suffices. (Since $n = 6$, this is the only other case to consider.) We have:

$$\Pr[6] = \frac{\binom{6}{6}\binom{6}{3}\binom{3}{0}}{\binom{6}{3}^2} = \frac{1}{20} \tag{14}$$

(Note that $3 \leq 6 \leq \min\{6, 6\}$.) Thus, the chance that two randomly chosen sets of size 3 have a union of exactly size 6 is 5%.

Putting it all together, the probability the metareduction succeeds in obtaining enough keys when $n = 6$, $t + 1 = 5$, and $t_c = 3$ is $\Pr[5] + \Pr[6] = \frac{9}{20} + \frac{1}{20}$, which is 50%.

Example 2. Our first impossibility result (Theorem 1) is for full adaptive security, corresponding to $t + 1 = 4$ instead of 5 (as above) for $t_c = 3$.

Cases $3 \leq k \leq 6$:

1. For $n = 6, k = 3, t_c = 3$, $2t_c - k = 3$, so $\binom{t_c}{2t_c-k} = \binom{3}{3}$. The probability that two randomly chosen sets of size 3 have a union of exactly size 3 is $\frac{1}{20}$. Note that $k = t_c$ means B_1 and B_2 are identical.
2. For $n = 6, k = 4, t_c = 3$, $2t_c - k = 2$, so $\binom{t_c}{2t_c-k} = \binom{3}{2}$. The probability that two randomly chosen sets of size 3 have a union of exactly size 4 is $\frac{9}{20}$.
3. We have calculated $n = 6, k = 5, t_c = 3$ above. The probability is $\frac{9}{20}$.
4. We have calculated $n = 6, k = 6, t_c = 3$ above. The probability is $\frac{1}{20}$.

Thus, the metareduction succeeds in obtaining enough keys with probability $\Pr[4] + \Pr[5] + \Pr[6] = \frac{9}{20} + \frac{9}{20} + \frac{1}{20} = 95\%$.